Multi-Agent Systems for Software Development: a Multi-Faceted Research Question-driven Reference Guide

Sandra Mitrović[a,*], Vincenzo Giuffrida[a], Fabio Barth[b], Alba Bonet-Jover[c], Lidia López[d], Shaibal Barua[e], Santiago Massé[f], Estela Saquete[c], Aitana Delgado[f], Mobyen Uddin Ahmed[e], Shahina Begum[e], Georg Rehm[b] and Matteo Salani[a]

[a]SUPSI – IDSIA, Campus Est, Via la Santa 1, Lugano-Viganello, 6962, Switzerland
[b]Deutsches Forschungszentrum für Künstliche Intelligenz GmbH (DFKI), Salzufer 15/16, Berlin, 10587, Germany
[c]Department of Software and Computing Systems, University of Alicante (UA), carretera San Vicente s/n, Alicante, 03690, Spain
[d]Universitat Politècnica de Catalunya (UPC-BarcelonaTech), Carrer Jordi Girona 1-3, Barcelona, 08034, Spain
[e]Department of Computer Science and Engineering, Mälardalen University, Universitetsplan 1, Västerås, 722 20, Vastmanland, Sweden
[f]IDENER.AI, Torre Sevilla, 11th Floor. C/ Gonzalo Jimenez de Quesada, 2, Seville, 41092, Spain

ARTICLE INFO

Keywords:
Multi-Agent Systems
Large Language Models
Software Development Lifecycle
Autonomous Agents
Agentic Frameworks
LLM-Based Agents
Code Generation

ABSTRACT

Multi-agent systems (MAS) based on large language models (LLMs) are on the rise due to their improved capabilities compared to single agents. In particular, among the emerging MAS for software development, those establishing automated agentic workflows contribute to the idea that software development is an essentially solved task. This study aims at critically assessing five MAS for software development that cover multiple phases and support automated workflow: MetaGPT, ChatDev, AgileCoder, HyperAgent and AgentLite. Here, 14 research questions are identified, specifically referring to architectural-, technical-, operational-, human interaction- and responsible AI-aspects. The findings from the study can help interested researchers and practitioners to understand which design choices bring advantages for particular aspects and where, on the contrary, existing solutions are suboptimal and what could be suggestions for addressing the gaps. Due to its research question-oriented layout, this paper is ideal as a reference which, for a particular aspect, facilitates grasping the overview of main design choices, capabilities, limitation and reconciliation approaches.

1. Introduction

Multi-agent systems (MAS) have become a foundational paradigm in distributed artificial intelligence (AI) due to their capability to coordinate autonomous entities collaboratively to work on and eventually successfully solve complex problems. MAS have demonstrated emerging and widespread applicability across diverse domains such as robotics, traffic management, healthcare, energy systems, and smart logistics, where autonomous agents are able to collaboratively achieve goals that are difficult for single systems to handle alone (Stone and Veloso, 2000).

With regard to the software development lifecycle (SDLC), MAS have been explored to manage complex workflows, with agents orchestrating tasks ranging from requirements analysis to testing and deployment. Several frameworks have been proposed to support SDLC, including AgentLite (Liu et al., 2024b), AgileCoder (Nguyen et al., 2025), ChatDev (Qian et al., 2024), HyperAgent (Phan et al., 2024), MetaGPT (Hong et al., 2023), AutoGen (Wu et al.), CAMEL (Li et al., 2023), and AgentCoder (Huang et al., 2023).

In this paper a comprehensive and critical examination is conducted on how MAS implement different phases of the SDLC and analyze the assumptions on which their foundations and architecture workflows have been built.

*Corresponding author
sandra.mitrovic@supsi.ch (S. Mitrović); vincenzo.giuffrida@supsi.ch (V. Giuffrida); fabio.barth@dfki.de (F. Barth); alba.bonet@dlsi.ua.es (A. Bonet-Jover); lidia.lopez@upc.edu (L. López); shaibal.barua@mdu.se (S. Barua); santiago.masse@idener.ai (S. Massé); stela@dlsi.ua.es (E. Saquete); aitana.delgado@idener.ai (A. Delgado); mobyen.uddin.ahmed@mdu.se (M.U. Ahmed); shahina.begum@mdu.se (S. Begum); georg.rehm@dfki.de (G. Rehm); matteo.salani@supsi.ch (M. Salani)
ORCID(s): 0000-0002-5697-5865 (S. Mitrović); 0009-0009-4627-7391 (V. Giuffrida); 0009-0004-7950-1672 (F. Barth); 0000-0002-7172-0094 (A. Bonet-Jover); 0000-0002-6901-9223 (L. López); 0000-0003-0730-4405 (S. Barua); 0000-0002-6001-5461 (E. Saquete); 0000-0003-1953-6086 (M.U. Ahmed); 0000-0002-1212-7637 (S. Begum); 0000-0002-7800-1893 (G. Rehm); 0000-0003-2809-4347 (M. Salani)

Preprint not peer reviewed
This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

In particular, this work focuses on frameworks that (i) have been designed explicitly for supporting the SDLC, and that (ii) implement autonomous agents covering multiple SDLC phases beyond *coding*. Based on these two criteria, AgentLite, AgileCoder, ChatDev, HyperAgent and MetaGPT are selected for in-depth analyses as they support end-to-end or multi-phase SDLC. On the other hand, AutoGen and CAMEL are out of scope as they are general-purpose MAS encapsulated in the frameworks HyperAgent, ChatDev and AgileCoder that are, in fact, analysed (see above). AgentCoder is excluded, too, as it targets the coding and testing phases, without providing support for other SDLC phases.

Despite the recent proliferation of MAS for SDLC, the literature currently focuses on presenting new frameworks and variants rather than analyzing and comparing their capabilities, limitations, and design trade-offs.

The general idea of having agents which are capable of implementing an entire software development project on their own or with minimal user intervention, contributes to the hype that coding has been resolved as a generative task. With this study we would like to perform a targeted overview underpinned by a critical assessment of these frameworks.

This study provides a multi-faceted assessment of these frameworks, examining *architectural, technological, operational, human interaction and responsible AI-related* aspects each organized in a dedicated section. Within each of these aspects, we propose several research questions (RQs) and address them providing:

- Contextualization of the RQ: providing motivation and, if necessary, definitions of the terminology used;
- Research Question-Focused Overview: positioning each of the considered frameworks with a strict focus on this concrete RQ;
- Comparison: comparing frameworks among themselves;
- Identified Gaps and Potential Improvements: a critical reflection emphasizing found gaps and indicating possible improvements.

The contribution of this study consists of a structured and critical analysis of key aspects related to multi-agent based software development as implemented in the selected frameworks. These analyses often go beyond the claims made in the original papers and, where necessary and possible, the analysis is supported by a careful examination of the associated code repositories.

Due to its transparent and research question-oriented organization, this study allows interested researchers and practitioners to use it as a reference and quickly to grasp the key design choices, advantages, disadvantages, and possible improvements of a particular aspect of interest.

2. Architectural aspects

2.1. SDLC models and workflows

Q1. *Which phases of the SDLC does the framework cover? What roles are defined within each phase, and how these roles are mapped to agents, for example, is there a single agent per role or are roles shared among multiple agents? Additionally, which SDLC models (e.g., waterfall, agile) does the framework support?*

Contextualization. The SDLC is a structured process that defines the phases, roles, and workflows involved in planning, creating, testing, deploying, and maintaining software systems. The most common SDLC are the Waterfall model, that follows a linear, sequential progression through distinct phases, with each stage completed in full before the next begins; and the Agile model, which adopts an iterative, incremental approach that delivers functional software in short cycles, enabling continuous feedback and adaptation. DevOps integrates development and operations practices to achieve continuous integration, delivery, and deployment through automation and close cross-functional collaboration.

Overview. The investigated multi-agent frameworks differ in the extent of SDLC coverage, the roles they define, their role-to-agent mapping strategies, the SDLC model they emulate (e.g., Waterfall, Agile), and the DevOps operations they cover. Most frameworks define roles explicitly, often using a one-to-one mapping between roles and agents, and operate in modular, phase-aligned workflows.

MetaGPT covers the full SDLC using a waterfall-like sequential model. It defines roles such as Product Manager, Architect, Project Manager, Engineer, and QA Engineer. Each role is instantiated as a separate agent with its own goal,

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

constraints, and behaviours. The framework uses a one-to-one mapping, where every agent acts in a single role, with communication structured according to Standard Operating Procedures (SOPs) and reinforced by a shared message environment.

Similarly, ChatDev adheres to a waterfall model and emulates a software company where each phase, from requirements analysis, system design, coding, and testing, is handled by dual-agent interactions. Roles include CEO, CTO, Programmer, Reviewer, and Tester. A typical structure involves hierarchical instructor-assistant agent pairs. While there is some reuse of roles across phases, each role is still mapped to a distinct agent instance per phase context.

In contrast, AgileCoder follows the Agile methodology, organising work into sprints with Planning, Development, Testing, and Review phases. Roles include Product Manager, Scrum Master, Developer, Senior Developer, and Tester. Each role corresponds to a dedicated agent, supporting a single-agent-per-role strategy, and, like ChatDev, it adopts a dual-agent interaction per phase. Agents collaborate iteratively across sprints, inheriting context and outputs for continuous refinement.

HyperAgent supports full SDLC automation, particularly targeting scalable and parallel task execution. It is not tightly bound to a specific SDLC model, but its structure is most compatible with task decomposition found in DevOps or hybrid workflows. It features a Planner agent, acting in a master role, and subordinate expert agents, where roles are implemented as specialised agents coordinated by the master. Role-to-agent mapping follows a hierarchical master-worker pattern.

Lastly, AgentLite allows custom SDLC implementations. In the example based on a DevOps SDLC, phases like Planning are decomposed into sub-tasks, such as requirements elicitation, backlog creation, and sprint planning. Roles need to be explicitly modeled and each role should represent a separate LLM-agent. The framework promotes one-to-one mapping between roles and agents but allows for flexible assignment. HyperAgent proposes a different approach more suitable for real case scenarios, assisting developers in integrating new functionalities in existing codebases, interacting with the repository.

Comparison. Most of the frameworks adopt a Waterfall model, while AgileCoder emulates an Agile SDLC. Although the different approaches could have their advantages and disadvantages, it is not clear which solution is more effective for autonomous software development and specific use cases. The Waterfall model provides a highly traceable workflow in which each phase is executed sequentially and uses the artefacts generated by the previous phases. The Agile methodology could introduce more complexity and costs, since the development is executed iteratively in sprints, but it could help to improve the final outcome with continuous refinement and more granular tasks decomposition. There are no significant differences regarding the SDLC phases covered by the frameworks (Figure 1). All of them perform the Planning phase, with MetaGPT and ChatDev also producing design artefacts before starting the coding phase. Coding, Reviewing and Testing are covered by most frameworks, with the exception of AgileCoder which does not include the code review phase.

Identified Gaps and Potential Improvements. The examined multi-agent frameworks demonstrate solid performance across the early and middle phases of the SDLC, particularly in planning, design, implementation, and testing. However, they provide little support for the later stages, especially deployment and maintenance. These phases are essential for real-world software delivery, encompassing continuous integration and continuous deployment CI/CD, system monitoring, rollback strategies, and long-term maintainability. Other Agile SDLCs, such as Extreme Programming (XP) and Kanban, are currently neglected by examined frameworks. Additionally, none of the considered frameworks support multiple SDLCs.

A limiting factor of the examined frameworks lies in context handling and scalability. Most frameworks rely on token-limited prompts and local working memory, which hampers their ability to reason over evolving codebases, logs, and test results in deployment settings. While AgileCoder's dynamic code graph marks progress in this direction, robust mechanisms for large-scale context grounding remain underdeveloped.

To progress toward full SDLC automation, future frameworks should explicitly extend their architecture to support deployment and maintenance workflows. This includes integrating with CI/CD pipelines, supporting test coverage monitoring, generating deployment manifests or configuration files, and implementing strategies for regression detection, rollback, and system health checks.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

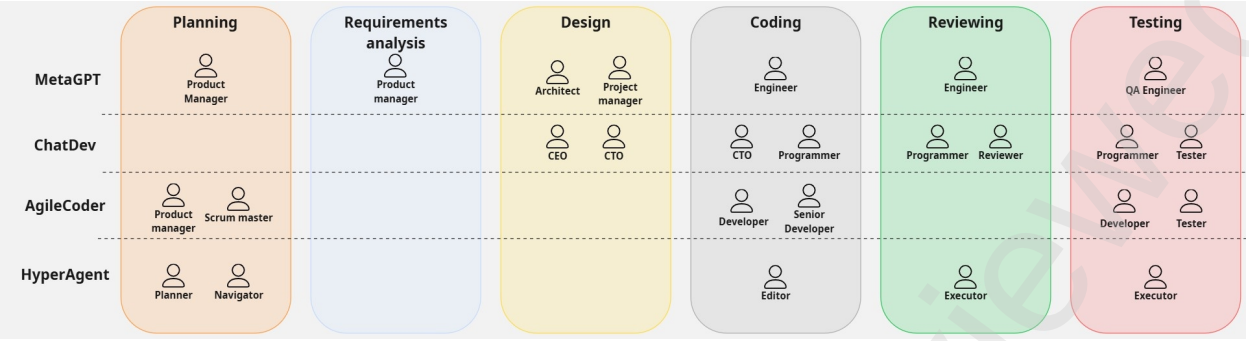


Figure 1: Phases and respective agent roles in SDLC in considered frameworks.

2.2. Agent architecture and communication

Q2. *What communication paradigm is used between agents, such as collaborative, competitive, or hybrid? Which interaction structure is implemented, such as hierarchical, decentralized, centralized, shared memory pool, or another architecture?*

Contextualization. In order to understand how the intra-agent communication works in different SDLC frameworks, we resort to the three aspects commonly considered when discussing interaction mechanisms in the literature on multi-agent frameworks.

The first aspect pertains to the communication paradigm (Guo et al., 2024b), which is commonly classified into three categories: competition, cooperation, and coopetition. In competition, as the name suggests, agents compete with each other. The competition can be oriented towards conflicting objectives or limited resources. In collaboration and coopetition, agents tend to have shared objectives, but the way they achieve them is different. In the former, they work together and exchange the necessary information, while in the latter, they rather compete. Hence, the coopetition is a mix of collaboration and competition, sometimes also referred to as a debate (Guo et al., 2024b), given that the agents argue on the best solution, each one defending its own standpoint before reaching a consensus.

The second aspect pertains to the interaction structure, that is, the topology of the agent communication network, which might be centralized, decentralized and layered. In the centralized architecture, one agent orchestrates the whole framework and communicates with all other agents, who do not communicate among themselves. In the decentralized architecture, communication is organized in peer-to-peer fashion, where every agent can potentially communicate with any other. In the layered architecture, agents are organized in different layers and the communication primarily takes place between agents of adjacent layers. Hierarchical structure is a special type of layered structure where higher-level layers control or manage lower-level layers.

Finally, the third aspect refers to any additional interaction mechanism that complements the interaction structure by enabling information exchange among the agents. These could be, for example, shared message pools accessible by all agents, or dedicated message queues and message streams for exchanging information between two particular agents.

Overview. All selected frameworks support several phases of SDLC are unanimously based on the collaborative agent setup. However, the underlying interaction structure varies due to the different supported SDLC phases. As such, AgentLite follows a hierarchical structure with the Manager agent assigning tasks to either individual working agents or teams of other agents. HyperAgent has a centralized interaction structure characterized by a master agent (Planner) coordinating specialized child agents. MetaGPT adheres to the layered architecture in which the sequence of layers reflects the sequence of defined subtasks, along with the corresponding roles. Agents in adjacent layers operate on equal footing, that is, they are equally important, without hierarchical relationships. Instead, ChatDev follows a layered architecture where each dual-agent interaction is hierarchically organized, using an instructor-assistant relationship. AgileCoder is similar to ChatDev, but with one main difference: the workflow in ChatDev is not fully aligned with

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

assigned roles. For example, some roles, such as Programmer, are reused in dual-agent interactions across different phases of the process.

Different frameworks use different mechanisms to exchange information between agents. Most of the frameworks use a simple queue interaction mechanism to enable agent interactions. In AgentLite, the Managing agent and Subordinate agent communicate directly via so-called Task Packages, which represent a synchronous method of communication, to let Subordinate agents to wait for feedback. Alternatively, in AgileCoder, a Message Stream acts as a working memory between two agents; in ChatDev, dual-agent communication is part of a so-called Chat Chain. In HyperAgent, asynchronous Message Queues are used to communicate between the Master agent and its Subordinate agents. An asynchronous queue has its advantages, as it decouples communication from time and allows the master agent to proceed with its activities without waiting for the Subordinate agents to process the message. Contrary to these, MetaGPT uses a global shared message pool with a publish-subscribe mechanism. This mechanism allows for each agent to share its results with other agents by publishing them to the shared message pool, as well as to access messages shared by other agents by subscribing to them. This global shared message pool could, in theory, allow for a fully peer-to-peer interaction solution, hence depending on a specific use case needs, MetaGPT could turn into a fully decentralized framework.

Comparison. The above-mentioned interaction choices have different advantages and disadvantages. With respect to communication overhead, dual-agent communication is resource-demanding. In such scenarios, having predefined interactions between different agent roles, such as in AgileCoder and ChatDev, is beneficial but leads to a reduction in flexibility and major changes if there is a need to intervene on agent roles or interactions. MetaGPT, on the other hand, allows for peer-to-peer interactions and, at the same time, thanks to the global shared message pool that is accessible to all agents, reduces communication overhead. Such an architecture is also suitable for the adaptations in the communication workflow. However, neither MetaGPT nor AgileCoder/ChatDev readily enable parallel processing of subtasks and dynamic load balancing, which HyperAgent and AgentLite instead support.

The centralized and hierarchical structure permits a higher scalability compared to other types of communication paradigms. Both architectures allow independent scaling of agents with the manager agents supported by powerful models with superior reasoning capabilities, while other agents are run by smaller models. The usage of an asynchronous Message Queue enables HyperAgent to distribute tasks among several agents and resources and to readily parallelize tasks, unlike other frameworks. AgentLite demonstrates high efficiency and scalability through its hierarchical communication paradigm.

Identified Gaps and Potential Improvements. Although the various frameworks explore effective communication structures that can offer several advantages, a current gap is the lack of a standard for inter-agent communication. Emerging communication protocols between agents and external tools, such as Agent-to-Agent communication protocol (A2A)[1] and Model Context Protocol (MCP)[2], could be integrated to make these frameworks more flexible and easily customizable, while also facilitating communication with external components or other multi-agent frameworks. While research on MAS integrated with A2A and MCP is already in progress (Liao et al., 2025), SDLC-focused MAS could benefit from these protocols to enhance integration with external services. In particular, A2A could support interoperability with external agents and services, whereas MCP could enable easier and more standardized tool integration.

3. Technological aspects

3.1. Framework definition

Q3. *How do agents interact in terms of communication style, such as natural language or pseudo-machine language? What types of input and output data are supported by the framework, including text-only content, code, images, and audio?*

Contextualization. MAS inherently rely on structured information exchange to perform tasks and ensure coherent collaboration. As each agent in MAS operates autonomously with a specialized role, the format and structure of the data

[1] https://a2a-protocol.org/latest/
[2] https://modelcontextprotocol.io

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

are fundamental to advance the workflow. LLMs enable agents to process and generate various input and output data formats beyond natural language, including images and code. Although most frameworks supporting SDLC primarily rely on text and code, they differ in the type of data processed or generated, depending on the SDLC phase and the agent involved. The choice of specific input and output formats, therefore, influences the development workflow and affects the framework coordination efficiency.

Overview. Each MAS specialized in SDLC proposes different solutions, highlighting the advantages of specific design choices. For instance, ChatDev and AgileCoder rely on a communication style based on natural language. Within each phase, two agents engage in a multi-turn dialog and work cooperatively until they achieve consensus, producing either text or code depending on the SDLC phase.

MetaGPT distinguishes itself by the use of SOPs (Standardized Operating Procedures) as the main data exchange format between agents. SOPs can contain text as well as other kinds of data, such as charts, graphs and code. Structured outputs generated by agents at different phases of the SDLC are passed as input, along with natural language text used to clarify roles and task specifications. Diagrams in MetaGPT are represented using a text-based graphical format. The Product Manager agent, for example, produces a requirements document (PRD) containing goals, user stories, and requirements, as well as a competitive quadrant chart. The Architect agent processes the PRD documents and generates system architecture diagrams and code interface definitions. Code is generated by the Engineer agent and processed by the QA Engineer, who produces additional code for unit tests.

HyperAgent uses natural language but in a more structured format, consisting of two fields: Context (containing background and rationale) and Request (containing actionable instructions). The Planner agent generates strategies and tasks for other agents based on user prompts. The Navigator agent interacts with IDEs and codebases to extract the necessary code and information. Finally, the Editor agent produces the code and integrates it into existing codebase.

As a general-purpose multi-agent framework, AgentLite relies more on natural language than on code. The Individual Agent composes a prompt based on user input, which is used by the LLM to generate an executable action. The Manager Agent employs a more structured message format, called Task Package, which is used to assign sub-tasks to Individual Agents.

All considered frameworks primarily support textual data as both input and output. In the context of inter-agent communication, the textual format may be more structured, as in the case of MetaGPT and AgentLite, while code is the primary data format used during the development phase of the SDLC.

Comparison. With respect to communication type, MetaGPT differs from the other considered frameworks in that its agents rely not only on natural language but also on various types of documents to interact with one another. The use of SOPs aims to reduce ambiguities and errors in agent communication; however, the absence of an ablation study on the use of structured output leaves unclear what benefits such data formats provide in the communication process between agents. In particular, some of the artifacts produced may be of limited usefulness for specific use cases, as, for example, the competitive analysis diagram. All frameworks use code as a data format, which is the output of the development phase and the input for the code review and testing phases. HyperAgent differs from other frameworks by primarily focusing on integrating code into existing codebases rather than generating new files.

Identified Gaps and Potential Improvements. Future studies could further investigate which types of graphical and structured data might be useful to support the software development in the context of MAS for the SDLC.

The use of structured data provided through human-in-the-loop interactions is another aspect that remains largely unexplored. This could enable SDLC frameworks to become better integrated tools within complex, real-world development processes and support the generation of higher-quality code.

Moreover, interaction with development tools such as IDEs and CI/CD systems, both to generate data that can be used as input by agents and to integrate agents' output into these systems, remains only partially explored. In real-world scenarios, frameworks supporting software development should interact with the tools used by developers and ensure that the generated code integrates into complex codebases.

3.2. Agent definition and functioning

Q4. *What LLM are used as backbones to support agents' capabilities across different systems (e.g., open source or proprietary, number of parameters)? Are these fine-tuned or pre-trained?*

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

Contextualization. LLMs can be classified as general-purpose or domain-specific. General-purpose LLMs have been trained on massive, diverse datasets providing support to a wide range of tasks (e.g., coding, reasoning, language understanding, summarization). Examples of these include GPT (Brown et al., 2020a), Llama (Grattafiori et al., 2024), Claude (Anthropic), Mistral (Jiang et al., 2023), and Gemini (Team et al., 2025) families. Domain-specific LLMs can be divided into two subgroups: models that are fine-tuned or adapted from general-purpose foundational models, and models that are trained from scratch. Among the first group, for the software development domain, we find Gemini Code Assist[3], Codex[4], Code LLaMA[5] and DeepSeek-Coder[6] trained for coding based on Gemini, GPT-3, LLaMA-2 and DeepSeek, respectively. A smaller number of code-focused LLMs have been trained from scratch, directly on code repositories without relying on a general LLM. These models are specifically designed and optimized for source code understanding and generation, for example, CodeGen[7] is a family of models from 350M to 16B parameters for program synthesis for Python; StarCode models[8] are 15.5B parameter models trained on 80+ programming languages from The Stack dataset[9] (v1.2); and, WizardCoder derived from StarCoder and CodeLlama, among others.

In relation to the availability of the models, there are proprietary models that are not available for download but can be used through an API, such as GPT, Claude, and Gemini. Their internal architectures, training datasets, and fine-tuning details remain undisclosed. In contrast, the so-called "open" models are available for download, for example, GPT-Neo, LLaMA, or Mistral, and these can be classified as open-source or open-weight. Open-source models (e.g., GPT-Neo) provide both the model and the necessary training details, enabling full reproducibility. With open-weight models, instead, the model is available to download (and use), but there is limited information related to the model creation (e.g., LLaMA and Mistral). The latter occasionally come with some use restrictions (e.g., non-commercial licenses). From the code-related LLMs mentioned above, Gemini Code Assist and Codex are proprietary (based on Gemini and GPT-3), the rest are open-weight (Code LLaMA3, DeepSeek-Coder, CodeGen, StarCode, and WizardCoder).

Overview. All the reviewed frameworks allow to configure several LLMs as a backbone for their agents. For AgentLite and HyperAgent, the backbone LLM is configured for each Agent, i.e., several LLMs can be used simultaneously in different agents. All the frameworks can be configured using several GPT versions from OpenAI. The GPT family provides proprietary models that are publicly accessed through an API. AgentLite, AgileCoder and HyperAgent can also use the proprietary models Claude 3, from Anthropic, also available through API. HyperAgent can use the proprietary Spark Lite model, a lightweight, general-purpose version of Spark from iFlytek. AgentLite, HyperAgent and MetaGPT can use open models (i.e., open-source or open-weight); AgentLite uses its own xLAM model (Zhang et al., 2025), HyperAgent can use LLaMA, an open-weight model from Meta, which can be downloaded, but the license comes with some use restrictions; and MetaGPT can be used with DeepSeek Coder 33B from DeepSeek AI. xLAM (advanced Large Action Model) has been trained on agent action trajectories from multiple environments, built based on the Open Source model Mixtral 8x7b Mix-of-Expert. The three models, DeepSeek, LLaMA, and xLAM, can be found in HuggingFace platform (note that xLAM has a restricted, non-commercial use). Overall, proprietary GPT-family models dominate across frameworks, while support for open-weight alternatives remains uneven and often secondary, Table 1 includes information on the LLMs used by the revised frameworks.

Framework publications provide limited information about the specific configuration of backbone LLMs. However, examining their publicly available source code repositories on GitHub, reveals that several frameworks expose inference-level configuration parameters. Although the range and granularity of configuration options differ significantly across systems, AgentLite allows adjustment of temperature, stopping sequences, and the maximum number of generated tokens. AgileCoder provides more extensive controls, including temperature, nucleus sampling (top-p), number of output choices, streaming, stop sequences, maximum tokens, frequency and presence penalties, logit bias, and user identifiers. ChatDev exposes only temperature and maximum token length, whereas HyperAgent only supports temperature configuration. In contrast, MetaGPT provides the most configuration options, including temperature, top-p, top-k, repetition, presence and frequency penalties, maximum tokens, stopping sequences, streaming, log probabilities, best-of sampling, reasoning-specific token budgets, and system prompt control.

[3] https://codeassist.google/
[4] https://openai.com/index/openai-codex/
[5] https://huggingface.co/codellama
[6] https://deepseekcoder.github.io/
[7] https://github.com/salesforce/CodeGen
[8] https://huggingface.co/bigcode/starcoder
[9] https://huggingface.co/datasets/bigcode/the-stack

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

Table 1
Comparison of used LLMs across considered multi-agent frameworks.

ModelType	LLM	AgentLite	AgileCoder	ChatDev	HyperAgent	MetaGPT
Open-source	xLAM-v0.1-r	✓				
Open-weight	DeepSeek Coder 33B					✓
	Spark Lite					
	Llama-3-70B				✓	
	Llama-3-8B				✓	
	Llama-4-Scout-17B-16E					✓
	Llama-4-Maverick-17B-128E					✓
Proprietary	Claude-3-Sonnet				✓	
	Claude-3-Haiku		✓		✓	
	gpt-3.5-turbo	✓	✓			✓
	gpt-3.5-turbo-16k	✓		✓		
	gpt-3.5-turbo-16k-0613	✓		✓		✓
	gpt-4	✓	✓	✓		
	gpt-4-0613	✓				✓
	gpt-4-turbo	✓		✓		✓
	gpt-4-32k	✓		✓		
	gpt-4-32k-0613	✓				
	gpt-4-1106-preview	✓				
	gpt-4o			✓	✓	
	gpt-4o-mini			✓		
	text-davinci-003	✓				
	text-ada-001	✓				

Comparison. There are two main differences between the considered frameworks with respect to the backbone LLMs: using one or more backbone LLMs and the use of open models. AgileCoder, ChatDev, and MetaGPT use the same LLM as a backbone for different agents. HyperAgent uses LLMs of different skills and performances for different agent roles, e.g. the Planner agent needs reasoning skills, and the CodeEditor and Executor agents need more powerful LLMs than the Navigator agent. AgentLite, in theory, supports using different LLMs per agent, although these need to be configured. Using a single LLM across heterogeneous agent roles may simplify configuration but risks suboptimal specialization, especially when reasoning, planning, and code synthesis require different capabilities.

Regarding the use of open models, not all frameworks support them. More specifically, AgileCoder and ChatDev support only proprietary models. Proprietary models often lead in benchmarks and offer high reliability of service and ease of use via API access, but they come with high costs, data privacy issues, limited transparency, and vendor lock-in. In contrast, open models - ranging from open-weight releases to fully open-source models - offer greater transparency, local deployability, and freedom for fine-tuning and auditing, often at no cost. However, they may lag slightly in raw performance or require more technical effort to operate at scale.

Identified Gaps and Potential Improvements. All compared frameworks show a clear convergence toward the use of general-purpose pre-trained LLMs; none report fine-tuning the backbone models for software engineering tasks. As a result, current systems largely rely on prompt engineering and agent orchestration rather than backbone specialization to address SDLC-specific requirements.

Future research should therefore explore the development and integration of domain-specific or fine-tuned LLMs within SDLC-oriented multi-agent frameworks. Promising directions include (a) lightweight fine-tuning or adapter-based specialization of open-source or open-weight models, particularly role-specific fine-tuning aligned with SDLC phases (e.g., requirements analysis, architectural design, testing), which could improve both task accuracy and inter-agent coordination; (b) retrieval-augmented generation leveraging project repositories, issue trackers, and technical documentation to ground agent reasoning in project-specific knowledge; and (c) the definition of standardized evaluation benchmarks that assess model effectiveness across SDLC activities rather than isolated coding tasks. Additionally, frameworks could incorporate explicit model-selection criteria or automated profiling mechanisms to support evidence-based backbone assignment per agent role.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

Finally, the choice between proprietary and open LLMs involves trade-offs between performance, transparency, control, and economic sustainability. Alongside the already mentioned comparison between open and proprietary models, economic considerations also play a critical role in backbone selection decisions. An in-depth cost–benefit analyses revealed that open-source LLMs can reduce inference and licensing costs by a factor of 5× to 29× compared to proprietary APIs, especially for continuous or high-volume workloads (Irugalbandara et al., 2024). Current frameworks provide limited support for systematically evaluating such cost–performance trade-offs during framework configuration, which represents an additional open challenge for future research.

Q5. *Are the roles and the actions of the agents static or dynamic?*

Contextualization. In multi-agent frameworks for software development, distinguishing between static and dynamic agent roles is fundamental to understanding their adaptability and autonomy. Static systems define a fixed set of roles and actions, such as Project Manager, Programmer, or Reviewer that remain constant throughout the development process. These agents operate according to predefined prompt templates and task sequences, which ensures reproducibility and stability but limits flexibility and context sensitivity. In contrast, dynamic frameworks introduce mechanisms that allow agents to assume new roles or modify their behavior based on evolving project contexts, task dependencies, or feedback loops. This dynamic configuration enables more context-aware collaboration and adaptive problem solving but also requires more sophisticated coordination and monitoring strategies to prevent task redundancy or role misalignment. Consequently, analyzing whether roles and actions are static or dynamic provides key insights into how current LLM-based frameworks balance control, flexibility, and autonomy across the SDLC.

Overview. AgentLite defines a BaseAgent with specific Action classes. Users can extend/customize these actions. The agents' roles are not obviously dynamically reassigned based on project state; rather users define agent behavior via prompt generation and defined action types. These behaviors suggest static role and action definitions, with some extensibility.

AgileCoder mimics Agile methodology; it uses sprints, backlog, evolving tasks, and a Dynamic Code Graph Generator that updates dependencies when the code changes. That implies actions are chosen based on the codebase context. The roles (Product Manager, Developer, Tester) are assigned, but because of the sprint and backlog structure and evolving graph context, actions are more dynamic.

ChatDev has predefined roles and structured phases of software development (e.g., a coder, tester, reviewer). These roles follow fixed prompt templates. The available information does not show role or action adaptation depending on changing internal project state (beyond what prompt engineering allows). Therefore, roles and actions seem to be static in structure.

HyperAgent seems to focus on letting users create and customize agents and actions, and offering prebuilt agents. But, we did not find evidence that it supports dynamic reassignment of roles or action selection based on evolving project context. More likely static agents whose behavior depends on user input or configuration rather than internal monitoring of project evolution.

MetaGPT has a roles system with specialized roles (Product Manager, Architect, Engineer, QA, etc.). These are somewhat static in naming. But, importantly, MetaGPT offers the ability to customize LLMs for roles or actions via configuration files, allowing different LLMs or different behavior depending on the action. That suggests dynamic assignment of behaviour in some cases. It also has an environment and memory tracking of actions, which enables some dynamic responsiveness. However, the structural roles remain fairly stable.

To sum up, ChatDev, AgentLite and HyperAgent (mostly) tend to define roles and what actions those roles perform in a fixed way. Even if the exact prompts or LLM used can change, the architecture does not adaptively change who does what mid-project (beyond user configuration). AgileCoder clearly shows more dynamic behavior (changing code dependency graph, evolving tasks, sprints). MetaGPT has dynamic components in how roles/actions use different models or configurations, but its roles are still named and defined statically in structure (though configuration files).

Comparison. In multi-agent frameworks powered by LLMs, the degree of flexibility in defining and adapting agents' roles and actions has a direct impact on collaboration, efficiency, and autonomy across the software development lifecycle. Systems adopting static roles and actions, such as ChatDev or AgentLite, typically define a fixed set of agents (e.g., Product Manager, Developer, Reviewer) and associate them with predetermined tasks and prompt templates. This

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

static structure simplifies orchestration, ensures reproducibility, and allows for clearer traceability of responsibilities. However, it also limits adaptability, agents cannot easily reconfigure themselves or assume new functions as project requirements evolve, reducing responsiveness to contextual changes or unexpected dependencies.

Conversely, frameworks incorporating dynamic roles and actions, such as AgileCoder or partially MetaGPT, enable agents to modify their behavior, allocate new roles, or select actions based on the evolving project state. This dynamic adaptability can enhance flexibility, context-awareness, and error recovery, as agents may redistribute tasks or redefine goals in response to emerging challenges. Nevertheless, increased dynamism introduces new complexities. It complicates coordination, raises risks of overlapping responsibilities or redundant actions, and can reduce system predictability and explainability if agent behaviors evolve beyond predefined constraints.

Overall, the choice between static and dynamic configurations reflects a trade-off between control and flexibility. Static systems are easier to monitor, audit, and replicate—key advantages in regulated or safety-critical domains—, while dynamic systems better support open-ended, exploratory, or iterative software development processes.

Identified Gaps and Potential Improvements. None of the frameworks seems to support roles that themselves evolve or are created or destroyed dynamically in reaction to project needs, changes in domain, or failure conditions. Also lacking are frameworks where agents adapt their actions in more complex ways based on the state of the development (e.g. error rates, test failures, code coverage, feedback loops) beyond just user-defined sprints or graphs.

Future research could also explore hybrid strategies that combine the structural stability of static roles with adaptive mechanisms that adjust task allocation and reasoning depth dynamically, guided by project feedback or performance metrics.

Q6. *What is the structure of agent workspaces? Which internal capabilities do agents possess, such as memory or Retrieval-Augmented Generation (RAG) at inference time? How similar are agent workspaces?*

Contextualization. To better understand how an agent is defined and how it operates, it is important to understand its workspace and the components it contains. We define a workspace as a local, temporary environment that contains all the information related to the current development session. This includes artifacts, messages exchanged between agents, external contextual documents and, possibly, tools available to the agents. The workspace may also include a memory module, a storage component responsible for storing all necessary prior information that might be necessary to successfully perform the current action. The stored information may include user inputs, messages from other agents and/or outputs coming from the agent itself. Retrieved memory content is typically used to enrich the original prompt of the agent. Recently, this is frequently accomplished using RAG. RAG is a method which allows an LLM to generate an output combining an external knowledge with the LLM's internal knowledge. This is achieved by retrieving the most relevant external documents based on the input query and providing them as a context to that query for output generation.

Overview. Different frameworks resort to different implementations of agent workspaces. In MetaGPT, an agent workspace encompasses both file system, storing the artifacts, and a global pool of shared messages, exchanged with the other agents. Different agent workspaces are basically shared; agents access the same locations but filter the relevant information based on their specific roles. Depending on the agent role, its workspace might include tool usage (e.g. Mermaid for Architect). In ChatDev, the workspace encompasses the file system, storing the artifacts, and a Chain structure, containing phase-level messages between two agents. In AgileCoder, in addition to files, the workspace includes a Message Stream containing messages from dual-agent interactions, a code dependency graph that dynamically traces code-related dependencies and a global message pool that also serves as a status task log. In HyperAgent, the codebase and tool suite are integrated into a shared workspace along the file system and the message queue. The tool suite includes, for example, specialized tools for code navigation, formatting and visualization. In AgentLite, the workspaces of Manager and Individual agents differ, in that the Manager's workspace consists of a collection of so-called TaskPackages, which serve to trace the status of tasks that the Manager agent has delegated to Individual agents. On the other hand, an individual agent's workspace encompasses only its internal memory module, which contains the history of its own actions.

Memory module is differently implemented from one framework to another. The simplest type of memory is short-term memory, which stores information relevant for a single phase and is implemented in all five frameworks. Limited context length of LLMs makes it unfeasible to keep all messages in the working memory. Therefore, frameworks use

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

different approaches to cope with this limitation. HyperAgent uses summarization of messages, that is, messages from child agents are shortened and included in the context. MetaGPT claims to do the same (summarization) from the past projects based on the agent role while AgileCoder uses only the final message from the message stream that contains the exchange between two agents. Unlike HyperAgent and AgentLite, where the memory remains a short-term concept, both MetaGPT and ChatDev use short-term and long-term memories. As a long-term memory, ChatDev stores only the outcome of the task as relevant information to be conserved. On the contrary, MetaGPT stores all information exchanged during an interaction in a long-term memory. It then relies on RAG for extracting the most important information from the long-term memory. More specifically, in MetaGPT, long-term memory serves as an external knowledge base from which the relevant context for subsequent actions is retrieved, typically using the most recent short-term information as a query.

In addition to the memory module, some frameworks equip their agents with tools. As such, three out of four agents in HyperAgent (Navigator, Editor and Executor) use different tools to carry out their main functionalities, such as enhancing code navigation, visualizing code structure, integrating keyword search, applying and refining code patches, automatically handling syntax and indentation issues, context-aware editing, summarization, as well as accessing testing and setup documentation. MetaGPT agents also use tools; for example, a Product Manager can use web search tools, or an Architect can use Mermeid to create architecture diagrams.

Comparison. The considered frameworks differ in how they structure agentic workspaces: a workspace can be individual (agent-level) or shared, meaning that more agents can have overlapping workspaces. More specifically, in HyperAgent and AgentLite, agent workspaces are individual, and so are the underlying memory modules. On the contrary, in ChatDev and AgileCoder, the agent workspaces are overlapping during the execution of a phase/conversation as the memory (containing their interactions) is shared between two agents. Furthermore, in MetaGPT, all agent workspaces are overlapping as they share both the global message pool and the common file system.

With respect to workspace structure, agents with different roles within a framework typically do not differ in how their workspaces are structured. The only exception is AgentLite, where not all the agents possess memory: only the Individual Agent, a base agent, has a memory module. Regarding different types of memory, MetaGPT and ChatDev support long-term memory. While ChatDev resorts to storing the final (most important) outcome, MetaGPT stores everything and later filters the most relevant parts using RAG.

Identified Gaps and Potential Improvements. The considered frameworks present a variety of implementations and design practices, including the use of file systems, global message pools, message streams, message chains, and separate memory modules, as well as architectures with or without long-term memory. However, it remains unclear which of these design choices should be considered as best practices and whether their suitability depends on the specific application. In addition, there is limited understanding of how each suggested solution performs in the long run, in contexts where projects require multiple development cycles. Finally, concurrency management should be studied across different agents' workspace configurations.

Q7. *What prompting strategies are employed to guide agent behavior, and to what extent do they influence coordination among agents?*

Contextualization. Research indicates that LLM text generation is significantly influenced by the input prompts they receive, as even the small modifications in wording or structure can affect the output (Brown et al., 2020b; Wei et al., 2022). To improve input prompts, several prompting strategies have been proposed in the literature. Although these strategies can apply to various types of prompts, due to the nature of the prompts found in the LLM-based multi-agent systems discussed here, we focus specifically on natural language-like prompts, sometimes also referred to as "hard prompts" (Hou et al., 2022). Apart from simple direct prompting, these strategies also encompass more advanced approaches. Some of the exemplars include few-shot prompting (Brown et al., 2020b), chain-of-thought (CoT) (Wei et al., 2022), role-playing (Wu et al., 2023) and Reason & Act (ReAct) prompting (Yao et al., 2022b). In few-shot prompting, a few input-output pairs as examples are provided to showcase the expected model behaviour, together with an initial instruction (direct) prompt. With CoT, the model is instructed to decompose the complex task into a series of less complex, intermediate steps; in other words, it is encouraged to reason internally before providing the final answer rather than simply generating a single answer. A role-playing prompt instructs the model to impersonate a specific role

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

and, typically, act as if it possesses the expertise of that role. ReAct interleaves reasoning with task-specific actions, which is typically performed in dynamic environments where intelligent and adaptive decision-making is critical.

Overview. The considered frameworks use different prompting strategies. MetaGPT uses ReAct prompting to let the agent reason and select the next action to be performed. MetaGPT's action-based prompts often include role-playing to encourage the agent to behave as a software development role that would normally be in charge of performing that action. Occasionally, an action-based prompt contains a most basic variant of CoT, for example "Think before writing:" in the *write_test* action. Besides, MetaGPT action-based prompts frequently include formatting examples, such as write_code action.

HyperAgent is likewise exploiting role-playing and ReAct prompting, except that instead of action-based prompts, each agent role has its own system prompt with role-playing instructions. Here, two agent roles, Planner and Navigator, are specifically instructed to "think step-by-step". Besides this, system prompts in HyperAgent clearly define agent hierarchy; for example, the Planner agent is explicitly informed about the "interns" at its disposal, while other agents are always instructed to focus on the Planner agent's requests.

ChatDev and AgileCoder share common traits in their prompting strategies, as both are based on the CAMEL framework. First, both frameworks include a base prompt. In the case of ChatDev, it is minimalistic, enumerating roles and introducing ChatDev as a software company. On the other hand, in AgileCoder, it is a more descriptive one, introducing AgileCoder as a software enterprise, explaining the type of SDLC model it follows, and also explaining the existing agent roles. Second, both introduce role-based prompts on the agent level, thus operationalizing role-playing prompts. Third, both frameworks possess phase-level prompts that further elaborate the purpose and the scope of that particular phase within the SDLC. In both frameworks, each phase involves two agents working collaboratively on the same task. The main difference is that ChatDev allows for role-flipping between the agents, thus exploiting the prompting strategy introduced in CAMEL as the "inception prompting". Inception prompting directly influences the agent coordination. Furthermore, these phase-level prompts occasionally also combine elements of CoT. While this is less pronounced in ChatDev, where "think step by step" instruction is supplied only during the coding phase, AgileCoder is more frequently exploiting CoT (e.g. in the following phases: ProductBacklogCreating, SprintBacklogCreating, Undone tasks, RolesEngagement, Coding, InheritCoding). Furthermore, during the code review phase of AgileCoder, multi-step prompting is shown to be a better-performing alternative to single-step prompting.

AgentLite includes PromptGen, a prompt-generation module. According to theoretical framing, agent prompts involve role-playing, action-based instructions, few-shot examples and even specification of constraints, but the final prompts to be used are left to user specification. Based on the examples provided for several use cases, the framework supports role-based prompts, ReAct (example of IndividualAgent) and CoT strategies (example for ManagerAgent provided).

Comparison. Depending on concrete scenarios, different prompting strategies can be more advantageous over others. For example, ReAct prompting is more suitable than pure CoT in situations where the agent might need additional resources to complete the task, such as access to additional data sources or tools, for example, MetaGPT's architect agent accessing Mermaid to create the architecture diagram. Instead, for pure reasoning tasks when tools are not needed, ReAct might instead bring additional complexity, leading to a pipeline that is more difficult to control and debug. With respect to agent coordination, the two prompting strategies that impact the most are inception prompting in ChatDev and HyperAgent's hierarchy-imposing prompts, which explicitly "prescribe" who is the next agent to interact with.

Identified Gaps and Potential Improvements. Considered frameworks typically opt for a single dominant prompting strategy to go with. That strategy is often chosen based on outperforming results, which are reported in the literature, possibly by some initial experimental evaluation, which, nevertheless, is not provided. It would certainly be insightful to have more concrete motivation for the choice of the prompting strategy. The only step in this direction was done by MetaGPT, where the comparison between a high-level instruction and a more detailed one was performed; however, it is difficult to quantify how detailed an instruction should be and to find a "sweet spot" between the token-related cost and performance. Furthermore, prompts are often "static", and their extension/modification is either restricted to concatenating different prompts or requires human intervention. Finally, different frameworks provide evidence of overriding this main strategy at different places (e.g. one function producing artifacts resorting to CoT and the other to a few-shot strategy). This underpins the need to exploit automatic prompt optimization in the multi-agent setup.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

Table 2
Comparison of used benchmarks for evaluation of the multi-agent frameworks

Data (sample size)	AgentLite	AgileCoder	ChatDev	HyperAgent	MetaGPT
Closed Source			✓		✓
SoftwareDev					✓
ChatDev internal			✓		
Open Source	✓	✓		✓	✓
GitHub				✓	
SWE-bench (2,294)				✓	
RepoExec (355)				✓	
Defects4J (357)				✓	
Crowd-Sourcing		✓			✓
HumanEval (164)		✓			✓
MBPP (974)		✓			✓
ProjectDev (14)		✓			
Other Public Data	✓				
WebShop (12, 087)	✓				
HotPotQA (113, 000)	✓				
BOLAA (300)	✓				

4. Operational aspects

4.1. Evaluation

Q8. *What datasets and real-world use cases are considered for the evaluation of the multi-agent framework?*

Contextualization. Evaluation of multi-agent frameworks can be performed through manual testing of, for instance, the agent-generated code in real-world scenarios, or by using suitable benchmarks to measure the system's performance (Hou et al., 2024). However, most of the considered agent systems relied only on benchmark evaluation. In this section, we describe the benchmarks used and analyze the categories chosen for the analysis of four agent-based frameworks.

Overview. AgentLite implemented various agents for their evaluation and showcased their performance on two widely used benchmarks. The first benchmark is based on HotPotQA (Yang et al., 2018), a retrieval-augmented question-answering benchmark for multistep reasoning that has been adapted for LLM agent evaluation in the AgentBench benchmark (Liu et al., 2024a). This benchmark tests the multi-step reasoning and decision-making abilities of an agent system, as each question requires at least two documents to answer. The second benchmark is performed in the Webshop (Yao et al., 2022a) environment and is based on the AgentBoard framework. The benchmark consists of web browsing decisions on the Webshop, an online shopping website simulation featuring 1.18 million real-world products. The tasks of these two benchmarks can be categorized into five categories: code, game, web, tool, and embodied AI (Liu et al., 2024b). Besides the benchmarking evaluation, AgentLite has been evaluated in five applied real-life scenarios. Those applications cover multiple settings such as human-in-the-loop, multi-modality, innovative potential, and practical application (Liu et al., 2024b).

AgileCoder utilizes HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021), two well-established benchmarks for evaluating the system's performance on code generation, as well as a newly introduced benchmark, ProjectDev (Nguyen et al., 2025). ProjectDev is a dataset generated by Nguyen et al. (2025) due to a lack of benchmarks covering intricate software development tasks. Each task includes a prompt and requires the system to produce a complete codebase with multiple executable files (Nguyen et al., 2025). The tasks simulate real-world software development by drawing on software requirements spanning categories such as games, image processing, and data visualization. Note that the authors point out that MetaGPT and ChatDev have produced concurrent work, which, however, is closed-source. The other two benchmarks, HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021), are two well-established software development benchmarks for Python coding. Both benchmarks are generated from crowd source software development problems and are used for agentic systems and LLMs in general.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

Table 3
An overview of which system was used as a baseline. The (✓) checkmarks in brackets indicate that in the paper, the other system was acknowledged in related work, for instance, but not used for evaluation.

Comparing ↓ \ Compared to →	AgentLite	AgileCoder	ChatDev	HyperAgent	MetaGPT
AgentLite	✓				
AgileCoder		✓	✓		✓
ChatDev			✓		✓
HyperAgent		(✓)	(✓)	✓	(✓)
MetaGPT			✓		✓

ChatDev introduced an internal benchmark (Qian et al., 2024); however, unlike the ProjectDev dataset, this benchmark was not publicly released by the authors. The dataset used by the authors includes software categories from multiple platforms, including Ubuntu, Google Play, Microsoft Store, and Apple Store. Each of the 1,200 tasks contains a prompt based on a textual description of the software requirements and can be categorized into five categories: Education, Work, Life, Game, and Creation.

MetaGPT is evaluated in a similar approach to ChatDev by generating an internal closed-source benchmark (Hong et al., 2023), and to AgileCoder by using HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021) for code generation evaluation. The internal benchmark covers seven experiments across three categories: game, web development, and data analysis. It consists of 70 prompts, which are evaluated using various statistics and metrics defined by the authors.

The HyperAgent authors selected a variety of open-source benchmarks to evaluate the framework's complexity and real-world applicability. They chose three benchmarks, SWE-bench (Austin et al., 2021), RepoExec (Hai et al., 2025), and Defects4J (Just et al., 2014), each of which requires multiple reasoning steps and retrieval from relevant code or text. SWE-bench is a code generation benchmark comprising over 2,000 samples from 12 open-source Python repositories on GitHub. It is a well-established benchmark widely used to evaluate code-generation LLMs, foundation models, and LLM-based agent systems. The samples are generated from GitHub issue resolutions. RepoExec (Hai et al., 2025) is also a code generation benchmark containing 355 samples; however, the tasks must be solved at the repository level. The last benchmark, Defects4J, evaluates fault localization in the form of bugs. The benchmark contains 854 bugs from open-source projects with the following properties: tracked (linked to an issue tracker and mentioned in the commit), atomic (fixed in a single commit), minimized (irrelevant changes removed), code-based (fixed via source code changes), and test-verified (has a reliable triggering test that fails before and passes after the fix).

Comparison. In general, except for the authors of ChatDev, all agentic systems are evaluated using public benchmarks. MetaGPT is evaluated using both closed- and open-source data. All open-source benchmarks used for any agentic system focus on narrow, code-centric tasks. Moreover, HumanEval and MBPP are human-generated programming problems; their outputs are small code snippets that are not comparable to the code generation required in real-world software engineering settings (Qian et al., 2024).

Identified Gaps and Potential Improvements. There are three main gaps in the evaluation of all agentic systems. First, all systems are evaluated using fewer than 5 benchmarks, which does not provide a holistic overview of their performance. Moreover, benchmarks such as SWE-bench, HumanEval, MBPP, and HotPotQA are well-established benchmarks that are used for evaluating foundation models rather than agentic systems (Scao et al., 2023). One of the authors of the HotPotQA benchmarks even raised his concerns about using this benchmark for evaluating agent systems in the first place (Qi, 2025).

Second, in addition to the benchmarks already described, most agent systems are not applied to real-world scenarios involving complex or long-term software projects. AgentLite demonstrates real-world applications across five different setups. These examples demonstrate practical application, human-in-the-loop, multi-modality, and innovative potential (Liu et al., 2024b). The setups are straightforward: for each application, the agents get assigned a name, possible actions, and a predefined role. Each agent then performs the dedicated task. On their GitHub[10], they also provide more extensive instructions for initializing these applications, with working examples. However,

[10] https://github.com/SalesforceAIResearch/AgentLite/tree/main/tutorials

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

those applications are not software-related settings, and their scalability is also not showcased. Three out of five applications are single-agent applications. For the multi-agent applications, there is no per-agent evaluation. Moreover, the performance of the tasks that should be solved is not measured.

AgileCoder, MetaGPT, and ChatDev are tested on real-world software development tasks using self-curated samples from the authors, but only the ProjectDev benchmark is open-source. Therefore, we only cover an analysis of AgileCoders' performance on ProjectDev[11]. ProjectDev contains practical applications of 20 tasks (Nguyen et al., 2025). Those tasks range from generating games such as Snake to building a weather App. AgileCoder generates executable code in 57.79% of the samples (Nguyen et al., 2025). Although they claim to pick intricate software development tasks, it is safe to say that those examples, such as generating a Snake game or a Flappy Bird game, are well-established tasks that have been used in the past to showcase the performance of other agent systems and LLMs for code generation.

The third shortcoming is that none of the agent systems addresses constraints related to limited resources or environments. AgileCoder, HyperAgent, and MetaGPT evaluate the cost efficiency and expenses of using agentic systems, but there are no benchmarks for on-premises computation or the energy costs of running those systems. Agentic systems that should support real-world software development should be language model-agnostic, covering both open-source and on-prem solutions. However, some authors of AgileCoder and ChatDev mention computational demands and environmental impact as limitations of their publications.

Q9. *What metrics are used to assess the LLM performance?*

Contextualization. The metrics that are used for evaluating the performance of the agentic systems can be divided into three categories (see Table 4): classic (NLP) metrics, code-related metrics, time- and cost-related metrics and mixed-mode metrics. In this section, we will describe all metrics used in these categories.

Overview. Public benchmarks for evaluating agentic systems are mostly assessed using NLP metrics. In HotPotQA, for instance, the performance is quantified using accuracy and F1-score as metrics. The Webshop has its own metric based on the average reward in the webshop environment. SWE-Bench is evaluated using three different metrics: the percentage of resolved instances, the average time cost, and the average cost of the generated tokens.

For Defect4J, the used metric is acc@k, which represents the number of correct bug detections in the top k generated outputs. HyperAgent was evaluated on accuracy, denoted as acc@1. HumanEval and MBPP use pass@k as a metric for the system evaluation. The authors of AgileCoder adopted this metric to evaluate the functional accuracy of the top-1 generated code, following the approach of ChatDev and MetaGPT (Qian et al., 2024; Hong et al., 2023). RepoExec is also evaluated with pass@k. The authors of HyperAgent used $k = \{1, 5\}$ for their evaluation. For the ProjectDev benchmark, the agent-generated code is evaluated through human assessment and statistical analysis, such as code runtime, token usage, or average number of sprints performed by the system.

As ChatDev supports a holistic evaluation, the authors introduced four objective metric dimensions that should provide a more comprehensive analysis than pass@k (Qian et al., 2024). The first dimension is completeness, which is the percentage of software without "placeholders" or missing parts. The second dimension is executability, which is calculated as the percentage of successfully compiled code. The third dimension is consistency, measured by the cosine distance between the generated software and the requirements' textual embeddings. The last dimension is quality, calculated by multiplying the first three dimensions. Note that an alternative quality dimension is the average sum of the first three dimensions.

Besides evaluating model performance using the standard benchmark metric, the authors of HyperAgent (Phan et al., 2024) calculated the average cost per benchmark in US dollars. On the SWE-Bench dataset, the average cost spans from 0.45 $ to 2.01 $ depending on the LLMs API used. For the RepoExec and Defects4J benchmarks, the average cost is reported as 0.18 $ as they only evaluated a single HyperAgent setup on both benchmarks. Note that these numbers may vary depending on the LLM used for the agents and the current price per token of the closed-source model.

For the internal benchmark of MetaGPT, the statistics and metrics cover five evaluation dimensions. First, executability: rate generated code from 1 to 4 by humans, with 1 indicating non-functional code and 4 indicating flawless code. The second dimension is cost, which includes running time, token usage, and expenses. The third

[11] https://huggingface.co/datasets/Fsoft-AIC/ProjectDev

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

Table 4
Metrics used for evaluating considered multi-agent frameworks.

Metric Type	Metric	AgentLite	AgileCoder	ChatDev	HyperAgent	MetaGPT
Standard	F1-Score	✓				
	Accuracy	✓			✓	
Code - Stats	Lines of code generated			✓		✓
Code - Executability	pass@k		✓		✓	✓
	Code Executability		✓	✓	✓	✓
	acc@k				✓	
Code - Quality	#Errors		✓	✓	✓	
	Code Completeness			✓		
	Code Consistency			✓		
	Code Quality			✓		
Time Perf	Entire Running Time in s		✓			✓
	Avg. Time/Sprint in s		✓			
	Human Revision Cost					✓
	#Sprints		✓			
Cost	Token Usage		✓	✓		✓
	Expenses (USD)		✓		✓	✓
Mixed	Productivity					✓

dimension covers code statistics, such as the number of generated code files, lines of code per file, and the total number of code lines. The fourth dimension is productivity, calculated as the number of tokens used divided by the number of lines of code. The last dimension is human revision cost, calculated as the time required for manual code corrections.

Comparison. For the public benchmarks, all agentic systems are evaluated using the same metric. The authors of AgileCoder adapted the unbiased version of the pass@k metric, based on the approach proposed by the authors for MetaGPT, to evaluate the functional accuracy of the top-k generated codes (Nguyen et al., 2025). For the closed-source benchmarks of ChatDev and MetaGPT, evaluation dimensions are defined to facilitate a more holistic evaluation. While both cover completeness and quality as evaluation dimensions within their proposed metrics, the authors of MetaGPT focus more on code statistics and costs with the three newly introduced dimensions. The authors of ChatDev focus on code completeness and consistency. Although the authors of ChatDev do not mention code statistics among their metrics, they also published insights into the generated code, including the number of generated tokens, files, and lines of code.

Identified Gaps and Potential Improvements. MetaGPT uses human-based evaluations on two of its five metric dimensions; however, the authors do not further explain how the human evaluation is performed. It is not stated if the human evaluation was performed by multiple specialists or by a single person, which lowers the reliability and trustworthiness of the metrics. With the human revision cost, they also introduce a highly subjective metric that needs to be quantified, but the authors of MetaGPT did not. Moreover, code statistics are used as a metric but not compared to any gold standard or set in relation to. For instance, the number of used tokens could have been taken in relation to the number of tokens needed to perform a task or to solve a problem. Code statistics should be used to provide further insights beyond the evaluation using established metrics, like the authors of ChatDev did.

In general, the metrics used for all evaluations of all agentic systems are narrowed down to code and text generation performance and expenses. Evaluations based on coding benchmarks or small app tasks tend to prioritise syntactic correctness and executability, but overlook metrics related to system performance in production, maintainability, and incident recovery, which are key dimensions for any framework aspiring to support the full SDLC. Also, the code statistics and expenses only show a current snapshot of the agentic systems, as the expenses, as well as, for instance, the number of generated tokens, may have changed since the time of publication. Beyond pass@1 and executability, metrics should include system uptime, recovery time, error traceability, and change impact. Benchmarks should model realistic DevOps pipelines and allow agent frameworks to be assessed across the entire lifecycle.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

Q10. *How does the system scale with increasing project complexity, and has computational cost been evaluated? Do small agent systems outperform state-of-the-art (SOTA) LLMs in code generation?*

Contextualization. Evaluating MAS for SDLC requires understanding how their performance is affected as project complexity increases. Agent systems introduce components such as task decomposition, coordination mechanisms, and communication between agents that make their evaluation difficult to perform.

Assessing scalability requires profiling data, including metrics such as execution time, token consumption, number of agent interactions, and resource utilization related to complex tasks. However, MAS are a relatively recent research field and systematic data collection and benchmark datasets remain limited. As of today, it is still unclear how these systems scale in practice or how their computational cost compares to single-model approaches. Moreover, evaluating agentic systems in real-world scenarios involving complex or long-term software projects (e.g., with versioning, product strategy, or backward compatibility) demonstrates their applicability. For a comprehensive framework evaluation, a comparison with LLMs on code generation can be a useful indicator of their performance, as well as the size of LLMs used to test them.

Overview. What concerns scalability, the authors of AgileCoder emphasized it as one major limitation when dealing with large and complex software (Nguyen et al., 2025). The authors of ChatDev claim that their agent system is more suitable for prototyping than for real-world applications (Qian et al., 2024). HyperAgent is explicitly designed to support parallel subtasks execution and targeted recomputation. In cases where the code under investigation is modified, HyperAgent performs a localized analysis, identifying and re-executing only the affected components of the codebase. Although AgentLite does not support parallel task execution, it offers a certain degree of scalability through its task decomposition-based design, in which subtasks are assigned to Individual Agents by the Manager Agent.

Regarding the size of LLMs used, all considered frameworks utilized large LLMs (with more than 7 billion parameters) as the backbone for their agents. None of the authors considered using smaller models to test them inside their proposed framework. Regarding benchmarking LLMs for code generation, the most comprehensive analysis is provided by the authors of AgileCoder (Nguyen et al., 2025). They compare their framework not only with two agent systems, MetaGPT and ChatDev, but also evaluate seven LLMs trained on code: CodeGen (Nijkamp et al., 2022), CodeGeeX (Zheng et al., 2023), PaLM Coder (Chowdhery et al., 2023), DeepSeek-Coder (Guo et al., 2024a), GPT-3.5, GPT-4 (OpenAI et al., 2024), and Claude 3 Haiku (Rahman et al., 2024). However, AgileCoder outperforms all LLMs by a large margin.

Other considered frameworks are only compared to other SOTA agent systems. The authors of MetaGPT compared their framework with AutoGPT (Significant Gravitas, 2023) and AgentVerse (Chen et al., 2024). The authors of ChatDev compared their results with MetaGPT and GPT-Engineer (Osika, 2023). GPT-Engineer is a single-agent system with a backbone LLM that understands task requirements and has reasoning capabilities.

Comparison. HyperAgent is the only framework among the considered ones designed to explicitly ensure scalability. A comparison of the cost evaluations of MetaGPT and HyperAgent highlights the different approaches in terms of scalability. Both frameworks show a wide range of costs on the tasks, with MetaGPT being six times more expensive on average (see Q8). A partial explanation of the observed cost differences can be drawn from the architectural design and operational principles of the agent frameworks. Specifically, as HyperAgent performs a localized intervention on the codebase, it is inherently more efficient in scenarios involving incremental code changes, reducing redundant computation and, consequently, the cost. On the contrary, other agentic systems, such as MetaGPT (Hong et al., 2023), lack this granularity and recompute the entire task pipeline whenever a change is detected. While this remains a theoretical interpretation in the absence of fine-grained profiling data, it offers an important perspective on the potential scalability advantages of HyperAgent over more monolithic designs, such as MetaGPT, ChatDev and AgileCoder. Regarding LLMs for code generation benchmarking, AgileCoder has been compared also with backbone LLMs, while MetaGPT and ChatDev only with other multi and single-agent systems.

Identified Gaps and Potential Improvements. A major gap is that the considered frameworks have not been tested on real-world use cases, so there is no evidence of their scalability with increasing project complexity.

A second potential improvement could be a comprehensive evaluation of the scaling capabilities of open-source LLMs. Recent research has shown that LLMs with fewer than 4 billion parameters achieve strong performance on

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

benchmarks such as HumanEval and MBPP (Bakouch et al., 2025). For software-oriented use by small and medium-sized companies, such performance insights are valuable for building an agile software development system.

Finally, it is worth noting that the computational cost of a language model can also be calculated by quantifying the compute of the agent systems. Especially for on-premise work and open-source models, that metric is a great indicator for end-users. Moreover, those evaluations are more persistent compared to cost evaluations of closed-source commercial models. Nonetheless, this perspective has not been analyzed for any of the frameworks.

5. Human interaction aspects

Q11. *Does the framework support human alignment, human-in-the-loop processes, or feedback incorporation? If so, what level of feedback is supported (e.g., intermediate vs. final feedback), and how is this implemented across agents?*

Contextualization. Human interaction plays a crucial role in aligning AI frameworks with expert knowledge. According to Ferrara (2023), human operators can act as a complementary validation layer of AI-generated outputs to identify potential issues and provide corrective feedback that progressively strengthens the reliability and fairness of the model. These same authors highlight the importance of human intervention in AI systems, which may involve conducting regular audits, retraining AI models with curated data, applying fairness metrics, or implementing human-in-the-loop (HITL) approaches. In these approaches, experts can guide AI by finding the right balance between automation and human judgment (Ferrara, 2023).

In this section, the different frameworks are analyzed in terms of feedback and human-machine interaction, whose degree of autonomy is further evaluated according to a five-level autonomy scale (Feng et al., 2025), which ranges from Level 1 (L1) to Level 5 (L5). L1 is the lowest level of autonomy, in which the user acts as an operator, directing and making decisions. In L2, both the user and the agent make decisions; they collaborate, and the agent can independently work on its own tasks. In L3, the user is mostly involved in providing feedback, preferences, and guidance, but it is the agent that plans and executes the activities. In L4, the user is only required to intervene when some blocking events occur and the agent cannot proceed with its tasks. L5 is the highest level of autonomy in which the user only monitors as an observer.

Overview. AgentLite can be placed at L2 in the five-level autonomy scale (Feng et al., 2025), since users have an active participation in the design and planning tasks, providing direct input and guidance as collaborators in the process. The framework allows users to define specialized HumanInstruction actions, which enable querying humans for input during execution and drive multi-round interactions in natural language, ensuring the user acts as a collaborator in task design and planning. A clear example is the Interactive Image Understanding application, where humans can engage in multi-round interactions in natural language by giving questions for several rounds, such as “What is in the image?, What is the color of the bridge?”.

AgileCoder can be placed at L5, since it does not rely on HITL. Similarly, the HyperAgent framework presents an autonomy L5, as all the workflow is performed autonomously by four agents and humans are not involved in the decision-making loop during task execution. ChatDev also has an L5 level of autonomy, even though the latest implementation enables an L3 configuration, allowing users to set a human node to provide input.

MetaGPT can operate at autonomy levels L5 or L3, depending on the configured workflow. The default configuration supports an end-to-end autonomous process; however, the framework allows users to define HumanInput actions that interrupt execution and require human input before proceeding.

Comparison. The multi-agent frameworks operate at a high level of autonomy, ranging between L3 and L5, with only AgentLite positioned at L2. This is because AgentLite allows users to be involved in the development workflow as consultants, meaning that tasks can be delegated to humans (HITL), rather than limiting their role to providing feedback or validation. Both MetaGPT and ChatDev, however, support an optional L3, enabling the definition of human input nodes within the autonomous workflow.

Identified Gaps and Potential Improvements. Overall, while AI frameworks increasingly explore HITL, the integration of human interaction remains limited in many cases, typically confined to initial stages or pre-configured

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

human input actions. A promising future direction may be to increase human-agent collaboration, like the HULA framework developed by Takerngsaksiri et al. (2025), which incorporates human feedback at each step of the software development tasks, not only to enhance input context, but also to ensure "the software development process remains collaborative rather than entirely reliant on fully autonomous agents". Human expertise integration has shown potential in mitigating LLM-related issues like bias or hallucinations. Additionally, intermediate output validation could reduce the risk of error propagation and requirements misalignment.

Q12. *Are there any usability considerations related to the interaction with the framework?*

Contextualization. The usability of the framework, that is, how easy it is for users to operate and configure the system, is an important consideration. More specifically, we define usability as the ease of installation and initial use of the framework, as well as the learning curve associated with customizing the agents and workflows for specific use-cases Abran et al. (2003).

Overview. AgentLite is a generalist framework that requires users to define their Individual and Manager agents. Although the framework does not provide any pre-configured agent setup, it allows users to define and test new agents with a few lines of code. It also provides a UI, but only for testing purposes. The documentation is clear and includes an explanation of agent definitions, along with several notebooks containing use-case examples. However, support for user learning is limited, especially with respect to guidance and feedback mechanisms.

As an SDLC-focused framework, AgileCoder gives less flexibility to the user in terms of agent definition and customization, but provides an easy command-line interface to run the default workflow execution. Installation and initial usage are straightforward, and a Web UI is available for testing and demonstration purposes. Similar to AgileCoder, ChatDev provides a simple installation process and command-line execution. Both AgileCoder and ChatDev offer an easy workflow customization interface, allowing users to modify and define workflows through JSON files by specifying the agents involved in the SDLC phases and the prompts used. The design of Hyperagent is characterized by its generalizability and scalability. Initial usage requires the installation of some dependencies; although the framework supports command-line execution, the limited documentation makes it less straightforward to understand how to define agents and configure workflows.

MetaGPT gives the user limited flexibility regarding task planning and coordination, since these processes follow Standardized Operating Procedures (SOPs). Once the user provides the requirement, the system autonomously executes the entire multi-agent workflow. The installation process is easy and well-documented, and can also be done via pip command. Moreover, the documentation illustrates several examples of customization capabilities. The framework does not provide an explicit agent-to-human feedback mechanism, but the possibility to pre-define a human agent to interact with the active workflow.

Comparison. AgentLite and HyperAgent adopt a general-purpose approach based on the definition of custom agents, resulting in greater flexibility across diverse scenarios and use cases. In contrast, ChatDev, AgileCoder, and MetaGPT focus primarily on code development, relying on preconfigured agents, thereby reducing flexibility in favor of ease of use and a lower learning curve. While ChatDev and AgileCoder provide a clearer and more easily customizable workflow interface through the use of a JSON file, MetaGPT is better documented and offers greater flexibility in integrating user interaction. AgentLite is better documented than HyperAgent, whose installation appears more complex due to software dependencies.

With respect to prompt customization, AgileCoder and ChatDev are much more user-friendly than MetaGPT. More specifically, in AgileCoder and ChatDev, prompt customization requires modification of a single configuration file containing phase prompts. On the contrary, in MetaGPT, just localizing appropriate prompts requires the inspection of various scripts in the codebase. In HyperAgent, this localization is fairly easy, but the user needs to have a good understanding of how the Planner agent interprets these descriptions to get the desired behavior, as the framework relies heavily on automated planning rather than fixed scripts. In AgentLite, it is easy to localize the PromptGen module, but the customization is not as trivial as in AgileCoder and ChatDev, as the user needs to understand the prompt template module.

Identified Gaps and Potential Improvements. None of the considered frameworks provide a proper UI that could allow users to use and configure the framework without inspecting the codebase. The adoption of a UI would

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

enable more immediate and extensive use, increasing agents and workflow customization possibilities, including the selection of backbone LLMs and interventions on corresponding prompts. Furthermore, complete integration with development tools such as IDEs and repositories is still an unexplored area, which could facilitate alignment with existing organizational development processes and contribute to broader adoption. Even though HyperAgent interacts with repositories and existing codebases, it is primarily intended for code completion and bug fixing rather than supporting full integration with real-world SDLC processes.

6. Trustworthy and Responsible AI aspects

Q13. *What are considered hallucinations in the context of software development? How are these detected and mitigated, particularly when errors spread across agents? Are mitigation strategies applied uniformly across all agents? Does the framework recommend specific prompting strategies or prompt engineering techniques to improve the reliability and trustworthiness of the output?*

Contextualization. Hallucinations are one of the critical problems in LLMs and LLM-based systems, in general. Hallucination definition in the literature varies across domains and tasks. Instead, the authors of the considered frameworks are more aligned in the definition of hallucinations.

As such, in MetaGPT, hallucinations are mentioned in the context of incomplete functions, missing dependencies and/or undiscovered bugs. MetaGPT also highlights hallucination risks, particularly due to imprecise definitions of tasks and the accumulation of errors as multiple agents interact in complex workflows. Although HyperAgent does not explicitly define hallucinations in a software development setup, it can be deduced that any incomplete/unfunctional code is considered a hallucination. As a potential cause of agent hallucination, HyperAgent indicates underlying LLM hallucination leading to premature task termination as well as context loss due to agent miscommunication. ChatDev borrows definition of Agnihotri and Chug (2020), considering hallucination to be any incomplete, incorrect or unexecutable code that does not satisfy the initial requirements. AgileCoder mentions problems such as error-prone code, misalignment of the code with the sprint backlog and failing to satisfy the acceptance criteria.

Overview. Most of the frameworks acknowledge hallucinations as a critical challenge. However, not all frameworks explicitly address this issue. More prominently, rather than detecting hallucinations, frameworks are more oriented towards hallucination prevention and, to lesser extent, hallucination mitigation.

In MetaGPT, hallucination prevention is implemented over four independent levels. First, on a workflow level, with a firmly enforced, predefined and simple workflow with role-specific actions. Second, on a communication level, with "subscription" mechanism which prevents an agent from accessing to an irrelevant content. Third, through an iterative code improvement with an executable feedback mechanism, which helps the agent benefit from its own historical information. And finally, through customized instructions in the prompts that are driven by SOP and require structured outputs, thus ensuring the agent's conformity to the established standards.

ChatDev introduces a communicative dehallucination mechanism to address the hallucination problem. This involves a structured dialogue pattern where the assistant agent proactively requests more detailed input before finalising its response. The interaction includes deliberate "role reversal", enabling the assistant to take on an instructor-like role to clarify ambiguous instructions. This iterative clarification reduces premature code generation and facilitates more accurate, context-aware outputs (Qian et al., 2024). Notably, this mitigation is localized to coding-related roles (code completion, review, and testing), where the risks of hallucination are most impactful. Qian et al. (2024) performed an ablation study, examining the effect of the presence/absence of this mechanism, and showed that its absence results in decreased performance.

To mitigate hallucinations in AgileCoder, a three-step code review process is applied. It verifies that: i) there is no incomplete code, ii) the code satisfies the backlog and, iii) the code passes acceptance tests and is error-free. The testing phase is another touchpoint where hallucination prevention is taking place within the AgileCoder. To support this process, it utilizes internally generated dynamic code dependency graph (CDG). Reversing topological order of CDG helps devise a logical testing plan and improve test cases that respect dependencies among coding scripts (Nguyen et al., 2025).

HyperAgent does not explicitly propose any mechanism for addressing hallucinations, although some of the specialized tools help mitigate hallucinations. For example, Repair Editor helps resolve problems such as incorrect

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

syntax or incorrect file paths. AgentLite does not explicitly define hallucinations, nor does it propose any coping mechanism in this regard.

Comparison. Considered frameworks distinguish in the way hallucination is addressed: MetaGPT's approach is more oriented towards prevention, while AgileCoder and HyperAgent mostly focus on mitigation; ChatDev is both prevention and mitigation oriented, while AgentLite does not consider hallucinations at all. Furthermore, the agents and roles are treated differently across most frameworks. MetaGPT is the only framework that has the mechanisms, apart from customized role-based and action-based prompts, which are agent-agnostic. For ChatDev and AgileCoder, hallucination mechanisms are considered only for development-review-test agent roles, rather than being applied uniformly across all agent types. HyperAgent acts similarly, except that the hallucination mitigation is "outsourced" to the tool, instead of the agent itself. The only prompting strategy (among those described in Q7) that is explicitly mentioned in the context of hallucination prevention is inception prompting in the communicative dehallucination mechanism of ChatDev.

Identified Gaps and Potential Improvements. One of the main drawbacks of handling hallucination in considered frameworks is the lack of hallucination quantification and hallucination detection, making the effectiveness of mitigation strategies unclear. Additionally, there is no benchmarking of hallucination prevention strategies. Likewise, a more systematic hallucination mitigation is needed across the framework, as the current coping strategies are typically local, tailored to the roles and functions of specific agents.

While MetaGPT introduced more global mechanisms, some of these are still pretty role-sensitive: agents tasked with critical design or engineering roles benefit most from SOP-guided execution and output validation. On the other hand, across other considered frameworks, hallucination mitigation remains incompletely addressed, particularly beyond code generation phases. Roles involved in requirements elicitation, architecture definition, or task coordination are equally prone to creating plausible yet inaccurate artifacts (e.g., ambiguous requirements, misleading task decompositions) and often lack explicit safeguards.

Furthermore, none of the frameworks mitigates the hallucination propagation problem where the errors compound across interactions (that is, unless it results in a compile-time or test-time failure downstream). In most frameworks, once an agent completes its assigned task, there is no formal mechanism for revising its output in the light of downstream evidence, making hallucinated content hard to trace. A potential workaround for this limitation would be the implementation of human-in-the-loop mechanisms at key decision points across the SDLC. In parallel, agents should be equipped with self-assessment triggers, such as low-confidence indicators or internal consistency checks that proactively request clarification.

With respect to prompting strategies, besides inception prompting, other strategies such as CoT, role-playing and few-shot prompting undoubtedly implicitly contribute to hallucination prevention and mitigation. Considered frameworks are missing a quantitative assessment of the prompting strategies efficiency in this regard. To this end, existing benchmarks such as CodePromptEval (Khojah et al., 2025), considering (a combination of) five different prompting strategies, could be exploited and extended to real-world projects. Additionally, approaches like prompt chunking, which break large prompts into smaller parts to reduce confusion and improve accuracy, could be applied (Darwish et al., 2025).

Q14. *How does the framework monitor responsible AI principles (e.g., transparency, explainability, fairness and bias, ethics, and accountability), and what mechanisms are used to detect and address violations or related issues?*

Contextualization. Responsible AI is an approach to developing and deploying AI systems, in this case, AI agents, that ensures these are built correctly and governed responsibly throughout their entire lifecycle. This process is driven by a set of core principles and guidelines, such as Responsible AI principles from OECD (OECD, 2024) or EU AI Act[12]. Even though these principles are frequently source-dependent and not unanimously aligned, they typically require that an AI agent should be designed to guarantee transparent, explainable, fair, inclusive, ethical and accountable interactions with humans across their entire software development lifecycle.

Transparency refers to how much information about the system and its outputs is available to users interacting with the system. It entails providing insight into the development, training, operation, and deployment of AI systems within

[12] https://digital-strategy.ec.europa.eu/en/library/ethics-guidelines-trustworthy-ai

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

a specific domain. It does not necessarily require the disclosure of complex proprietary code or data. *Explainability* focuses on making agent outputs and reasoning comprehensible to users with different expertise levels. *Fairness* ensures that agent decisions and interactions remain unbiased and inclusive across all demographic and professional groups. *Bias* across multi-step agent workflows, also known as bias compounding, occurs when small biases introduced in early stages accumulate and amplify throughout the workflow, ultimately leading to larger systemic bias in the final output. *Ethics* principles reinforce system alignment to human values such as ethics and morality. *Accountability* refers to establishing clear responsibility for different outcomes of the AI system.

Overview and Comparison. In terms of *transparency*, there are several aspects that are partially covered. In order not to repeat information, we will directly provide the overview and compare the frameworks. With respect to datasets, none of the frameworks report any data used for training. The only framework that mentions the use of prior data through so-called iterative self-improvement mechanisms is MetaGPT, which, however, does not disclose it. AgileCoder is the only framework disclosing its custom test dataset (although explicitly mentioning, neither MetaGPT nor ChatDev is disclosing theirs). Logging mechanism is implemented across different frameworks, but ChatDev's and AgileCoder's logs are much better structured and more readable than those of MetaGPT and HyperAgent. Likewise, prompt transparency is much higher in ChatDev and AgileCoder, where they are configured in a single file. On the contrary, MetaGPT's prompts are spread across different scripts and sometimes also difficult to trace. Unlike MetaGPT, ChatDev and AgileCoder, where agent responsibilities are distributed clearly in pre-defined roles, in HyperAgent, there exist different sub-roles of the agents that are not even mentioned in the paper describing the framework, but can be observed from the logs. Contrary to all of these, transparency in AgentLite relies on the developer's implementation choices.

In terms of *explainability*, although frameworks offer collaboration trails documented in logs, their internal reasoning remains hidden, and they cannot provide an explanation of their decision. This particularly holds for MetaGPT, where additionally, the communication between agents relies on documents and diagrams rather than raw dialogue. On the contrary, in case of ChatDev and AgileCoder, thanks to the natural language used in a "live" chat chain between the agent, some insights regarding decisions and clarifications can be obtained.

What concerns *ethics*, none of the considered frameworks implements any specific mechanisms for enforcing it. While ethical concerns are mentioned (only) in MetaGPT, these actually refer to other types of concerns, such as privacy, security and other socially-oriented dimensions like programming skill democratization and software development labour market.

With respect to *fairness and bias*, in theory, every user, regardless of demographic background, can potentially intervene at different checkpoints within any of the considered frameworks (for more details on HITL and autonomy levels per framework see Q11). Nevertheless, none of the considered frameworks addresses bias monitoring, the effects of bias compounding in a multi-step agent workflow, or explicitly enforces fairness. The treatment and control of these key principles depend on explicit human intervention through prompt design and careful human supervision. Similarly, none of the considered frameworks monitors or implements any specific mechanism for enforcing *accountability*. Accountability is explicitly mentioned only in MetaGPT, stating that human engineers, as framework users, are accountable for the obtained results.

Identified Gaps and Potential Improvements. Considered frameworks support some aspects of responsible AI, but they are not comprehensible and lack specific requirements to become fully responsible frameworks. One possible way to overcome this gap is to resort to the RACI (Responsible–Accountable–Consulted–Informed) (Dubinsky et al., 2010) collaboration model, which by design ensures that interactions between human and AI agent remain transparent, traceable, explainable, and inclusive throughout the software development lifecycle. As an alternative, each of the mentioned principles can be addressed independently (or with some overlaps between principles but less holistically), following the existing best practices from the literature.

In terms of *transparency*, the LLM-MAS framework (de Cerqueira et al., 2024) introduces transparency through structured multi-agent debates, where each agent presents and justifies its reasoning over multiple discussion rounds. This process exposes the logic behind decisions and allows external observers to trace the ethical and technical rationale. The ALI-Agent framework (Wang et al., 2024) ensures transparency through its two-stage evaluation process: emulation and refinement, where users can inspect how certain scenarios are generated, refined, and assessed.

With respect to *interpretability*, LLM-MAS (de Cerqueira et al., 2024) reinforces it through thematic and comparative analysis of agent debates, helping users from diverse backgrounds grasp how different types of issues

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

are detected and addressed. ALI-Agent enhances interpretability by providing scenario explanations and a memory module.

To ensure *fairness* and reduce *bias*, LLM-MAS (de Cerqueira et al., 2024) incorporates diverse agent roles and viewpoints in its debate structure, minimizing dominance of a single perspective. A Unified Framework proposed by Ranjan et al. (2025) uses agents with different roles (like checking for fairness) to participate in structured debates that help identify where bias might appear or grow during the workflow.

To develop an *ethical* agent, it is necessary to incorporate Moral Alignment or Intrinsic Reward with the LLMs by training them with rewards based on moral values, specifically those required in SDLC. Several mechanisms have been proposed for ethical AI agents that could be integrated into the AI agents for SDLC. Lei et al. (2024) proposed FairMindSim, a Belief-Reward Alignment Model to analyse the moral and challenging situations, e.g., behavior, emotions and beliefs between human and LLM agents. Schuett et al. (2025) proposed a human-centred institutional mechanism for designing and setting up an AI ethics board to reduce AI-related risks and promote ethical practices. Reinforcement Learning from Human Feedback (RLHF) and Direct Preference Optimization (DPO) are two popular methods that can be considered for ethical agents with agency and autonomy (Tennant et al., 2025).

When it comes to *accountability*, it is important to also consider its dependence on traceability. In theory, within each of the considered frameworks, agents are designed to be traceable to a certain degree. This, to some extent, permits tracking decisions and errors back to the specific agent role or action responsible. However, in practice, identifying the accountable party would require considerable effort to detect all relevant prompts, analyze their subordinate instructions, and the logs. The latter is particularly cumbersome due to the different levels of detail and structuring of logging mechanisms, as already mentioned. One potential way to overcome this problem is by introducing an explicit agent that monitors the other agents' behaviour and intervenes, in real time, in case it detects any malicious behavior, as suggested by Tamang and Bora (2025), and performed by their so-called "Enforcement Agent". This would enable traceability of the decisions and mistakes back to a specific agent.

7. Conclusions and perspectives

This study presented a research question–driven analysis of five multi-agent frameworks designed to support different phases of the software development lifecycle. By examining architectural, technological, operational, human interaction, and responsible AI aspects, the analysis aimed to provide a structured overview of their design choices, capabilities, and limitations. These frameworks represent remarkable contributions towards more autonomous, multi-agent-based software development. However, several drawbacks are still evident.

First, these frameworks do not encompass all the phases of SDLC. Second, they are not evaluated on complex, long-term real-life software development projects. Third, some of the frameworks still heavily rely on closed LLMs, which is problematic in settings where data privacy, cost, and auditability matter. Moreover, research demonstrates that locally adapting open models provides better data control, superior performance, and lower costs compared to privacy-preserving methods for closed models (Hanke et al., 2024). Additionally, using open LLMs contributes to the greater transparency and independent oversight of the system as a whole, another point where considered frameworks could remarkably improve. In fact, with respect to trustworthy and responsible AI aspects, these frameworks are still lacking maturity, since the coping mechanisms for several important aspects, such as fairness and bias, ethics and accountability, are not put in place. Nevertheless, establishing such mechanisms is indispensable as many of these problems inevitably occur (for example, bias arises from training data, fine-tuning processes, or even human evaluators themselves). This might include additional monitoring agent(s) (such as the so-called, "watchdog agent"), designed to observe the performance of other agents, detect anomalies or unproductive loops, enforce constraints, prevent critical external adversarial threats and trigger recovery actions.

Finally, although some of these frameworks include mechanisms that allow them to be easily adapted, they generally offer limited flexibility for the user because the workflows are largely automated. The latter undermines human agency and hampers effective oversight. Although HITL strategies cannot fully eliminate issues like bias or hallucinations, incorporating human expertise has shown potential in mitigating, among others, these problems. Finding a practical balance between automation and human oversight is not easy, but it is a crucial task as the excessive dependence on human feedback can make the process both expensive and inefficient. A promising future direction may be to increase the autonomy of agents focused on coding-related tasks, such as code generation or deployment, while humans concentrate on those that require stronger human agency, such as planning and evaluation. Moreover, explicit human

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

intervention to address responsible aspects, such as bias, might not be the most optimal: for high-level autonomy, an algorithmic approach must be used by agents.

8. Acknowledgments

This work has been supported by funding from the HIVE- MIND project – Horizon Europe call HORIZON-CL4-2024- DIGITAL-EMERGING-01 under Grant Agreement Number 101189745.

CRediT authorship contribution statement

Sandra Mitrović: Conceptualization; Methodology; Investigation; Writing – original draft: Q2, Q6, Q7, Q13, Introduction, Conclusion; Validation: Q1, Q3, Q4, Q8, Q9, Q11, Q12, Q14; Writing – review editing; Project administration; Supervision. **Vincenzo Giuffrida:** Conceptualization; Methodology; Visualization; Investigation; Writing – original draft: Q1, Q3, Q11, Q12; Validation: Q2, Q5, Q6, Q10; Writing – review editing. **Fabio Barth:** Conceptualization; Investigation & Writing – original draft: Q8, Q9, Q10; Validation: Q7; Writing – review editing. **Alba Bonet-Jover:** Conceptualization; Investigation & Writing – original draft: Q11, Q12, Conclusion; Validation: Q13; Writing – review editing. **Lidia López:** Conceptualization; Investigation & Writing – original draft: Q4, Q5, Conclusion; Writing – review editing. **Shaibal Barua:** Conceptualization; Investigation & Writing – original draft: Q13, Q14, Conclusion. **Santiago Massé:** Conceptualization; Investigation & Writing – original draft: Q1; Writing – review editing; Funding acquisition. **Estela Saquete:** Conceptualization; Writing – review editing. **Aitana Delgado:** Writing – review editing. **Mobyen Uddin Ahmed:** Conceptualization; Investigation & Writing – original draft: Q13, Q14; Funding acquisition. **Shahina Begum:** Conceptualization; Funding acquisition. **Georg Rehm:** Conceptualization; Funding acquisition. **Matteo Salani:** Conceptualization; Methodology; Writing – original draft: Introduction, Conclusion; Validation; Writing – review & editing; Project administration; Supervision; Funding acquisition.

References

Abran, A., Khelifi, A., Suryn, W., Seffah, A., 2003. Usability meanings and interpretations in iso standards. Software Quality Journal 11, 325–338. URL: http://dx.doi.org/10.1023/A:1025869312943, doi:10.1023/a:1025869312943.

Agnihotri, M., Chug, A., 2020. A systematic literature survey of software metrics, code smells and refactoring techniques. JIPS (Journal of Information Processing Systems) 16, 915–934.

Anthropic, . The claude 3 model family: Opus, sonnet, haiku. URL: https://api.semanticscholar.org/CorpusID:268232499.

Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., Sutton, C., 2021. Program synthesis with large language models. URL: https://arxiv.org/abs/2108.07732, arXiv:2108.07732.

Bakouch, E., Ben Allal, L., Lozhkov, A., Tazi, N., Tunstall, L., Patiño, C.M., Beeching, E., Roucher, A., Reedi, A.J., Gallouédec, Q., Rasul, K., Habib, N., Fourrier, C., Kydlicek, H., Penedo, G., Larcher, H., Morlon, M., Srivastav, V., Lochner, J., Nguyen, X.S., Raffel, C., von Werra, L., Wolf, T., 2025. SmolLM3: smol, multilingual, long-context reasoner. https://huggingface.co/blog/smollm3.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J.D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., Amodei, D., 2020a. Language models are few-shot learners, in: Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., Lin, H. (Eds.), Advances in Neural Information Processing Systems, Curran Associates, Inc.. pp. 1877–1901. URL: https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfcb4967418bfb8ac142f64a-Paper.pdf.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J.D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al., 2020b. Language models are few-shot learners. Advances in neural information processing systems 33, 1877–1901.

de Cerqueira, J.A.S., Agbese, M., Rousi, R., Xi, N., Hamari, J., Abrahamsson, P., 2024. Can we trust ai agents? a case study of an llm-based multi-agent system for ethical ai. arXiv preprint arXiv:2411.08881 .

Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H.P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F.P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W.H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A.N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., Zaremba, W., 2021. Evaluating large language models trained on code. URL: https://arxiv.org/abs/2107.03374, arXiv:2107.03374.

Chen, W., Su, Y., Zuo, J., Yang, C., Yuan, C., Chan, C.M., Yu, H., Lu, Y., Hung, Y.H., Qian, C., Qin, Y., Cong, X., Xie, R., Liu, Z., Sun, M., Zhou, J., 2024. Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors, in: Kim, B., Yue, Y., Chaudhuri, S., Fragkiadaki, K., Khan, M., Sun, Y. (Eds.), International Conference on Learning Representations, pp. 20094–20136. URL: https://proceedings.iclr.cc/paper_files/paper/2024/file/578e65cdee35d00c708d4c64bce32971-Paper-Conference.pdf.

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., et al., 2023. Palm: scaling language modeling with pathways. J. Mach. Learn. Res. 24.

Darwish, A.M., Rashed, E.A., Khoriba, G., 2025. Mitigating llm hallucinations using a multi-agent framework. Information 16, 517.

Dubinsky, Y., Yaeli, A., Kofman, A., 2010. Effective management of roles and responsibilities: Driving accountability in software development teams. IBM Journal of Research and Development 54, 4:1–4:11. doi:10.1147/JRD.2009.2039896.

Feng, K., McDonald, D.W., Zhang, A.X., 2025. Levels of autonomy for ai agents. arXiv preprint arXiv:2506.12469 .

Ferrara, E., 2023. Should chatgpt be biased? challenges and risks of bias in large language models. First Monday 28. URL: https://firstmonday.org/ojs/index.php/fm/article/view/13346, doi:10.5210/fm.v28i11.13346.

Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., et al., 2024. The llama 3 herd of models. URL: https://arxiv.org/abs/2407.21783, arXiv:2407.21783.

Guo, D., Zhu, Q., Yang, D., Xie, Z., Dong, K., Zhang, W., Chen, G., Bi, X., Wu, Y., Li, Y.K., Luo, F., Xiong, Y., Liang, W., 2024a. Deepseek-coder: When the large language model meets programming – the rise of code intelligence. URL: https://arxiv.org/abs/2401.14196, arXiv:2401.14196.

Guo, T., Chen, X., Wang, Y., Chang, R., Pei, S., Chawla, N.V., Wiest, O., Zhang, X., 2024b. Large language model based multi-agents: A survey of progress and challenges, in: Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI 2024, Jeju, South Korea, August 3-9, 2024, ijcai.org. pp. 8048–8057. URL: https://www.ijcai.org/proceedings/2024/890.

Hai, N.L., Nguyen, D.M., Bui, N.D.Q., 2025. On the impacts of contexts on repository-level code generation, in: Chiruzzo, L., Ritter, A., Wang, L. (Eds.), Findings of the Association for Computational Linguistics: NAACL 2025, Association for Computational Linguistics, Albuquerque, New Mexico. pp. 1496–1524. URL: https://aclanthology.org/2025.findings-naacl.82/, doi:10.18653/v1/2025.findings-naacl.82.

Hanke, V., Blanchard, T., Boenisch, F., Olatunji, I., Backes, M., Dziedzic, A., 2024. Open llms are necessary for current private adaptations and outperform their closed alternatives. Advances in Neural Information Processing Systems 37, 1220–1250.

Hong, S., Zheng, X., Chen, J., Cheng, Y., Wang, J., Zhang, C., Wang, Z., Yau, S.K.S., Lin, Z., Zhou, L., 2023. Metagpt: Meta programming for multi-agent collaborative framework. arXiv preprint arXiv:2308.00352 3, 6. URL: http://cnaiwiki.com/aiweb/file/EN_METAGPT.pdf.

Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J., Wang, H., 2024. Large language models for software engineering: A systematic literature review. ACM Trans. Softw. Eng. Methodol. 33. URL: https://doi.org/10.1145/3695988, doi:10.1145/3695988.

Hou, Y., Dong, H., Wang, X., Li, B., Che, W., 2022. Metaprompting: Learning to learn better prompts, in: Proceedings of the 29th international conference on computational linguistics, pp. 3251–3262.

Huang, D., Zhang, J.M., Luck, M., Bu, Q., Qing, Y., Cui, H., 2023. Agentcoder: Multi-agent-based code generation with iterative testing and optimisation. arXiv preprint arXiv:2312.13010 .

Irugalbandara, C., Mahendra, A., Daynauth, R., Arachchige, T.K., Dantanarayana, J., Flautner, K., Tang, L., Kang, Y., Mars, J., 2024. Scaling down to scale up: A cost-benefit analysis of replacing openai's llm with open source slms in production, in: 2024 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS), IEEE. pp. 280–291.

Jiang, A.Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D.S., de las Casas, D., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., Lavaud, L.R., Lachaux, M.A., Stock, P., Scao, T.L., Lavril, T., Wang, T., Lacroix, T., Sayed, W.E., 2023. Mistral 7b. URL: https://arxiv.org/abs/2310.06825, arXiv:2310.06825.

Just, R., Jalali, D., Ernst, M.D., 2014. Defects4j: a database of existing faults to enable controlled testing studies for java programs, in: Proceedings of the 2014 International Symposium on Software Testing and Analysis, Association for Computing Machinery, New York, NY, USA. p. 437–440. URL: https://doi.org/10.1145/2610384.2628055, doi:10.1145/2610384.2628055.

Khojah, R., de Oliveira Neto, F.G., Mohamad, M., Leitner, P., 2025. The impact of prompt programming on function-level code generation. IEEE Transactions on Software Engineering .

Lei, Y., Liu, H., Xie, C., Liu, S., Yin, Z., Chen, C., Li, G., Torr, P., Wu, Z., 2024. Fairmindsim: Alignment of behavior, emotion, and belief in humans and llm agents amid ethical dilemmas. arXiv preprint arXiv:2410.10398 .

Li, G., Hammoud, H., Itani, H., Khizbullin, D., Ghanem, B., 2023. Camel: Communicative agents for" mind" exploration of large language model society. Advances in Neural Information Processing Systems 36, 51991–52008.

Liao, C.C., Liao, D., Gadiraju, S.S., 2025. AgentMaster: A multi-agent conversational framework using A2A and MCP protocols for multimodal information retrieval and analysis, in: Habernal, I., Schulam, P., Tiedemann, J. (Eds.), Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, Association for Computational Linguistics, Suzhou, China. pp. 52–72. URL: https://aclanthology.org/2025.emnlp-demos.5/, doi:10.18653/v1/2025.emnlp-demos.5.

Liu, X., Yu, H., Zhang, H., Xu, Y., Lei, X., Lai, H., Gu, Y., Ding, H., Men, K., Yang, K., Zhang, S., Deng, X., Zeng, A., Du, Z., Zhang, C., Shen, S., Zhang, T., Su, Y., Sun, H., Huang, M., Dong, Y., Tang, J., 2024a. Agentbench: Evaluating llms as agents, in: Kim, B., Yue, Y., Chaudhuri, S., Fragkiadaki, K., Khan, M., Sun, Y. (Eds.), International Conference on Learning Representations, pp. 52989–53046. URL: https://proceedings.iclr.cc/paper_files/paper/2024/file/e9df36b21ff4ee211a8b71ee8b7e9f57-Paper-Conference.pdf.

Liu, Z., Yao, W., Zhang, J., Yang, L., Liu, Z., Tan, J., Choubey, P.K., Lan, T., Wu, J., Wang, H., Heinecke, S., Xiong, C., Savarese, S., 2024b. AgentLite: A Lightweight Library for Building and Advancing Task-Oriented LLM Agent System. URL: http://arxiv.org/abs/2402.15538, doi:10.48550/arXiv.2402.15538. arXiv:2402.15538 [cs].

Nguyen, M.H., Phan Chau, T., Nguyen, P.X., Bui, N.D.Q., 2025. Agilecoder: Dynamic collaborative agents for software development based on agile methodology, in: 2025 IEEE/ACM Second International Conference on AI Foundation Models and Software Engineering (Forge), pp. 156–167. doi:10.1109/Forge66646.2025.00026.

Nijkamp, E., Pang, B., Hayashi, H., Tu, L., Wang, H., Zhou, Y., Savarese, S., Xiong, C., 2022. Codegen: An open large language model for code with multi-turn program synthesis, in: International Conference on Learning Representations. URL: https://api.semanticscholar.org/CorpusID:252668917.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

OECD, 2024. Oecd principles on artificial intelligence (updated may 2024). https://www.oecd.org/en/topics/sub-issues/ai-principles.html. Updated international standard on trustworthy AI adopted in May 2024; includes values-based principles and recommendations for policymakers.

OpenAI, Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., et al., 2024. Gpt-4 technical report. URL: https://arxiv.org/abs/2303.08774, arXiv:2303.08774.

Osika, A., 2023. Gpt-engineer. URL: https://github.com/AntonOsika/gpt-engineer.

Phan, H.N., Nguyen, T.N., Nguyen, P.X., Bui, N.D.Q., 2024. HyperAgent: Generalist Software Engineering Agents to Solve Coding Tasks at Scale. URL: https://arxiv.org/abs/2409.16299, doi:10.48550/ARXIV.2409.16299.

Qi, P., 2025. Why you should stop using hotpotqa for ai agents evaluation in 2025. URL: https://qipeng.me/blog/stop-using-hotpotqa/.

Qian, C., Liu, W., Liu, H., Chen, N., Dang, Y., Li, J., Yang, C., Chen, W., Su, Y., Cong, X., Xu, J., Li, D., Liu, Z., Sun, M., 2024. ChatDev: Communicative agents for software development, in: Ku, L.W., Martins, A., Srikumar, V. (Eds.), Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Association for Computational Linguistics, Bangkok, Thailand. pp. 15174–15186. URL: https://aclanthology.org/2024.acl-long.810/, doi:10.18653/v1/2024.acl-long.810.

Rahman, M., Khatoonabadi, S., Abdellatif, A., Shihab, E., 2024. Automatic detection of llm-generated code: A case study of claude 3 haiku. URL: https://arxiv.org/abs/2409.01382, arXiv:2409.01382.

Ranjan, R., Gupta, S., Singh, S.N., 2025. Fairness in agentic ai: A unified framework for ethical and equitable multi-agent system. arXiv preprint arXiv:2502.07254 .

Scao, T.L., Fan, A., Akiki, C., Pavlick, E., Ilić, S., Hesslow, D., Castagné, R., et. al, 2023. Bloom: A 176b-parameter open-access multilingual language model. URL: https://arxiv.org/abs/2211.05100, arXiv:2211.05100.

Schuett, J., Reuel, A.K., Carlier, A., 2025. How to design an ai ethics board. AI and Ethics 5, 863–881.

Significant Gravitas, 2023. AutoGPT. URL: https://github.com/Significant-Gravitas/AutoGPT.

Stone, P., Veloso, M., 2000. Multiagent systems: A survey from a machine learning perspective. Autonomous Robots 8, 345–383.

Takerngsaksiri, W., Pasuksmit, J., Thongtanunam, P., Tantithamthavorn, C., Zhang, R., Jiang, F., Li, J., Cook, E., Chen, K., Wu, M., 2025. Human-in-the-loop software development agents, in: 2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), pp. 342–352. doi:10.1109/ICSE-SEIP66354.2025.00036.

Tamang, S., Bora, D.J., 2025. Enforcement agents: Enhancing accountability and resilience in multi-agent ai frameworks. arXiv preprint arXiv:2504.04070 .

Team, G., Anil, R., Borgeaud, S., Alayrac, J.B., Yu, J., Soricut, R., et al., 2025. Gemini: A family of highly capable multimodal models. URL: https://arxiv.org/abs/2312.11805, arXiv:2312.11805.

Tennant, E., Hailes, S., Musolesi, M., 2025. Moral alignment for LLM agents, in: The Thirteenth International Conference on Learning Representations.

Wang, H., Zhang, A., Duy Tai, N., Sun, J., Chua, T.S., et al., 2024. Ali-agent: Assessing llms' alignment with human values via agent-based evaluation. Advances in Neural Information Processing Systems 37, 99040–99088.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q.V., Zhou, D., et al., 2022. Chain-of-thought prompting elicits reasoning in large language models. Advances in neural information processing systems 35, 24824–24837.

Wu, N., Gong, M., Shou, L., Liang, S., Jiang, D., 2023. Large language models are diverse role-players for summarization evaluation, in: CCF international conference on natural language processing and Chinese computing, Springer. pp. 695–707.

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., et al., . Autogen: Enabling next-gen llm applications via multi-agent conversations, in: First Conference on Language Modeling.

Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W., Salakhutdinov, R., Manning, C.D., 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering, in: Riloff, E., Chiang, D., Hockenmaier, J., Tsujii, J. (Eds.), Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, Brussels, Belgium. pp. 2369–2380. URL: https://aclanthology.org/D18-1259/, doi:10.18653/v1/D18-1259.

Yao, S., Chen, H., Yang, J., Narasimhan, K., 2022a. Webshop: towards scalable real-world web interaction with grounded language agents, in: Proceedings of the 36th International Conference on Neural Information Processing Systems, Curran Associates Inc., Red Hook, NY, USA.

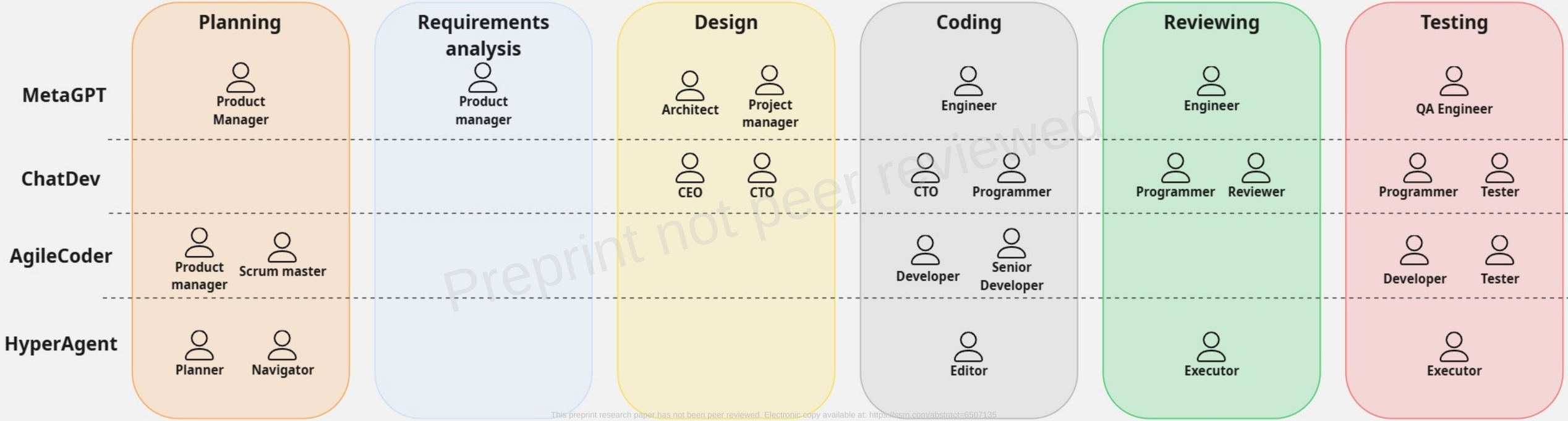
Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K.R., Cao, Y., 2022b. React: Synergizing reasoning and acting in language models, in: The eleventh international conference on learning representations.

Zhang, J., Lan, T., Zhu, M., Liu, Z., Hoang, T., Kokane, S., Yao, W., Tan, J., Liu, Z., Feng, Y., Niebles, J.C., Heinecke, S., Wang, H., Savarese, S., Xiong, C., 2025. xLAM: A family of large action models to empower AI agent systems, in: Chiruzzo, L., Ritter, A., Wang, L. (Eds.), Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), Association for Computational Linguistics, Albuquerque, New Mexico. pp. 11583–11597. URL: https://aclanthology.org/2025.naacl-long.578/, doi:10.18653/v1/2025.naacl-long.578.

Zheng, Q., Xia, X., Zou, X., Dong, Y., Wang, S., Xue, Y., Shen, L., Wang, Z., Wang, A., Li, Y., Su, T., Yang, Z., Tang, J., 2023. Codegeex: A pre-trained model for code generation with multilingual benchmarking on humaneval-x, in: Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Association for Computing Machinery, New York, NY, USA. p. 5673–5684. URL: https://doi.org/10.1145/3580305.3599790, doi:10.1145/3580305.3599790.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135



Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

Multi-Agent Systems for Software Development: a Multi-Faceted Research Question-driven Reference Guide

Author1[a,*], Author2[a], Author3[b], Author4[c], Author4[d], Author5[e], Author6[f], Author7[c], Author8[f], Author9, Author10[e], Author11[b] and Author12[a]

ARTICLE INFO

Keywords:
Multi-Agent Systems
Large Language Models
Software Development Lifecycle
Autonomous Agents
Agentic Frameworks
LLM-Based Agents
Code Generation

ABSTRACT

Multi-agent systems (MAS) based on large language models (LLMs) are on the rise due to their improved capabilities compared to single agents. In particular, among the emerging MAS for software development, those establishing automated agentic workflows contribute to the idea that software development is an essentially solved task. This study aims at critically assessing five MAS for software development that cover multiple phases and support automated workflow: MetaGPT, ChatDev, AgileCoder, HyperAgent and AgentLite. Here, 14 research questions are identified, specifically referring to architectural-, technical-, operational-, human interaction- and responsible AI-aspects. The findings from the study can help interested researchers and practitioners to understand which design choices bring advantages for particular aspects and where, on the contrary, existing solutions are suboptimal and what could be suggestions for addressing the gaps. Due to its research question-oriented layout, this paper is ideal as a reference which, for a particular aspect, facilitates grasping the overview of main design choices, capabilities, limitation and reconciliation approaches.

1. Introduction

Multi-agent systems (MAS) have become a foundational paradigm in distributed artificial intelligence (AI) due to their capability to coordinate autonomous entities collaboratively to work on and eventually successfully solve complex problems. MAS have demonstrated emerging and widespread applicability across diverse domains such as robotics, traffic management, healthcare, energy systems, and smart logistics, where autonomous agents are able to collaboratively achieve goals that are difficult for single systems to handle alone (Stone and Veloso, 2000).

With regard to the software development lifecycle (SDLC), MAS have been explored to manage complex workflows, with agents orchestrating tasks ranging from requirements analysis to testing and deployment. Several frameworks have been proposed to support SDLC, including AgentLite (Liu et al., 2024b), AgileCoder (Nguyen et al., 2025), ChatDev (Qian et al., 2024), HyperAgent (Phan et al., 2024), MetaGPT (Hong et al., 2023), AutoGen (Wu et al.), CAMEL (Li et al., 2023), and AgentCoder (Huang et al., 2023).

In this paper a comprehensive and critical examination is conducted on how MAS implement different phases of the SDLC and analyze the assumptions on which their foundations and architecture workflows have been built. In particular, this work focuses on frameworks that (i) have been designed explicitly for supporting the SDLC, and that (ii) implement autonomous agents covering multiple SDLC phases beyond *coding*. Based on these two criteria, AgentLite, AgileCoder, ChatDev, HyperAgent and MetaGPT are selected for in-depth analyses as they support end-to-end or multi-phase SDLC. On the other hand, AutoGen and CAMEL are out of scope as they are general-purpose MAS encapsulated in the frameworks HyperAgent, ChatDev and AgileCoder that are, in fact, analysed (see above). AgentCoder is excluded, too, as it targets the coding and testing phases, without providing support for other SDLC phases.

Despite the recent proliferation of MAS for SDLC, the literature currently focuses on presenting new frameworks and variants rather than analyzing and comparing their capabilities, limitations, and design trade-offs.

The general idea of having agents which are capable of implementing an entire software development project on their own or with minimal user intervention, contributes to the hype that coding has been resolved as a generative task. With this study we would like to perform a targeted overview underpinned by a critical assessment of these frameworks.

*Corresponding author
email1 (Author1); email2 (Author2); email3 (Author3)
ORCID(s):

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

This study provides a multi-faceted assessment of these frameworks, examining *architectural, technological, operational, human interaction and responsible AI-related* aspects each organized in a dedicated section. Within each of these aspects, we propose several research questions (RQs) and address them providing:

- Contextualization of the RQ: providing motivation and, if necessary, definitions of the terminology used;
- Research Question-Focused Overview: positioning each of the considered frameworks with a strict focus on this concrete RQ;
- Comparison: comparing frameworks among themselves;
- Identified Gaps and Potential Improvements: a critical reflection emphasizing found gaps and indicating possible improvements.

The contribution of this study consists of a structured and critical analysis of key aspects related to multi-agent based software development as implemented in the selected frameworks. These analyses often go beyond the claims made in the original papers and, where necessary and possible, the analysis is supported by a careful examination of the associated code repositories.

Due to its transparent and research question-oriented organization, this study allows interested researchers and practitioners to use it as a reference and quickly to grasp the key design choices, advantages, disadvantages, and possible improvements of a particular aspect of interest.

2. Architectural aspects

2.1. SDLC models and workflows

Q1. *Which phases of the SDLC does the framework cover? What roles are defined within each phase, and how these roles are mapped to agents, for example, is there a single agent per role or are roles shared among multiple agents? Additionally, which SDLC models (e.g., waterfall, agile) does the framework support?*

Contextualization. The SDLC is a structured process that defines the phases, roles, and workflows involved in planning, creating, testing, deploying, and maintaining software systems. The most common SDLC are the Waterfall model, that follows a linear, sequential progression through distinct phases, with each stage completed in full before the next begins; and the Agile model, which adopts an iterative, incremental approach that delivers functional software in short cycles, enabling continuous feedback and adaptation. DevOps integrates development and operations practices to achieve continuous integration, delivery, and deployment through automation and close cross-functional collaboration.

Overview. The investigated multi-agent frameworks differ in the extent of SDLC coverage, the roles they define, their role-to-agent mapping strategies, the SDLC model they emulate (e.g., Waterfall, Agile), and the DevOps operations they cover. Most frameworks define roles explicitly, often using a one-to-one mapping between roles and agents, and operate in modular, phase-aligned workflows.

MetaGPT covers the full SDLC using a waterfall-like sequential model. It defines roles such as Product Manager, Architect, Project Manager, Engineer, and QA Engineer. Each role is instantiated as a separate agent with its own goal, constraints, and behaviours. The framework uses a one-to-one mapping, where every agent acts in a single role, with communication structured according to Standard Operating Procedures (SOPs) and reinforced by a shared message environment.

Similarly, ChatDev adheres to a waterfall model and emulates a software company where each phase, from requirements analysis, system design, coding, and testing, is handled by dual-agent interactions. Roles include CEO, CTO, Programmer, Reviewer, and Tester. A typical structure involves hierarchical instructor-assistant agent pairs. While there is some reuse of roles across phases, each role is still mapped to a distinct agent instance per phase context.

In contrast, AgileCoder follows the Agile methodology, organising work into sprints with Planning, Development, Testing, and Review phases. Roles include Product Manager, Scrum Master, Developer, Senior Developer, and Tester. Each role corresponds to a dedicated agent, supporting a single-agent-per-role strategy, and, like ChatDev, it adopts a dual-agent interaction per phase. Agents collaborate iteratively across sprints, inheriting context and outputs for continuous refinement.

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

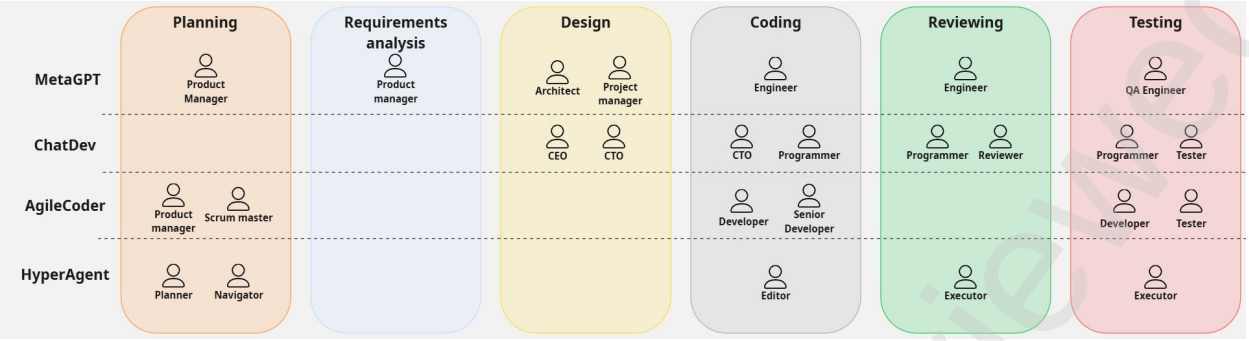


Figure 1: Phases and respective agent roles in SDLC in considered frameworks.

HyperAgent supports full SDLC automation, particularly targeting scalable and parallel task execution. It is not tightly bound to a specific SDLC model, but its structure is most compatible with task decomposition found in DevOps or hybrid workflows. It features a Planner agent, acting in a master role, and subordinate expert agents, where roles are implemented as specialised agents coordinated by the master. Role-to-agent mapping follows a hierarchical master-worker pattern.

Lastly, AgentLite allows custom SDLC implementations. In the example based on a DevOps SDLC, phases like Planning are decomposed into sub-tasks, such as requirements elicitation, backlog creation, and sprint planning. Roles need to be explicitly modeled and each role should represent a separate LLM-agent. The framework promotes one-to-one mapping between roles and agents but allows for flexible assignment. HyperAgent proposes a different approach more suitable for real case scenarios, assisting developers in integrating new functionalities in existing codebases, interacting with the repository.

Comparison. Most of the frameworks adopt a Waterfall model, while AgileCoder emulates an Agile SDLC. Although the different approaches could have their advantages and disadvantages, it is not clear which solution is more effective for autonomous software development and specific use cases. The Waterfall model provides a highly traceable workflow in which each phase is executed sequentially and uses the artefacts generated by the previous phases. The Agile methodology could introduce more complexity and costs, since the development is executed iteratively in sprints, but it could help to improve the final outcome with continuous refinement and more granular tasks decomposition. There are no significant differences regarding the SDLC phases covered by the frameworks (Figure 1). All of them perform the Planning phase, with MetaGPT and ChatDev also producing design artefacts before starting the coding phase. Coding, Reviewing and Testing are covered by most frameworks, with the exception of AgileCoder which does not include the code review phase.

Identified Gaps and Potential Improvements. The examined multi-agent frameworks demonstrate solid performance across the early and middle phases of the SDLC, particularly in planning, design, implementation, and testing. However, they provide little support for the later stages, especially deployment and maintenance. These phases are essential for real-world software delivery, encompassing continuous integration and continuous deployment CI/CD, system monitoring, rollback strategies, and long-term maintainability. Other Agile SDLCs, such as Extreme Programming (XP) and Kanban, are currently neglected by examined frameworks. Additionally, none of the considered frameworks support multiple SDLCs.

A limiting factor of the examined frameworks lies in context handling and scalability. Most frameworks rely on token-limited prompts and local working memory, which hampers their ability to reason over evolving codebases, logs, and test results in deployment settings. While AgileCoder's dynamic code graph marks progress in this direction, robust mechanisms for large-scale context grounding remain underdeveloped.

To progress toward full SDLC automation, future frameworks should explicitly extend their architecture to support deployment and maintenance workflows. This includes integrating with CI/CD pipelines, supporting test coverage monitoring, generating deployment manifests or configuration files, and implementing strategies for regression detection, rollback, and system health checks.

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

2.2. Agent architecture and communication

Q2. *What communication paradigm is used between agents, such as collaborative, competitive, or hybrid? Which interaction structure is implemented, such as hierarchical, decentralized, centralized, shared memory pool, or another architecture?*

Contextualization. In order to understand how the intra-agent communication works in different SDLC frameworks, we resort to the three aspects commonly considered when discussing interaction mechanisms in the literature on multi-agent frameworks.

The first aspect pertains to the communication paradigm (Guo et al., 2024b), which is commonly classified into three categories: competition, cooperation, and coopetition. In competition, as the name suggests, agents compete with each other. The competition can be oriented towards conflicting objectives or limited resources. In collaboration and coopetition, agents tend to have shared objectives, but the way they achieve them is different. In the former, they work together and exchange the necessary information, while in the latter, they rather compete. Hence, the coopetition is a mix of collaboration and competition, sometimes also referred to as a debate (Guo et al., 2024b), given that the agents argue on the best solution, each one defending its own standpoint before reaching a consensus.

The second aspect pertains to the interaction structure, that is, the topology of the agent communication network, which might be centralized, decentralized and layered. In the centralized architecture, one agent orchestrates the whole framework and communicates with all other agents, who do not communicate among themselves. In the decentralized architecture, communication is organized in peer-to-peer fashion, where every agent can potentially communicate with any other. In the layered architecture, agents are organized in different layers and the communication primarily takes place between agents of adjacent layers. Hierarchical structure is a special type of layered structure where higher-level layers control or manage lower-level layers.

Finally, the third aspect refers to any additional interaction mechanism that complements the interaction structure by enabling information exchange among the agents. These could be, for example, shared message pools accessible by all agents, or dedicated message queues and message streams for exchanging information between two particular agents.

Overview. All selected frameworks support several phases of SDLC are unanimously based on the collaborative agent setup. However, the underlying interaction structure varies due to the different supported SDLC phases. As such, AgentLite follows a hierarchical structure with the Manager agent assigning tasks to either individual working agents or teams of other agents. HyperAgent has a centralized interaction structure characterized by a master agent (Planner) coordinating specialized child agents. MetaGPT adheres to the layered architecture in which the sequence of layers reflects the sequence of defined subtasks, along with the corresponding roles. Agents in adjacent layers operate on equal footing, that is, they are equally important, without hierarchical relationships. Instead, ChatDev follows a layered architecture where each dual-agent interaction is hierarchically organized, using an instructor-assistant relationship. AgileCoder is similar to ChatDev, but with one main difference: the workflow in ChatDev is not fully aligned with assigned roles. For example, some roles, such as Programmer, are reused in dual-agent interactions across different phases of the process.

Different frameworks use different mechanisms to exchange information between agents. Most of the frameworks use a simple queue interaction mechanism to enable agent interactions. In AgentLite, the Managing agent and Subordinate agent communicate directly via so-called Task Packages, which represent a synchronous method of communication, to let Subordinate agents to wait for feedback. Alternatively, in AgileCoder, a Message Stream acts as a working memory between two agents; in ChatDev, dual-agent communication is part of a so-called Chat Chain. In HyperAgent, asynchronous Message Queues are used to communicate between the Master agent and its Subordinate agents. An asynchronous queue has its advantages, as it decouples communication from time and allows the master agent to proceed with its activities without waiting for the Subordinate agents to process the message. Contrary to these, MetaGPT uses a global shared message pool with a publish-subscribe mechanism. This mechanism allows for each agent to share its results with other agents by publishing them to the shared message pool, as well as to access messages shared by other agents by subscribing to them. This global shared message pool could, in theory, allow for a fully peer-to-peer interaction solution, hence depending on a specific use case needs, MetaGPT could turn into a fully decentralized framework.

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

Comparison. The above-mentioned interaction choices have different advantages and disadvantages. With respect to communication overhead, dual-agent communication is resource-demanding. In such scenarios, having predefined interactions between different agent roles, such as in AgileCoder and ChatDev, is beneficial but leads to a reduction in flexibility and major changes if there is a need to intervene on agent roles or interactions. MetaGPT, on the other hand, allows for peer-to-peer interactions and, at the same time, thanks to the global shared message pool that is accessible to all agents, reduces communication overhead. Such an architecture is also suitable for the adaptations in the communication workflow. However, neither MetaGPT nor AgileCoder/ChatDev readily enable parallel processing of subtasks and dynamic load balancing, which HyperAgent and AgentLite instead support.

The centralized and hierarchical structure permits a higher scalability compared to other types of communication paradigms. Both architectures allow independent scaling of agents with the manager agents supported by powerful models with superior reasoning capabilities, while other agents are run by smaller models. The usage of an asynchronous Message Queue enables HyperAgent to distribute tasks among several agents and resources and to readily parallelize tasks, unlike other frameworks. AgentLite demonstrates high efficiency and scalability through its hierarchical communication paradigm.

Identified Gaps and Potential Improvements. Although the various frameworks explore effective communication structures that can offer several advantages, a current gap is the lack of a standard for inter-agent communication. Emerging communication protocols between agents and external tools, such as Agent-to-Agent communication protocol (A2A)[1] and Model Context Protocol (MCP)[2], could be integrated to make these frameworks more flexible and easily customizable, while also facilitating communication with external components or other multi-agent frameworks. While research on MAS integrated with A2A and MCP is already in progress (Liao et al., 2025), SDLC-focused MAS could benefit from these protocols to enhance integration with external services. In particular, A2A could support interoperability with external agents and services, whereas MCP could enable easier and more standardized tool integration.

3. Technological aspects

3.1. Framework definition

Q3. *How do agents interact in terms of communication style, such as natural language or pseudo-machine language? What types of input and output data are supported by the framework, including text-only content, code, images, and audio?*

Contextualization. MAS inherently rely on structured information exchange to perform tasks and ensure coherent collaboration. As each agent in MAS operates autonomously with a specialized role, the format and structure of the data are fundamental to advance the workflow. LLMs enable agents to process and generate various input and output data formats beyond natural language, including images and code. Although most frameworks supporting SDLC primarily rely on text and code, they differ in the type of data processed or generated, depending on the SDLC phase and the agent involved. The choice of specific input and output formats, therefore, influences the development workflow and affects the framework coordination efficiency.

Overview. Each MAS specialized in SDLC proposes different solutions, highlighting the advantages of specific design choices. For instance, ChatDev and AgileCoder rely on a communication style based on natural language. Within each phase, two agents engage in a multi-turn dialog and work cooperatively until they achieve consensus, producing either text or code depending on the SDLC phase.

MetaGPT distinguishes itself by the use of SOPs (Standardized Operating Procedures) as the main data exchange format between agents. SOPs can contain text as well as other kinds of data, such as charts, graphs and code. Structured outputs generated by agents at different phases of the SDLC are passed as input, along with natural language text used to clarify roles and task specifications. Diagrams in MetaGPT are represented using a text-based graphical format. The Product Manager agent, for example, produces a requirements document (PRD) containing goals, user stories, and requirements, as well as a competitive quadrant chart. The Architect agent processes the PRD documents and

[1] https://a2a-protocol.org/latest/
[2] https://modelcontextprotocol.io

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

generates system architecture diagrams and code interface definitions. Code is generated by the Engineer agent and processed by the QA Engineer, who produces additional code for unit tests.

HyperAgent uses natural language but in a more structured format, consisting of two fields: Context (containing background and rationale) and Request (containing actionable instructions). The Planner agent generates strategies and tasks for other agents based on user prompts. The Navigator agent interacts with IDEs and codebases to extract the necessary code and information. Finally, the Editor agent produces the code and integrates it into existing codebase.

As a general-purpose multi-agent framework, AgentLite relies more on natural language than on code. The Individual Agent composes a prompt based on user input, which is used by the LLM to generate an executable action. The Manager Agent employs a more structured message format, called Task Package, which is used to assign sub-tasks to Individual Agents.

All considered frameworks primarily support textual data as both input and output. In the context of inter-agent communication, the textual format may be more structured, as in the case of MetaGPT and AgentLite, while code is the primary data format used during the development phase of the SDLC.

Comparison. With respect to communication type, MetaGPT differs from the other considered frameworks in that its agents rely not only on natural language but also on various types of documents to interact with one another. The use of SOPs aims to reduce ambiguities and errors in agent communication; however, the absence of an ablation study on the use of structured output leaves unclear what benefits such data formats provide in the communication process between agents. In particular, some of the artifacts produced may be of limited usefulness for specific use cases, as, for example, the competitive analysis diagram. All frameworks use code as a data format, which is the output of the development phase and the input for the code review and testing phases. HyperAgent differs from other frameworks by primarily focusing on integrating code into existing codebases rather than generating new files.

Identified Gaps and Potential Improvements. Future studies could further investigate which types of graphical and structured data might be useful to support the software development in the context of MAS for the SDLC.

The use of structured data provided through human-in-the-loop interactions is another aspect that remains largely unexplored. This could enable SDLC frameworks to become better integrated tools within complex, real-world development processes and support the generation of higher-quality code.

Moreover, interaction with development tools such as IDEs and CI/CD systems, both to generate data that can be used as input by agents and to integrate agents' output into these systems, remains only partially explored. In real-world scenarios, frameworks supporting software development should interact with the tools used by developers and ensure that the generated code integrates into complex codebases.

3.2. Agent definition and functioning

Q4. *What LLM are used as backbones to support agents' capabilities across different systems (e.g., open source or proprietary, number of parameters)? Are these fine-tuned or pre-trained?*

Contextualization. LLMs can be classified as general-purpose or domain-specific. General-purpose LLMs have been trained on massive, diverse datasets providing support to a wide range of tasks (e.g., coding, reasoning, language understanding, summarization). Examples of these include GPT (Brown et al., 2020a), Llama (Grattafiori et al., 2024), Claude (Anthropic), Mistral (Jiang et al., 2023), and Gemini (Team et al., 2025) families. Domain-specific LLMs can be divided into two subgroups: models that are fine-tuned or adapted from general-purpose foundational models, and models that are trained from scratch. Among the first group, for the software development domain, we find Gemini Code Assist[3], Codex[4], Code LLaMA[5] and DeepSeek-Coder[6] trained for coding based on Gemini, GPT-3, LLaMA-2 and DeepSeek, respectively. A smaller number of code-focused LLMs have been trained from scratch, directly on code repositories without relying on a general LLM. These models are specifically designed and optimized for source code understanding and generation, for example, CodeGen[7] is a family of models from 350M to 16B parameters for program

[3] https://codeassist.google/
[4] https://openai.com/index/openai-codex/
[5] https://huggingface.co/codellama
[6] https://deepseekcoder.github.io/
[7] https://github.com/salesforce/CodeGen

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

synthesis for Python; StarCode models[8] are 15.5B parameter models trained on 80+ programming languages from The Stack dataset[9] (v1.2); and, WizardCoder derived from StarCoder and CodeLlama, among others.

In relation to the availability of the models, there are proprietary models that are not available for download but can be used through an API, such as GPT, Claude, and Gemini. Their internal architectures, training datasets, and fine-tuning details remain undisclosed. In contrast, the so-called "open" models are available for download, for example, GPT-Neo, LLaMA, or Mistral, and these can be classified as open-source or open-weight. Open-source models (e.g., GPT-Neo) provide both the model and the necessary training details, enabling full reproducibility. With open-weight models, instead, the model is available to download (and use), but there is limited information related to the model creation (e.g., LLaMA and Mistral). The latter occasionally come with some use restrictions (e.g., non-commercial licenses). From the code-related LLMs mentioned above, Gemini Code Assist and Codex are proprietary (based on Gemini and GPT-3), the rest are open-weight (Code LLaMA3, DeepSeek-Coder, CodeGen, StarCode, and WizardCoder).

Overview. All the reviewed frameworks allow to configure several LLMs as a backbone for their agents. For AgentLite and HyperAgent, the backbone LLM is configured for each Agent, i.e., several LLMs can be used simultaneously in different agents. All the frameworks can be configured using several GPT versions from OpenAI. The GPT family provides proprietary models that are publicly accessed through an API. AgentLite, AgileCoder and HyperAgent can also use the proprietary models Claude 3, from Anthropic, also available through API. HyperAgent can use the proprietary Spark Lite model, a lightweight, general-purpose version of Spark from iFlytek. AgentLite, HyperAgent and MetaGPT can use open models (i.e., open-source or open-weight); AgentLite uses its own xLAM model (Zhang et al., 2025), HyperAgent can use LLaMA, an open-weight model from Meta, which can be downloaded, but the license comes with some use restrictions; and MetaGPT can be used with DeepSeek Coder 33B from DeepSeek AI. xLAM (advanced Large Action Model) has been trained on agent action trajectories from multiple environments, built based on the Open Source model Mixtral 8x7b Mix-of-Expert. The three models, DeepSeek, LLaMA, and xLAM, can be found in HuggingFace platform (note that xLAM has a restricted, non-commercial use). Overall, proprietary GPT-family models dominate across frameworks, while support for open-weight alternatives remains uneven and often secondary, Table 1 includes information on the LLMs used by the revised frameworks.

Framework publications provide limited information about the specific configuration of backbone LLMs. However, examining their publicly available source code repositories on GitHub, reveals that several frameworks expose inference-level configuration parameters. Although the range and granularity of configuration options differ significantly across systems, AgentLite allows adjustment of temperature, stopping sequences, and the maximum number of generated tokens. AgileCoder provides more extensive controls, including temperature, nucleus sampling (top-p), number of output choices, streaming, stop sequences, maximum tokens, frequency and presence penalties, logit bias, and user identifiers. ChatDev exposes only temperature and maximum token length, whereas HyperAgent only supports temperature configuration. In contrast, MetaGPT provides the most configuration options, including temperature, top-p, top-k, repetition, presence and frequency penalties, maximum tokens, stopping sequences, streaming, log probabilities, best-of sampling, reasoning-specific token budgets, and system prompt control.

Comparison. There are two main differences between the considered frameworks with respect to the backbone LLMs: using one or more backbone LLMs and the use of open models. AgileCoder, ChatDev, and MetaGPT use the same LLM as a backbone for different agents. HyperAgent uses LLMs of different skills and performances for different agent roles, e.g. the Planner agent needs reasoning skills, and the CodeEditor and Executor agents need more powerful LLMs than the Navigator agent. AgentLite, in theory, supports using different LLMs per agent, although these need to be configured. Using a single LLM across heterogeneous agent roles may simplify configuration but risks suboptimal specialization, especially when reasoning, planning, and code synthesis require different capabilities.

Regarding the use of open models, not all frameworks support them. More specifically, AgileCoder and ChatDev support only proprietary models. Proprietary models often lead in benchmarks and offer high reliability of service and ease of use via API access, but they come with high costs, data privacy issues, limited transparency, and vendor lock-in. In contrast, open models - ranging from open-weight releases to fully open-source models - offer greater transparency, local deployability, and freedom for fine-tuning and auditing, often at no cost. However, they may lag slightly in raw performance or require more technical effort to operate at scale.

[8] https://huggingface.co/bigcode/starcoder
[9] https://huggingface.co/datasets/bigcode/the-stack

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

Table 1
Comparison of used LLMs across considered multi-agent frameworks.

ModelType	LLM	AgentLite	AgileCoder	ChatDev	HyperAgent	MetaGPT
Open-source	xLAM-v0.1-r	✓				
Open-weight	DeepSeek Coder 33B					✓
	Spark Lite					
	Llama-3-70B				✓	
	Llama-3-8B				✓	
	Llama-4-Scout-17B-16E					✓
	Llama-4-Maverick-17B-128E					✓
Proprietary	Claude-3-Sonnet				✓	
	Claude-3-Haiku		✓		✓	
	gpt-3.5-turbo	✓	✓			✓
	gpt-3.5-turbo-16k	✓		✓		
	gpt-3.5-turbo-16k-0613	✓		✓		✓
	gpt-4	✓	✓	✓		
	gpt-4-0613	✓				✓
	gpt-4-turbo	✓		✓		✓
	gpt-4-32k	✓		✓		
	gpt-4-32k-0613	✓				
	gpt-4-1106-preview	✓				
	gpt-4o			✓	✓	
	gpt-4o-mini			✓		
	text-davinci-003	✓				
	text-ada-001	✓				

Identified Gaps and Potential Improvements. All compared frameworks show a clear convergence toward the use of general-purpose pre-trained LLMs; none report fine-tuning the backbone models for software engineering tasks. As a result, current systems largely rely on prompt engineering and agent orchestration rather than backbone specialization to address SDLC-specific requirements.

Future research should therefore explore the development and integration of domain-specific or fine-tuned LLMs within SDLC-oriented multi-agent frameworks. Promising directions include (a) lightweight fine-tuning or adapter-based specialization of open-source or open-weight models, particularly role-specific fine-tuning aligned with SDLC phases (e.g., requirements analysis, architectural design, testing), which could improve both task accuracy and inter-agent coordination; (b) retrieval-augmented generation leveraging project repositories, issue trackers, and technical documentation to ground agent reasoning in project-specific knowledge; and (c) the definition of standardized evaluation benchmarks that assess model effectiveness across SDLC activities rather than isolated coding tasks. Additionally, frameworks could incorporate explicit model-selection criteria or automated profiling mechanisms to support evidence-based backbone assignment per agent role.

Finally, the choice between proprietary and open LLMs involves trade-offs between performance, transparency, control, and economic sustainability. Alongside the already mentioned comparison between open and proprietary models, economic considerations also play a critical role in backbone selection decisions. An in-depth cost–benefit analyses revealed that open-source LLMs can reduce inference and licensing costs by a factor of 5× to 29× compared to proprietary APIs, especially for continuous or high-volume workloads (Irugalbandara et al., 2024). Current frameworks provide limited support for systematically evaluating such cost–performance trade-offs during framework configuration, which represents an additional open challenge for future research.

Q5. *Are the roles and the actions of the agents static or dynamic?*

Contextualization. In multi-agent frameworks for software development, distinguishing between static and dynamic agent roles is fundamental to understanding their adaptability and autonomy. Static systems define a fixed set of roles and actions, such as Project Manager, Programmer, or Reviewer that remain constant throughout the

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

development process. These agents operate according to predefined prompt templates and task sequences, which ensures reproducibility and stability but limits flexibility and context sensitivity. In contrast, dynamic frameworks introduce mechanisms that allow agents to assume new roles or modify their behavior based on evolving project contexts, task dependencies, or feedback loops. This dynamic configuration enables more context-aware collaboration and adaptive problem solving but also requires more sophisticated coordination and monitoring strategies to prevent task redundancy or role misalignment. Consequently, analyzing whether roles and actions are static or dynamic provides key insights into how current LLM-based frameworks balance control, flexibility, and autonomy across the SDLC.

Overview. AgentLite defines a BaseAgent with specific Action classes. Users can extend/customize these actions. The agents' roles are not obviously dynamically reassigned based on project state; rather users define agent behavior via prompt generation and defined action types. These behaviors suggest static role and action definitions, with some extensibility.

AgileCoder mimics Agile methodology; it uses sprints, backlog, evolving tasks, and a Dynamic Code Graph Generator that updates dependencies when the code changes. That implies actions are chosen based on the codebase context. The roles (Product Manager, Developer, Tester) are assigned, but because of the sprint and backlog structure and evolving graph context, actions are more dynamic.

ChatDev has predefined roles and structured phases of software development (e.g., a coder, tester, reviewer). These roles follow fixed prompt templates. The available information does not show role or action adaptation depending on changing internal project state (beyond what prompt engineering allows). Therefore, roles and actions seem to be static in structure.

HyperAgent seems to focus on letting users create and customize agents and actions, and offering prebuilt agents. But, we did not find evidence that it supports dynamic reassignment of roles or action selection based on evolving project context. More likely static agents whose behavior depends on user input or configuration rather than internal monitoring of project evolution.

MetaGPT has a roles system with specialized roles (Product Manager, Architect, Engineer, QA, etc.). These are somewhat static in naming. But, importantly, MetaGPT offers the ability to customize LLMs for roles or actions via configuration files, allowing different LLMs or different behavior depending on the action. That suggests dynamic assignment of behaviour in some cases. It also has an environment and memory tracking of actions, which enables some dynamic responsiveness. However, the structural roles remain fairly stable.

To sum up, ChatDev, AgentLite and HyperAgent (mostly) tend to define roles and what actions those roles perform in a fixed way. Even if the exact prompts or LLM used can change, the architecture does not adaptively change who does what mid-project (beyond user configuration). AgileCoder clearly shows more dynamic behavior (changing code dependency graph, evolving tasks, sprints). MetaGPT has dynamic components in how roles/actions use different models or configurations, but its roles are still named and defined statically in structure (though configuration files).

Comparison. In multi-agent frameworks powered by LLMs, the degree of flexibility in defining and adapting agents' roles and actions has a direct impact on collaboration, efficiency, and autonomy across the software development lifecycle. Systems adopting static roles and actions, such as ChatDev or AgentLite, typically define a fixed set of agents (e.g., Product Manager, Developer, Reviewer) and associate them with predetermined tasks and prompt templates. This static structure simplifies orchestration, ensures reproducibility, and allows for clearer traceability of responsibilities. However, it also limits adaptability, agents cannot easily reconfigure themselves or assume new functions as project requirements evolve, reducing responsiveness to contextual changes or unexpected dependencies.

Conversely, frameworks incorporating dynamic roles and actions, such as AgileCoder or partially MetaGPT, enable agents to modify their behavior, allocate new roles, or select actions based on the evolving project state. This dynamic adaptability can enhance flexibility, context-awareness, and error recovery, as agents may redistribute tasks or redefine goals in response to emerging challenges. Nevertheless, increased dynamism introduces new complexities. It complicates coordination, raises risks of overlapping responsibilities or redundant actions, and can reduce system predictability and explainability if agent behaviors evolve beyond predefined constraints.

Overall, the choice between static and dynamic configurations reflects a trade-off between control and flexibility. Static systems are easier to monitor, audit, and replicate—key advantages in regulated or safety-critical domains—, while dynamic systems better support open-ended, exploratory, or iterative software development processes.

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

Identified Gaps and Potential Improvements. None of the frameworks seems to support roles that themselves evolve or are created or destroyed dynamically in reaction to project needs, changes in domain, or failure conditions. Also lacking are frameworks where agents adapt their actions in more complex ways based on the state of the development (e.g. error rates, test failures, code coverage, feedback loops) beyond just user-defined sprints or graphs.

Future research could also explore hybrid strategies that combine the structural stability of static roles with adaptive mechanisms that adjust task allocation and reasoning depth dynamically, guided by project feedback or performance metrics.

Q6. *What is the structure of agent workspaces? Which internal capabilities do agents possess, such as memory or Retrieval-Augmented Generation (RAG) at inference time? How similar are agent workspaces?*

Contextualization. To better understand how an agent is defined and how it operates, it is important to understand its workspace and the components it contains. We define a workspace as a local, temporary environment that contains all the information related to the current development session. This includes artifacts, messages exchanged between agents, external contextual documents and, possibly, tools available to the agents. The workspace may also include a memory module, a storage component responsible for storing all necessary prior information that might be necessary to successfully perform the current action. The stored information may include user inputs, messages from other agents and/or outputs coming from the agent itself. Retrieved memory content is typically used to enrich the original prompt of the agent. Recently, this is frequently accomplished using RAG. RAG is a method which allows an LLM to generate an output combining an external knowledge with the LLM's internal knowledge. This is achieved by retrieving the most relevant external documents based on the input query and providing them as a context to that query for output generation.

Overview. Different frameworks resort to different implementations of agent workspaces. In MetaGPT, an agent workspace encompasses both file system, storing the artifacts, and a global pool of shared messages, exchanged with the other agents. Different agent workspaces are basically shared; agents access the same locations but filter the relevant information based on their specific roles. Depending on the agent role, its workspace might include tool usage (e.g. Mermaid for Architect). In ChatDev, the workspace encompasses the file system, storing the artifacts, and a Chain structure, containing phase-level messages between two agents. In AgileCoder, in addition to files, the workspace includes a Message Stream containing messages from dual-agent interactions, a code dependency graph that dynamically traces code-related dependencies and a global message pool that also serves as a status task log. In HyperAgent, the codebase and tool suite are integrated into a shared workspace along the file system and the message queue. The tool suite includes, for example, specialized tools for code navigation, formatting and visualization. In AgentLite, the workspaces of Manager and Individual agents differ, in that the Manager's workspace consists of a collection of so-called TaskPackages, which serve to trace the status of tasks that the Manager agent has delegated to Individual agents. On the other hand, an individual agent's workspace encompasses only its internal memory module, which contains the history of its own actions.

Memory module is differently implemented from one framework to another. The simplest type of memory is short-term memory, which stores information relevant for a single phase and is implemented in all five frameworks. Limited context length of LLMs makes it unfeasible to keep all messages in the working memory. Therefore, frameworks use different approaches to cope with this limitation. HyperAgent uses summarization of messages, that is, messages from child agents are shortened and included in the context. MetaGPT claims to do the same (summarization) from the past projects based on the agent role while AgileCoder uses only the final message from the message stream that contains the exchange between two agents. Unlike HyperAgent and AgentLite, where the memory remains a short-term concept, both MetaGPT and ChatDev use short-term and long-term memories. As a long-term memory, ChatDev stores only the outcome of the task as relevant information to be conserved. On the contrary, MetaGPT stores all information exchanged during an interaction in a long-term memory. It then relies on RAG for extracting the most important information from the long-term memory. More specifically, in MetaGPT, long-term memory serves as an external knowledge base from which the relevant context for subsequent actions is retrieved, typically using the most recent short-term information as a query.

In addition to the memory module, some frameworks equip their agents with tools. As such, three out of four agents in HyperAgent (Navigator, Editor and Executor) use different tools to carry out their main functionalities, such as enhancing code navigation, visualizing code structure, integrating keyword search, applying and refining

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

code patches, automatically handling syntax and indentation issues, context-aware editing, summarization, as well as accessing testing and setup documentation. MetaGPT agents also use tools; for example, a Product Manager can use web search tools, or an Architect can use Mermeid to create architecture diagrams.

Comparison. The considered frameworks differ in how they structure agentic workspaces: a workspace can be individual (agent-level) or shared, meaning that more agents can have overlapping workspaces. More specifically, in HyperAgent and AgentLite, agent workspaces are individual, and so are the underlying memory modules. On the contrary, in ChatDev and AgileCoder, the agent workspaces are overlapping during the execution of a phase/conversation as the memory (containing their interactions) is shared between two agents. Furthermore, in MetaGPT, all agent workspaces are overlapping as they share both the global message pool and the common file system.

With respect to workspace structure, agents with different roles within a framework typically do not differ in how their workspaces are structured. The only exception is AgentLite, where not all the agents possess memory: only the Individual Agent, a base agent, has a memory module. Regarding different types of memory, MetaGPT and ChatDev support long-term memory. While ChatDev resorts to storing the final (most important) outcome, MetaGPT stores everything and later filters the most relevant parts using RAG.

Identified Gaps and Potential Improvements. The considered frameworks present a variety of implementations and design practices, including the use of file systems, global message pools, message streams, message chains, and separate memory modules, as well as architectures with or without long-term memory. However, it remains unclear which of these design choices should be considered as best practices and whether their suitability depends on the specific application. In addition, there is limited understanding of how each suggested solution performs in the long run, in contexts where projects require multiple development cycles. Finally, concurrency management should be studied across different agents' workspace configurations.

Q7. *What prompting strategies are employed to guide agent behavior, and to what extent do they influence coordination among agents?*

Contextualization. Research indicates that LLM text generation is significantly influenced by the input prompts they receive, as even the small modifications in wording or structure can affect the output (Brown et al., 2020b; Wei et al., 2022). To improve input prompts, several prompting strategies have been proposed in the literature. Although these strategies can apply to various types of prompts, due to the nature of the prompts found in the LLM-based multi-agent systems discussed here, we focus specifically on natural language-like prompts, sometimes also referred to as "hard prompts" (Hou et al., 2022). Apart from simple direct prompting, these strategies also encompass more advanced approaches. Some of the exemplars include few-shot prompting (Brown et al., 2020b), chain-of-thought (CoT) (Wei et al., 2022), role-playing (Wu et al., 2023) and Reason & Act (ReAct) prompting (Yao et al., 2022b). In few-shot prompting, a few input-output pairs as examples are provided to showcase the expected model behaviour, together with an initial instruction (direct) prompt. With CoT, the model is instructed to decompose the complex task into a series of less complex, intermediate steps; in other words, it is encouraged to reason internally before providing the final answer rather than simply generating a single answer. A role-playing prompt instructs the model to impersonate a specific role and, typically, act as if it possesses the expertise of that role. ReAct interleaves reasoning with task-specific actions, which is typically performed in dynamic environments where intelligent and adaptive decision-making is critical.

Overview. The considered frameworks use different prompting strategies. MetaGPT uses ReAct prompting to let the agent reason and select the next action to be performed. MetaGPT's action-based prompts often include role-playing to encourage the agent to behave as a software development role that would normally be in charge of performing that action. Occasionally, an action-based prompt contains a most basic variant of CoT, for example "Think before writing:" in the *write_test* action. Besides, MetaGPT action-based prompts frequently include formatting examples, such as write_code action.

HyperAgent is likewise exploiting role-playing and ReAct prompting, except that instead of action-based prompts, each agent role has its own system prompt with role-playing instructions. Here, two agent roles, Planner and Navigator, are specifically instructed to "think step-by-step". Besides this, system prompts in HyperAgent clearly define agent

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

hierarchy; for example, the Planner agent is explicitly informed about the "interns" at its disposal, while other agents are always instructed to focus on the Planner agent's requests.

ChatDev and AgileCoder share common traits in their prompting strategies, as both are based on the CAMEL framework. First, both frameworks include a base prompt. In the case of ChatDev, it is minimalistic, enumerating roles and introducing ChatDev as a software company. On the other hand, in AgileCoder, it is a more descriptive one, introducing AgileCoder as a software enterprise, explaining the type of SDLC model it follows, and also explaining the existing agent roles. Second, both introduce role-based prompts on the agent level, thus operationalizing role-playing prompts. Third, both frameworks possess phase-level prompts that further elaborate the purpose and the scope of that particular phase within the SDLC. In both frameworks, each phase involves two agents working collaboratively on the same task. The main difference is that ChatDev allows for role-flipping between the agents, thus exploiting the prompting strategy introduced in CAMEL as the "inception prompting". Inception prompting directly influences the agent coordination. Furthermore, these phase-level prompts occasionally also combine elements of CoT. While this is less pronounced in ChatDev, where "think step by step" instruction is supplied only during the coding phase, AgileCoder is more frequently exploiting CoT (e.g. in the following phases: ProductBacklogCreating, SprintBacklogCreating, Undone tasks, RolesEngagement, Coding, InheritCoding). Furthermore, during the code review phase of AgileCoder, multi-step prompting is shown to be a better-performing alternative to single-step prompting.

AgentLite includes PromptGen, a prompt-generation module. According to theoretical framing, agent prompts involve role-playing, action-based instructions, few-shot examples and even specification of constraints, but the final prompts to be used are left to user specification. Based on the examples provided for several use cases, the framework supports role-based prompts, ReAct (example of IndividualAgent) and CoT strategies (example for ManagerAgent provided).

Comparison. Depending on concrete scenarios, different prompting strategies can be more advantageous over others. For example, ReAct prompting is more suitable than pure CoT in situations where the agent might need additional resources to complete the task, such as access to additional data sources or tools, for example, MetaGPT's architect agent accessing Mermaid to create the architecture diagram. Instead, for pure reasoning tasks when tools are not needed, ReAct might instead bring additional complexity, leading to a pipeline that is more difficult to control and debug. With respect to agent coordination, the two prompting strategies that impact the most are inception prompting in ChatDev and HyperAgent's hierarchy-imposing prompts, which explicitly "prescribe" who is the next agent to interact with.

Identified Gaps and Potential Improvements. Considered frameworks typically opt for a single dominant prompting strategy to go with. That strategy is often chosen based on outperforming results, which are reported in the literature, possibly by some initial experimental evaluation, which, nevertheless, is not provided. It would certainly be insightful to have more concrete motivation for the choice of the prompting strategy. The only step in this direction was done by MetaGPT, where the comparison between a high-level instruction and a more detailed one was performed; however, it is difficult to quantify how detailed an instruction should be and to find a "sweet spot" between the token-related cost and performance. Furthermore, prompts are often "static", and their extension/modification is either restricted to concatenating different prompts or requires human intervention. Finally, different frameworks provide evidence of overriding this main strategy at different places (e.g. one function producing artifacts resorting to CoT and the other to a few-shot strategy). This underpins the need to exploit automatic prompt optimization in the multi-agent setup.

4. Operational aspects

4.1. Evaluation

Q8. *What datasets and real-world use cases are considered for the evaluation of the multi-agent framework?*

Contextualization. Evaluation of multi-agent frameworks can be performed through manual testing of, for instance, the agent-generated code in real-world scenarios, or by using suitable benchmarks to measure the system's performance (Hou et al., 2024). However, most of the considered agent systems relied only on benchmark evaluation. In this section, we describe the benchmarks used and analyze the categories chosen for the analysis of four agent-based frameworks.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

Table 2
Comparison of used benchmarks for evaluation of the multi-agent frameworks

Data (sample size)	AgentLite	AgileCoder	ChatDev	HyperAgent	MetaGPT
Closed Source			✓		✓
SoftwareDev					✓
ChatDev internal			✓		
Open Source	✓	✓		✓	✓
GitHub				✓	
SWE-bench (2,294)				✓	
RepoExec (355)				✓	
Defects4J (357)				✓	
Crowd-Sourcing		✓			✓
HumanEval (164)		✓			✓
MBPP (974)		✓			✓
ProjectDev (14)		✓			
Other Public Data	✓				
WebShop (12, 087)	✓				
HotPotQA (113, 000)	✓				
BOLAA (300)	✓				

Table 3
An overview of which system was used as a baseline. The (✓) checkmarks in brackets indicate that in the paper, the other system was acknowledged in related work, for instance, but not used for evaluation.

Compared to → / **Comparing ↓**	**AgentLite**	**AgileCoder**	**ChatDev**	**HyperAgent**	**MetaGPT**
AgentLite	✓				
AgileCoder		✓	✓		✓
ChatDev			✓		✓
HyperAgent		(✓)	(✓)	✓	(✓)
MetaGPT			✓		✓

Overview. AgentLite implemented various agents for their evaluation and showcased their performance on two widely used benchmarks. The first benchmark is based on HotPotQA (Yang et al., 2018), a retrieval-augmented question-answering benchmark for multistep reasoning that has been adapted for LLM agent evaluation in the AgentBench benchmark (Liu et al., 2024a). This benchmark tests the multi-step reasoning and decision-making abilities of an agent system, as each question requires at least two documents to answer. The second benchmark is performed in the Webshop (Yao et al., 2022a) environment and is based on the AgentBoard framework. The benchmark consists of web browsing decisions on the Webshop, an online shopping website simulation featuring 1.18 million real-world products. The tasks of these two benchmarks can be categorized into five categories: code, game, web, tool, and embodied AI (Liu et al., 2024b). Besides the benchmarking evaluation, AgentLite has been evaluated in five applied real-life scenarios. Those applications cover multiple settings such as human-in-the-loop, multi-modality, innovative potential, and practical application (Liu et al., 2024b).

AgileCoder utilizes HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021), two well-established benchmarks for evaluating the system's performance on code generation, as well as a newly introduced benchmark, ProjectDev (Nguyen et al., 2025). ProjectDev is a dataset generated by Nguyen et al. (2025) due to a lack of benchmarks covering intricate software development tasks. Each task includes a prompt and requires the system to produce a complete codebase with multiple executable files (Nguyen et al., 2025). The tasks simulate real-world software development by drawing on software requirements spanning categories such as games, image processing, and data visualization. Note that the authors point out that MetaGPT and ChatDev have produced concurrent work, which, however, is closed-source. The other two benchmarks, HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021), are two well-established software development benchmarks for Python coding. Both benchmarks are generated from crowd source software development problems and are used for agentic systems and LLMs in general.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

ChatDev introduced an internal benchmark (Qian et al., 2024); however, unlike the ProjectDev dataset, this benchmark was not publicly released by the authors. The dataset used by the authors includes software categories from multiple platforms, including Ubuntu, Google Play, Microsoft Store, and Apple Store. Each of the 1,200 tasks contains a prompt based on a textual description of the software requirements and can be categorized into five categories: Education, Work, Life, Game, and Creation.

MetaGPT is evaluated in a similar approach to ChatDev by generating an internal closed-source benchmark (Hong et al., 2023), and to AgileCoder by using HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021) for code generation evaluation. The internal benchmark covers seven experiments across three categories: game, web development, and data analysis. It consists of 70 prompts, which are evaluated using various statistics and metrics defined by the authors.

The HyperAgent authors selected a variety of open-source benchmarks to evaluate the framework's complexity and real-world applicability. They chose three benchmarks, SWE-bench (Austin et al., 2021), RepoExec (Hai et al., 2025), and Defects4J (Just et al., 2014), each of which requires multiple reasoning steps and retrieval from relevant code or text. SWE-bench is a code generation benchmark comprising over 2,000 samples from 12 open-source Python repositories on GitHub. It is a well-established benchmark widely used to evaluate code-generation LLMs, foundation models, and LLM-based agent systems. The samples are generated from GitHub issue resolutions. RepoExec (Hai et al., 2025) is also a code generation benchmark containing 355 samples; however, the tasks must be solved at the repository level. The last benchmark, Defects4J, evaluates fault localization in the form of bugs. The benchmark contains 854 bugs from open-source projects with the following properties: tracked (linked to an issue tracker and mentioned in the commit), atomic (fixed in a single commit), minimized (irrelevant changes removed), code-based (fixed via source code changes), and test-verified (has a reliable triggering test that fails before and passes after the fix).

Comparison. In general, except for the authors of ChatDev, all agentic systems are evaluated using public benchmarks. MetaGPT is evaluated using both closed- and open-source data. All open-source benchmarks used for any agentic system focus on narrow, code-centric tasks. Moreover, HumanEval and MBPP are human-generated programming problems; their outputs are small code snippets that are not comparable to the code generation required in real-world software engineering settings (Qian et al., 2024).

Identified Gaps and Potential Improvements. There are three main gaps in the evaluation of all agentic systems. First, all systems are evaluated using fewer than 5 benchmarks, which does not provide a holistic overview of their performance. Moreover, benchmarks such as SWE-bench, HumanEval, MBPP, and HotPotQA are well-established benchmarks that are used for evaluating foundation models rather than agentic systems (Scao et al., 2023). One of the authors of the HotPotQA benchmarks even raised his concerns about using this benchmark for evaluating agent systems in the first place (Qi, 2025).

Second, in addition to the benchmarks already described, most agent systems are not applied to real-world scenarios involving complex or long-term software projects. AgentLite demonstrates real-world applications across five different setups. These examples demonstrate practical application, human-in-the-loop, multi-modality, and innovative potential (Liu et al., 2024b). The setups are straightforward: for each application, the agents get assigned a name, possible actions, and a predefined role. Each agent then performs the dedicated task. On their GitHub[10], they also provide more extensive instructions for initializing these applications, with working examples. However, those applications are not software-related settings, and their scalability is also not showcased. Three out of five applications are single-agent applications. For the multi-agent applications, there is no per-agent evaluation. Moreover, the performance of the tasks that should be solved is not measured.

AgileCoder, MetaGPT, and ChatDev are tested on real-world software development tasks using self-curated samples from the authors, but only the ProjectDev benchmark is open-source. Therefore, we only cover an analysis of AgileCoders' performance on ProjectDev[11]. ProjectDev contains practical applications of 20 tasks (Nguyen et al., 2025). Those tasks range from generating games such as Snake to building a weather App. AgileCoder generates executable code in 57.79% of the samples (Nguyen et al., 2025). Although they claim to pick intricate software development tasks, it is safe to say that those examples, such as generating a Snake game or a Flappy Bird game, are well-established tasks that have been used in the past to showcase the performance of other agent systems and LLMs for code generation.

[10] https://github.com/SalesforceAIResearch/AgentLite/tree/main/tutorials
[11] https://huggingface.co/datasets/Fsoft-AIC/ProjectDev

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

The third shortcoming is that none of the agent systems addresses constraints related to limited resources or environments. AgileCoder, HyperAgent, and MetaGPT evaluate the cost efficiency and expenses of using agentic systems, but there are no benchmarks for on-premises computation or the energy costs of running those systems. Agentic systems that should support real-world software development should be language model-agnostic, covering both open-source and on-prem solutions. However, some authors of AgileCoder and ChatDev mention computational demands and environmental impact as limitations of their publications.

Q9. *What metrics are used to assess the LLM performance?*

Contextualization. The metrics that are used for evaluating the performance of the agentic systems can be divided into three categories (see Table 4): classic (NLP) metrics, code-related metrics, time- and cost-related metrics and mixed-mode metrics. In this section, we will describe all metrics used in these categories.

Overview. Public benchmarks for evaluating agentic systems are mostly assessed using NLP metrics. In HotPotQA, for instance, the performance is quantified using accuracy and F1-score as metrics. The Webshop has its own metric based on the average reward in the webshop environment. SWE-Bench is evaluated using three different metrics: the percentage of resolved instances, the average time cost, and the average cost of the generated tokens.

For Defect4J, the used metric is acc@k, which represents the number of correct bug detections in the top k generated outputs. HyperAgent was evaluated on accuracy, denoted as acc@1. HumanEval and MBPP use pass@k as a metric for the system evaluation. The authors of AgileCoder adopted this metric to evaluate the functional accuracy of the top-1 generated code, following the approach of ChatDev and MetaGPT (Qian et al., 2024; Hong et al., 2023). RepoExec is also evaluated with pass@k. The authors of HyperAgent used $k = \{1, 5\}$ for their evaluation. For the ProjectDev benchmark, the agent-generated code is evaluated through human assessment and statistical analysis, such as code runtime, token usage, or average number of sprints performed by the system.

As ChatDev supports a holistic evaluation, the authors introduced four objective metric dimensions that should provide a more comprehensive analysis than pass@k (Qian et al., 2024). The first dimension is completeness, which is the percentage of software without "placeholders" or missing parts. The second dimension is executability, which is calculated as the percentage of successfully compiled code. The third dimension is consistency, measured by the cosine distance between the generated software and the requirements' textual embeddings. The last dimension is quality, calculated by multiplying the first three dimensions. Note that an alternative quality dimension is the average sum of the first three dimensions.

Besides evaluating model performance using the standard benchmark metric, the authors of HyperAgent (Phan et al., 2024) calculated the average cost per benchmark in US dollars. On the SWE-Bench dataset, the average cost spans from 0.45 $ to 2.01 $ depending on the LLMs API used. For the RepoExec and Defects4J benchmarks, the average cost is reported as 0.18 $ as they only evaluated a single HyperAgent setup on both benchmarks. Note that these numbers may vary depending on the LLM used for the agents and the current price per token of the closed-source model.

For the internal benchmark of MetaGPT, the statistics and metrics cover five evaluation dimensions. First, executability: rate generated code from 1 to 4 by humans, with 1 indicating non-functional code and 4 indicating flawless code. The second dimension is cost, which includes running time, token usage, and expenses. The third dimension covers code statistics, such as the number of generated code files, lines of code per file, and the total number of code lines. The fourth dimension is productivity, calculated as the number of tokens used divided by the number of lines of code. The last dimension is human revision cost, calculated as the time required for manual code corrections.

Comparison. For the public benchmarks, all agentic systems are evaluated using the same metric. The authors of AgileCoder adapted the unbiased version of the pass@k metric, based on the approach proposed by the authors for MetaGPT, to evaluate the functional accuracy of the top-k generated codes (Nguyen et al., 2025). For the closed-source benchmarks of ChatDev and MetaGPT, evaluation dimensions are defined to facilitate a more holistic evaluation. While both cover completeness and quality as evaluation dimensions within their proposed metrics, the authors of MetaGPT focus more on code statistics and costs with the three newly introduced dimensions. The authors of ChatDev focus on code completeness and consistency. Although the authors of ChatDev do not mention code statistics among their metrics, they also published insights into the generated code, including the number of generated tokens, files, and lines of code.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

Table 4
Metrics used for evaluating considered multi-agent frameworks.

Metric Type	Metric	AgentLite	AgileCoder	ChatDev	HyperAgent	MetaGPT
Standard	F1-Score	✓				
	Accuracy	✓			✓	
Code - Stats	Lines of code generated			✓		✓
Code - Executability	pass@k		✓		✓	✓
	Code Executability		✓	✓	✓	✓
	acc@k				✓	
Code - Quality	#Errors		✓	✓	✓	
	Code Completeness			✓		
	Code Consistency			✓		
	Code Quality			✓		
Time Perf	Entire Running Time in s		✓			✓
	Avg. Time/Sprint in s		✓			
	Human Revision Cost					✓
	#Sprints		✓			
Cost	Token Usage		✓	✓		✓
	Expenses (USD)		✓		✓	✓
Mixed	Productivity					✓

Identified Gaps and Potential Improvements. MetaGPT uses human-based evaluations on two of its five metric dimensions; however, the authors do not further explain how the human evaluation is performed. It is not stated if the human evaluation was performed by multiple specialists or by a single person, which lowers the reliability and trustworthiness of the metrics. With the human revision cost, they also introduce a highly subjective metric that needs to be quantified, but the authors of MetaGPT did not. Moreover, code statistics are used as a metric but not compared to any gold standard or set in relation to. For instance, the number of used tokens could have been taken in relation to the number of tokens needed to perform a task or to solve a problem. Code statistics should be used to provide further insights beyond the evaluation using established metrics, like the authors of ChatDev did.

In general, the metrics used for all evaluations of all agentic systems are narrowed down to code and text generation performance and expenses. Evaluations based on coding benchmarks or small app tasks tend to prioritise syntactic correctness and executability, but overlook metrics related to system performance in production, maintainability, and incident recovery, which are key dimensions for any framework aspiring to support the full SDLC. Also, the code statistics and expenses only show a current snapshot of the agentic systems, as the expenses, as well as, for instance, the number of generated tokens, may have changed since the time of publication. Beyond pass@1 and executability, metrics should include system uptime, recovery time, error traceability, and change impact. Benchmarks should model realistic DevOps pipelines and allow agent frameworks to be assessed across the entire lifecycle.

Q10. *How does the system scale with increasing project complexity, and has computational cost been evaluated? Do small agent systems outperform state-of-the-art (SOTA) LLMs in code generation?*

Contextualization. Evaluating MAS for SDLC requires understanding how their performance is affected as project complexity increases. Agent systems introduce components such as task decomposition, coordination mechanisms, and communication between agents that make their evaluation difficult to perform.

Assessing scalability requires profiling data, including metrics such as execution time, token consumption, number of agent interactions, and resource utilization related to complex tasks. However, MAS are a relatively recent research field and systematic data collection and benchmark datasets remain limited. As of today, it is still unclear how these systems scale in practice or how their computational cost compares to single-model approaches. Moreover, evaluating agentic systems in real-world scenarios involving complex or long-term software projects (e.g., with versioning, product strategy, or backward compatibility) demonstrates their applicability. For a comprehensive framework evaluation, a comparison with LLMs on code generation can be a useful indicator of their performance, as well as the size of LLMs used to test them.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

Overview. What concerns scalability, the authors of AgileCoder emphasized it as one major limitation when dealing with large and complex software (Nguyen et al., 2025). The authors of ChatDev claim that their agent system is more suitable for prototyping than for real-world applications (Qian et al., 2024). HyperAgent is explicitly designed to support parallel subtasks execution and targeted recomputation. In cases where the code under investigation is modified, HyperAgent performs a localized analysis, identifying and re-executing only the affected components of the codebase. Although AgentLite does not support parallel task execution, it offers a certain degree of scalability through its task decomposition-based design, in which subtasks are assigned to Individual Agents by the Manager Agent.

Regarding the size of LLMs used, all considered frameworks utilized large LLMs (with more than 7 billion parameters) as the backbone for their agents. None of the authors considered using smaller models to test them inside their proposed framework. Regarding benchmarking LLMs for code generation, the most comprehensive analysis is provided by the authors of AgileCoder (Nguyen et al., 2025). They compare their framework not only with two agent systems, MetaGPT and ChatDev, but also evaluate seven LLMs trained on code: CodeGen (Nijkamp et al., 2022), CodeGeeX (Zheng et al., 2023), PaLM Coder (Chowdhery et al., 2023), DeepSeek-Coder (Guo et al., 2024a), GPT-3.5, GPT-4 (OpenAI et al., 2024), and Claude 3 Haiku (Rahman et al., 2024). However, AgileCoder outperforms all LLMs by a large margin.

Other considered frameworks are only compared to other SOTA agent systems. The authors of MetaGPT compared their framework with AutoGPT (Significant Gravitas, 2023) and AgentVerse (Chen et al., 2024). The authors of ChatDev compared their results with MetaGPT and GPT-Engineer (Osika, 2023). GPT-Engineer is a single-agent system with a backbone LLM that understands task requirements and has reasoning capabilities.

Comparison. HyperAgent is the only framework among the considered ones designed to explicitly ensure scalability. A comparison of the cost evaluations of MetaGPT and HyperAgent highlights the different approaches in terms of scalability. Both frameworks show a wide range of costs on the tasks, with MetaGPT being six times more expensive on average (see Q8). A partial explanation of the observed cost differences can be drawn from the architectural design and operational principles of the agent frameworks. Specifically, as HyperAgent performs a localized intervention on the codebase, it is inherently more efficient in scenarios involving incremental code changes, reducing redundant computation and, consequently, the cost. On the contrary, other agentic systems, such as MetaGPT (Hong et al., 2023), lack this granularity and recompute the entire task pipeline whenever a change is detected. While this remains a theoretical interpretation in the absence of fine-grained profiling data, it offers an important perspective on the potential scalability advantages of HyperAgent over more monolithic designs, such as MetaGPT, ChatDev and AgileCoder. Regarding LLMs for code generation benchmarking, AgileCoder has been compared also with backbone LLMs, while MetaGPT and ChatDev only with other multi and single-agent systems.

Identified Gaps and Potential Improvements. A major gap is that the considered frameworks have not been tested on real-world use cases, so there is no evidence of their scalability with increasing project complexity.

A second potential improvement could be a comprehensive evaluation of the scaling capabilities of open-source LLMs. Recent research has shown that LLMs with fewer than 4 billion parameters achieve strong performance on benchmarks such as HumanEval and MBPP (Bakouch et al., 2025). For software-oriented use by small and medium-sized companies, such performance insights are valuable for building an agile software development system.

Finally, it is worth noting that the computational cost of a language model can also be calculated by quantifying the compute of the agent systems. Especially for on-premise work and open-source models, that metric is a great indicator for end-users. Moreover, those evaluations are more persistent compared to cost evaluations of closed-source commercial models. Nonetheless, this perspective has not been analyzed for any of the frameworks.

5. Human interaction aspects

Q11. *Does the framework support human alignment, human-in-the-loop processes, or feedback incorporation? If so, what level of feedback is supported (e.g., intermediate vs. final feedback), and how is this implemented across agents?*

Contextualization. Human interaction plays a crucial role in aligning AI frameworks with expert knowledge. According to Ferrara (2023), human operators can act as a complementary validation layer of AI-generated outputs

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

to identify potential issues and provide corrective feedback that progressively strengthens the reliability and fairness of the model. These same authors highlight the importance of human intervention in AI systems, which may involve conducting regular audits, retraining AI models with curated data, applying fairness metrics, or implementing human-in-the-loop (HITL) approaches. In these approaches, experts can guide AI by finding the right balance between automation and human judgment (Ferrara, 2023).

In this section, the different frameworks are analyzed in terms of feedback and human-machine interaction, whose degree of autonomy is further evaluated according to a five-level autonomy scale (Feng et al., 2025), which ranges from Level 1 (L1) to Level 5 (L5). L1 is the lowest level of autonomy, in which the user acts as an operator, directing and making decisions. In L2, both the user and the agent make decisions; they collaborate, and the agent can independently work on its own tasks. In L3, the user is mostly involved in providing feedback, preferences, and guidance, but it is the agent that plans and executes the activities. In L4, the user is only required to intervene when some blocking events occur and the agent cannot proceed with its tasks. L5 is the highest level of autonomy in which the user only monitors as an observer.

Overview. AgentLite can be placed at L2 in the five-level autonomy scale (Feng et al., 2025), since users have an active participation in the design and planning tasks, providing direct input and guidance as collaborators in the process. The framework allows users to define specialized HumanInstruction actions, which enable querying humans for input during execution and drive multi-round interactions in natural language, ensuring the user acts as a collaborator in task design and planning. A clear example is the Interactive Image Understanding application, where humans can engage in multi-round interactions in natural language by giving questions for several rounds, such as "What is in the image?, What is the color of the bridge?".

AgileCoder can be placed at L5, since it does not rely on HITL. Similarly, the HyperAgent framework presents an autonomy L5, as all the workflow is performed autonomously by four agents and humans are not involved in the decision-making loop during task execution. ChatDev also has an L5 level of autonomy, even though the latest implementation enables an L3 configuration, allowing users to set a human node to provide input.

MetaGPT can operate at autonomy levels L5 or L3, depending on the configured workflow. The default configuration supports an end-to-end autonomous process; however, the framework allows users to define HumanInput actions that interrupt execution and require human input before proceeding.

Comparison. The multi-agent frameworks operate at a high level of autonomy, ranging between L3 and L5, with only AgentLite positioned at L2. This is because AgentLite allows users to be involved in the development workflow as consultants, meaning that tasks can be delegated to humans (HITL), rather than limiting their role to providing feedback or validation. Both MetaGPT and ChatDev, however, support an optional L3, enabling the definition of human input nodes within the autonomous workflow.

Identified Gaps and Potential Improvements. Overall, while AI frameworks increasingly explore HITL, the integration of human interaction remains limited in many cases, typically confined to initial stages or pre-configured human input actions. A promising future direction may be to increase human-agent collaboration, like the HULA framework developed by Takerngsaksiri et al. (2025), which incorporates human feedback at each step of the software development tasks, not only to enhance input context, but also to ensure "the software development process remains collaborative rather than entirely reliant on fully autonomous agents". Human expertise integration has shown potential in mitigating LLM-related issues like bias or hallucinations. Additionally, intermediate output validation could reduce the risk of error propagation and requirements misalignment.

Q12. *Are there any usability considerations related to the interaction with the framework?*

Contextualization. The usability of the framework, that is, how easy it is for users to operate and configure the system, is an important consideration. More specifically, we define usability as the ease of installation and initial use of the framework, as well as the learning curve associated with customizing the agents and workflows for specific use-cases Abran et al. (2003).

Overview. AgentLite is a generalist framework that requires users to define their Individual and Manager agents. Although the framework does not provide any pre-configured agent setup, it allows users to define and test new agents

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

with a few lines of code. It also provides a UI, but only for testing purposes. The documentation is clear and includes an explanation of agent definitions, along with several notebooks containing use-case examples. However, support for user learning is limited, especially with respect to guidance and feedback mechanisms.

As an SDLC-focused framework, AgileCoder gives less flexibility to the user in terms of agent definition and customization, but provides an easy command-line interface to run the default workflow execution. Installation and initial usage are straightforward, and a Web UI is available for testing and demonstration purposes. Similar to AgileCoder, ChatDev provides a simple installation process and command-line execution. Both AgileCoder and ChatDev offer an easy workflow customization interface, allowing users to modify and define workflows through JSON files by specifying the agents involved in the SDLC phases and the prompts used. The design of Hyperagent is characterized by its generalizability and scalability. Initial usage requires the installation of some dependencies; although the framework supports command-line execution, the limited documentation makes it less straightforward to understand how to define agents and configure workflows.

MetaGPT gives the user limited flexibility regarding task planning and coordination, since these processes follow Standardized Operating Procedures (SOPs). Once the user provides the requirement, the system autonomously executes the entire multi-agent workflow. The installation process is easy and well-documented, and can also be done via pip command. Moreover, the documentation illustrates several examples of customization capabilities. The framework does not provide an explicit agent-to-human feedback mechanism, but the possibility to pre-define a human agent to interact with the active workflow.

Comparison. AgentLite and HyperAgent adopt a general-purpose approach based on the definition of custom agents, resulting in greater flexibility across diverse scenarios and use cases. In contrast, ChatDev, AgileCoder, and MetaGPT focus primarily on code development, relying on preconfigured agents, thereby reducing flexibility in favor of ease of use and a lower learning curve. While ChatDev and AgileCoder provide a clearer and more easily customizable workflow interface through the use of a JSON file, MetaGPT is better documented and offers greater flexibility in integrating user interaction. AgentLite is better documented than HyperAgent, whose installation appears more complex due to software dependencies.

With respect to prompt customization, AgileCoder and ChatDev are much more user-friendly than MetaGPT. More specifically, in AgileCoder and ChatDev, prompt customization requires modification of a single configuration file containing phase prompts. On the contrary, in MetaGPT, just localizing appropriate prompts requires the inspection of various scripts in the codebase. In HyperAgent, this localization is fairly easy, but the user needs to have a good understanding of how the Planner agent interprets these descriptions to get the desired behavior, as the framework relies heavily on automated planning rather than fixed scripts. In AgentLite, it is easy to localize the PromptGen module, but the customization is not as trivial as in AgileCoder and ChatDev, as the user needs to understand the prompt template module.

Identified Gaps and Potential Improvements. None of the considered frameworks provide a proper UI that could allow users to use and configure the framework without inspecting the codebase. The adoption of a UI would enable more immediate and extensive use, increasing agents and workflow customization possibilities, including the selection of backbone LLMs and interventions on corresponding prompts. Furthermore, complete integration with development tools such as IDEs and repositories is still an unexplored area, which could facilitate alignment with existing organizational development processes and contribute to broader adoption. Even though HyperAgent interacts with repositories and existing codebases, it is primarily intended for code completion and bug fixing rather than supporting full integration with real-world SDLC processes.

6. Trustworthy and Responsible AI aspects

Q13. *What are considered hallucinations in the context of software development? How are these detected and mitigated, particularly when errors spread across agents? Are mitigation strategies applied uniformly across all agents? Does the framework recommend specific prompting strategies or prompt engineering techniques to improve the reliability and trustworthiness of the output?*

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

Contextualization. Hallucinations are one of the critical problems in LLMs and LLM-based systems, in general. Hallucination definition in the literature varies across domains and tasks. Instead, the authors of the considered frameworks are more aligned in the definition of hallucinations.

As such, in MetaGPT, hallucinations are mentioned in the context of incomplete functions, missing dependencies and/or undiscovered bugs. MetaGPT also highlights hallucination risks, particularly due to imprecise definitions of tasks and the accumulation of errors as multiple agents interact in complex workflows. Although HyperAgent does not explicitly define hallucinations in a software development setup, it can be deduced that any incomplete/unfunctional code is considered a hallucination. As a potential cause of agent hallucination, HyperAgent indicates underlying LLM hallucination leading to premature task termination as well as context loss due to agent miscommunication. ChatDev borrows definition of Agnihotri and Chug (2020), considering hallucination to be any incomplete, incorrect or unexecutable code that does not satisfy the initial requirements. AgileCoder mentions problems such as error-prone code, misalignment of the code with the sprint backlog and failing to satisfy the acceptance criteria.

Overview. Most of the frameworks acknowledge hallucinations as a critical challenge. However, not all frameworks explicitly address this issue. More prominently, rather than detecting hallucinations, frameworks are more oriented towards hallucination prevention and, to lesser extent, hallucination mitigation.

In MetaGPT, hallucination prevention is implemented over four independent levels. First, on a workflow level, with a firmly enforced, predefined and simple workflow with role-specific actions. Second, on a communication level, with "subscription" mechanism which prevents an agent from accessing to an irrelevant content. Third, through an iterative code improvement with an executable feedback mechanism, which helps the agent benefit from its own historical information. And finally, through customized instructions in the prompts that are driven by SOP and require structured outputs, thus ensuring the agent's conformity to the established standards.

ChatDev introduces a communicative dehallucination mechanism to address the hallucination problem. This involves a structured dialogue pattern where the assistant agent proactively requests more detailed input before finalising its response. The interaction includes deliberate "role reversal", enabling the assistant to take on an instructor-like role to clarify ambiguous instructions. This iterative clarification reduces premature code generation and facilitates more accurate, context-aware outputs (Qian et al., 2024). Notably, this mitigation is localized to coding-related roles (code completion, review, and testing), where the risks of hallucination are most impactful. Qian et al. (2024) performed an ablation study, examining the effect of the presence/absence of this mechanism, and showed that its absence results in decreased performance.

To mitigate hallucinations in AgileCoder, a three-step code review process is applied. It verifies that: i) there is no incomplete code, ii) the code satisfies the backlog and, iii) the code passes acceptance tests and is error-free. The testing phase is another touchpoint where hallucination prevention is taking place within the AgileCoder. To support this process, it utilizes internally generated dynamic code dependency graph (CDG). Reversing topological order of CDG helps devise a logical testing plan and improve test cases that respect dependencies among coding scripts (Nguyen et al., 2025).

HyperAgent does not explicitly propose any mechanism for addressing hallucinations, although some of the specialized tools help mitigate hallucinations. For example, Repair Editor helps resolve problems such as incorrect syntax or incorrect file paths. AgentLite does not explicitly define hallucinations, nor does it propose any coping mechanism in this regard.

Comparison. Considered frameworks distinguish in the way hallucination is addressed: MetaGPT's approach is more oriented towards prevention, while AgileCoder and HyperAgent mostly focus on mitigation; ChatDev is both prevention and mitigation oriented, while AgentLite does not consider hallucinations at all. Furthermore, the agents and roles are treated differently across most frameworks. MetaGPT is the only framework that has the mechanisms, apart from customized role-based and action-based prompts, which are agent-agnostic. For ChatDev and AgileCoder, hallucination mechanisms are considered only for development-review-test agent roles, rather than being applied uniformly across all agent types. HyperAgent acts similarly, except that the hallucination mitigation is "outsourced" to the tool, instead of the agent itself. The only prompting strategy (among those described in Q7) that is explicitly mentioned in the context of hallucination prevention is inception prompting in the communicative dehallucination mechanism of ChatDev.

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

Identified Gaps and Potential Improvements. One of the main drawbacks of handling hallucination in considered frameworks is the lack of hallucination quantification and hallucination detection, making the effectiveness of mitigation strategies unclear. Additionally, there is no benchmarking of hallucination prevention strategies. Likewise, a more systematic hallucination mitigation is needed across the framework, as the current coping strategies are typically local, tailored to the roles and functions of specific agents.

While MetaGPT introduced more global mechanisms, some of these are still pretty role-sensitive: agents tasked with critical design or engineering roles benefit most from SOP-guided execution and output validation. On the other hand, across other considered frameworks, hallucination mitigation remains incompletely addressed, particularly beyond code generation phases. Roles involved in requirements elicitation, architecture definition, or task coordination are equally prone to creating plausible yet inaccurate artifacts (e.g., ambiguous requirements, misleading task decompositions) and often lack explicit safeguards.

Furthermore, none of the frameworks mitigates the hallucination propagation problem where the errors compound across interactions (that is, unless it results in a compile-time or test-time failure downstream). In most frameworks, once an agent completes its assigned task, there is no formal mechanism for revising its output in the light of downstream evidence, making hallucinated content hard to trace. A potential workaround for this limitation would be the implementation of human-in-the-loop mechanisms at key decision points across the SDLC. In parallel, agents should be equipped with self-assessment triggers, such as low-confidence indicators or internal consistency checks that proactively request clarification.

With respect to prompting strategies, besides inception prompting, other strategies such as CoT, role-playing and few-shot prompting undoubtedly implicitly contribute to hallucination prevention and mitigation. Considered frameworks are missing a quantitative assessment of the prompting strategies efficiency in this regard. To this end, existing benchmarks such as CodePromptEval (Khojah et al., 2025), considering (a combination of) five different prompting strategies, could be exploited and extended to real-world projects. Additionally, approaches like prompt chunking, which break large prompts into smaller parts to reduce confusion and improve accuracy, could be applied (Darwish et al., 2025).

Q14. *How does the framework monitor responsible AI principles (e.g., transparency, explainability, fairness and bias, ethics, and accountability), and what mechanisms are used to detect and address violations or related issues?*

Contextualization. Responsible AI is an approach to developing and deploying AI systems, in this case, AI agents, that ensures these are built correctly and governed responsibly throughout their entire lifecycle. This process is driven by a set of core principles and guidelines, such as Responsible AI principles from OECD (OECD, 2024) or EU AI Act[12]. Even though these principles are frequently source-dependent and not unanimously aligned, they typically require that an AI agent should be designed to guarantee transparent, explainable, fair, inclusive, ethical and accountable interactions with humans across their entire software development lifecycle.

Transparency refers to how much information about the system and its outputs is available to users interacting with the system. It entails providing insight into the development, training, operation, and deployment of AI systems within a specific domain. It does not necessarily require the disclosure of complex proprietary code or data. *Explainability* focuses on making agent outputs and reasoning comprehensible to users with different expertise levels. *Fairness* ensures that agent decisions and interactions remain unbiased and inclusive across all demographic and professional groups. *Bias* across multi-step agent workflows, also known as bias compounding, occurs when small biases introduced in early stages accumulate and amplify throughout the workflow, ultimately leading to larger systemic bias in the final output. *Ethics* principles reinforce system alignment to human values such as ethics and morality. *Accountability* refers to establishing clear responsibility for different outcomes of the AI system.

Overview and Comparison. In terms of *transparency*, there are several aspects that are partially covered. In order not to repeat information, we will directly provide the overview and compare the frameworks. With respect to datasets, none of the frameworks report any data used for training. The only framework that mentions the use of prior data through so-called iterative self-improvement mechanisms is MetaGPT, which, however, does not disclose it. AgileCoder is the only framework disclosing its custom test dataset (although explicitly mentioning, neither MetaGPT nor ChatDev is disclosing theirs). Logging mechanism is implemented across different frameworks, but ChatDev's and

[12] https://digital-strategy.ec.europa.eu/en/library/ethics-guidelines-trustworthy-ai

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

AgileCoder's logs are much better structured and more readable than those of MetaGPT and HyperAgent. Likewise, prompt transparency is much higher in ChatDev and AgileCoder, where they are configured in a single file. On the contrary, MetaGPT's prompts are spread across different scripts and sometimes also difficult to trace. Unlike MetaGPT, ChatDev and AgileCoder, where agent responsibilities are distributed clearly in pre-defined roles, in HyperAgent, there exist different sub-roles of the agents that are not even mentioned in the paper describing the framework, but can be observed from the logs. Contrary to all of these, transparency in AgentLite relies on the developer's implementation choices.

In terms of *explainability*, although frameworks offer collaboration trails documented in logs, their internal reasoning remains hidden, and they cannot provide an explanation of their decision. This particularly holds for MetaGPT, where additionally, the communication between agents relies on documents and diagrams rather than raw dialogue. On the contrary, in case of ChatDev and AgileCoder, thanks to the natural language used in a "live" chat chain between the agent, some insights regarding decisions and clarifications can be obtained.

What concerns *ethics*, none of the considered frameworks implements any specific mechanisms for enforcing it. While ethical concerns are mentioned (only) in MetaGPT, these actually refer to other types of concerns, such as privacy, security and other socially-oriented dimensions like programming skill democratization and software development labour market.

With respect to *fairness and bias*, in theory, every user, regardless of demographic background, can potentially intervene at different checkpoints within any of the considered frameworks (for more details on HITL and autonomy levels per framework see Q11). Nevertheless, none of the considered frameworks addresses bias monitoring, the effects of bias compounding in a multi-step agent workflow, or explicitly enforces fairness. The treatment and control of these key principles depend on explicit human intervention through prompt design and careful human supervision. Similarly, none of the considered frameworks monitors or implements any specific mechanism for enforcing *accountability*. Accountability is explicitly mentioned only in MetaGPT, stating that human engineers, as framework users, are accountable for the obtained results.

Identified Gaps and Potential Improvements. Considered frameworks support some aspects of responsible AI, but they are not comprehensible and lack specific requirements to become fully responsible frameworks. One possible way to overcome this gap is to resort to the RACI (Responsible–Accountable–Consulted–Informed) (Dubinsky et al., 2010) collaboration model, which by design ensures that interactions between human and AI agent remain transparent, traceable, explainable, and inclusive throughout the software development lifecycle. As an alternative, each of the mentioned principles can be addressed independently (or with some overlaps between principles but less holistically), following the existing best practices from the literature.

In terms of *transparency*, the LLM-MAS framework (de Cerqueira et al., 2024) introduces transparency through structured multi-agent debates, where each agent presents and justifies its reasoning over multiple discussion rounds. This process exposes the logic behind decisions and allows external observers to trace the ethical and technical rationale. The ALI-Agent framework (Wang et al., 2024) ensures transparency through its two-stage evaluation process: emulation and refinement, where users can inspect how certain scenarios are generated, refined, and assessed.

With respect to *interpretability*, LLM-MAS (de Cerqueira et al., 2024) reinforces it through thematic and comparative analysis of agent debates, helping users from diverse backgrounds grasp how different types of issues are detected and addressed. ALI-Agent enhances interpretability by providing scenario explanations and a memory module.

To ensure *fairness* and reduce *bias*, LLM-MAS (de Cerqueira et al., 2024) incorporates diverse agent roles and viewpoints in its debate structure, minimizing dominance of a single perspective. A Unified Framework proposed by Ranjan et al. (2025) uses agents with different roles (like checking for fairness) to participate in structured debates that help identify where bias might appear or grow during the workflow.

To develop an *ethical* agent, it is necessary to incorporate Moral Alignment or Intrinsic Reward with the LLMs by training them with rewards based on moral values, specifically those required in SDLC. Several mechanisms have been proposed for ethical AI agents that could be integrated into the AI agents for SDLC. Lei et al. (2024) proposed FairMindSim, a Belief-Reward Alignment Model to analyse the moral and challenging situations, e.g., behavior, emotions and beliefs between human and LLM agents. Schuett et al. (2025) proposed a human-centred institutional mechanism for designing and setting up an AI ethics board to reduce AI-related risks and promote ethical practices. Reinforcement Learning from Human Feedback (RLHF) and Direct Preference Optimization (DPO) are two popular methods that can be considered for ethical agents with agency and autonomy (Tennant et al., 2025).

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

When it comes to *accountability*, it is important to also consider its dependence on traceability. In theory, within each of the considered frameworks, agents are designed to be traceable to a certain degree. This, to some extent, permits tracking decisions and errors back to the specific agent role or action responsible. However, in practice, identifying the accountable party would require considerable effort to detect all relevant prompts, analyze their subordinate instructions, and the logs. The latter is particularly cumbersome due to the different levels of detail and structuring of logging mechanisms, as already mentioned. One potential way to overcome this problem is by introducing an explicit agent that monitors the other agents' behaviour and intervenes, in real time, in case it detects any malicious behavior, as suggested by Tamang and Bora (2025), and performed by their so-called "Enforcement Agent". This would enable traceability of the decisions and mistakes back to a specific agent.

7. Conclusions and perspectives

This study presented a research question–driven analysis of five multi-agent frameworks designed to support different phases of the software development lifecycle. By examining architectural, technological, operational, human interaction, and responsible AI aspects, the analysis aimed to provide a structured overview of their design choices, capabilities, and limitations. These frameworks represent remarkable contributions towards more autonomous, multi-agent-based software development. However, several drawbacks are still evident.

First, these frameworks do not encompass all the phases of SDLC. Second, they are not evaluated on complex, long-term real-life software development projects. Third, some of the frameworks still heavily rely on closed LLMs, which is problematic in settings where data privacy, cost, and auditability matter. Moreover, research demonstrates that locally adapting open models provides better data control, superior performance, and lower costs compared to privacy-preserving methods for closed models (Hanke et al., 2024). Additionally, using open LLMs contributes to the greater transparency and independent oversight of the system as a whole, another point where considered frameworks could remarkably improve. In fact, with respect to trustworthy and responsible AI aspects, these frameworks are still lacking maturity, since the coping mechanisms for several important aspects, such as fairness and bias, ethics and accountability, are not put in place. Nevertheless, establishing such mechanisms is indispensable as many of these problems inevitably occur (for example, bias arises from training data, fine-tuning processes, or even human evaluators themselves). This might include additional monitoring agent(s) (such as the so-called, "watchdog agent"), designed to observe the performance of other agents, detect anomalies or unproductive loops, enforce constraints, prevent critical external adversarial threats and trigger recovery actions.

Finally, although some of these frameworks include mechanisms that allow them to be easily adapted, they generally offer limited flexibility for the user because the workflows are largely automated. The latter undermines human agency and hampers effective oversight. Although HITL strategies cannot fully eliminate issues like bias or hallucinations, incorporating human expertise has shown potential in mitigating, among others, these problems. Finding a practical balance between automation and human oversight is not easy, but it is a crucial task as the excessive dependence on human feedback can make the process both expensive and inefficient. A promising future direction may be to increase the autonomy of agents focused on coding-related tasks, such as code generation or deployment, while humans concentrate on those that require stronger human agency, such as planning and evaluation. Moreover, explicit human intervention to address responsible aspects, such as bias, might not be the most optimal: for high-level autonomy, an algorithmic approach must be used by agents.

References

Abran, A., Khelifi, A., Suryn, W., Seffah, A., 2003. Usability meanings and interpretations in iso standards. Software Quality Journal 11, 325–338. URL: http://dx.doi.org/10.1023/A:1025869312943, doi:10.1023/a:1025869312943.

Agnihotri, M., Chug, A., 2020. A systematic literature survey of software metrics, code smells and refactoring techniques. JIPS (Journal of Information Processing Systems) 16, 915–934.

Anthropic, . The claude 3 model family: Opus, sonnet, haiku. URL: https://api.semanticscholar.org/CorpusID:268232499.

Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., Sutton, C., 2021. Program synthesis with large language models. URL: https://arxiv.org/abs/2108.07732, arXiv:2108.07732.

Bakouch, E., Ben Allal, L., Lozhkov, A., Tazi, N., Tunstall, L., Patiño, C.M., Beeching, E., Roucher, A., Reedi, A.J., Gallouédec, Q., Rasul, K., Habib, N., Fourrier, C., Kydlicek, H., Penedo, G., Larcher, H., Morlon, M., Srivastav, V., Lochner, J., Nguyen, X.S., Raffel, C., von Werra, L., Wolf, T., 2025. SmolLM3: smol, multilingual, long-context reasoner. https://huggingface.co/blog/smollm3.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J.D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler,

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., Amodei, D., 2020a. Language models are few-shot learners, in: Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., Lin, H. (Eds.), Advances in Neural Information Processing Systems, Curran Associates, Inc.. pp. 1877–1901. URL: https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfcb4967418bfb8ac142f64a-Paper.pdf.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J.D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al., 2020b. Language models are few-shot learners. Advances in neural information processing systems 33, 1877–1901.

de Cerqueira, J.A.S., Agbese, M., Rousi, R., Xi, N., Hamari, J., Abrahamsson, P., 2024. Can we trust ai agents? a case study of an llm-based multi-agent system for ethical ai. arXiv preprint arXiv:2411.08881 .

Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H.P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F.P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W.H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A.N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., Zaremba, W., 2021. Evaluating large language models trained on code. URL: https://arxiv.org/abs/2107.03374, arXiv:2107.03374.

Chen, W., Su, Y., Zuo, J., Yang, C., Yuan, C., Chan, C.M., Yu, H., Lu, Y., Hung, Y.H., Qian, C., Qin, Y., Cong, X., Xie, R., Liu, Z., Sun, M., Zhou, J., 2024. Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors, in: Kim, B., Yue, Y., Chaudhuri, S., Fragkiadaki, K., Khan, M., Sun, Y. (Eds.), International Conference on Learning Representations, pp. 20094–20136. URL: https://proceedings.iclr.cc/paper_files/paper/2024/file/578e65cdee35d00c708d4c64bce32971-Paper-Conference.pdf.

Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., et al., 2023. Palm: scaling language modeling with pathways. J. Mach. Learn. Res. 24.

Darwish, A.M., Rashed, E.A., Khoriba, G., 2025. Mitigating llm hallucinations using a multi-agent framework. Information 16, 517.

Dubinsky, Y., Yaeli, A., Kofman, A., 2010. Effective management of roles and responsibilities: Driving accountability in software development teams. IBM Journal of Research and Development 54, 4:1–4:11. doi:10.1147/JRD.2009.2039896.

Feng, K., McDonald, D.W., Zhang, A.X., 2025. Levels of autonomy for ai agents. arXiv preprint arXiv:2506.12469 .

Ferrara, E., 2023. Should chatgpt be biased? challenges and risks of bias in large language models. First Monday 28. URL: https://firstmonday.org/ojs/index.php/fm/article/view/13346, doi:10.5210/fm.v28i11.13346.

Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., et al., 2024. The llama 3 herd of models. URL: https://arxiv.org/abs/2407.21783, arXiv:2407.21783.

Guo, D., Zhu, Q., Yang, D., Xie, Z., Dong, K., Zhang, W., Chen, G., Bi, X., Wu, Y., Li, Y.K., Luo, F., Xiong, Y., Liang, W., 2024a. Deepseek-coder: When the large language model meets programming – the rise of code intelligence. URL: https://arxiv.org/abs/2401.14196, arXiv:2401.14196.

Guo, T., Chen, X., Wang, Y., Chang, R., Pei, S., Chawla, N.V., Wiest, O., Zhang, X., 2024b. Large language model based multi-agents: A survey of progress and challenges, in: Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI 2024, Jeju, South Korea, August 3-9, 2024, ijcai.org. pp. 8048–8057. URL: https://www.ijcai.org/proceedings/2024/890.

Hai, N.L., Nguyen, D.M., Bui, N.D.Q., 2025. On the impacts of contexts on repository-level code generation, in: Chiruzzo, L., Ritter, A., Wang, L. (Eds.), Findings of the Association for Computational Linguistics: NAACL 2025, Association for Computational Linguistics, Albuquerque, New Mexico. pp. 1496–1524. URL: https://aclanthology.org/2025.findings-naacl.82/, doi:10.18653/v1/2025.findings-naacl.82.

Hanke, V., Blanchard, T., Boenisch, F., Olatunji, I., Backes, M., Dziedzic, A., 2024. Open llms are necessary for current private adaptations and outperform their closed alternatives. Advances in Neural Information Processing Systems 37, 1220–1250.

Hong, S., Zheng, X., Chen, J., Cheng, Y., Wang, J., Zhang, C., Wang, Z., Yau, S.K.S., Lin, Z., Zhou, L., 2023. Metagpt: Meta programming for multi-agent collaborative framework. arXiv preprint arXiv:2308.00352 3, 6. URL: http://cnaiwiki.com/aiweb/file/EN_METAGPT.pdf.

Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J., Wang, H., 2024. Large language models for software engineering: A systematic literature review. ACM Trans. Softw. Eng. Methodol. 33. URL: https://doi.org/10.1145/3695988, doi:10.1145/3695988.

Hou, Y., Dong, H., Wang, X., Li, B., Che, W., 2022. Metaprompting: Learning to learn better prompts, in: Proceedings of the 29th international conference on computational linguistics, pp. 3251–3262.

Huang, D., Zhang, J.M., Luck, M., Bu, Q., Qing, Y., Cui, H., 2023. Agentcoder: Multi-agent-based code generation with iterative testing and optimisation. arXiv preprint arXiv:2312.13010 .

Irugalbandara, C., Mahendra, A., Daynauth, R., Arachchige, T.K., Dantanarayana, J., Flautner, K., Tang, L., Kang, Y., Mars, J., 2024. Scaling down to scale up: A cost-benefit analysis of replacing openai's llm with open source slms in production, in: 2024 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS), IEEE. pp. 280–291.

Jiang, A.Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D.S., de las Casas, D., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., Lavaud, L.R., Lachaux, M.A., Stock, P., Scao, T.L., Lavril, T., Wang, T., Lacroix, T., Sayed, W.E., 2023. Mistral 7b. URL: https://arxiv.org/abs/2310.06825, arXiv:2310.06825.

Just, R., Jalali, D., Ernst, M.D., 2014. Defects4j: a database of existing faults to enable controlled testing studies for java programs, in: Proceedings of the 2014 International Symposium on Software Testing and Analysis, Association for Computing Machinery, New York, NY, USA. p. 437–440. URL: https://doi.org/10.1145/2610384.2628055, doi:10.1145/2610384.2628055.

Khojah, R., de Oliveira Neto, F.G., Mohamad, M., Leitner, P., 2025. The impact of prompt programming on function-level code generation. IEEE Transactions on Software Engineering .

Lei, Y., Liu, H., Xie, C., Liu, S., Yin, Z., Chen, C., Li, G., Torr, P., Wu, Z., 2024. Fairmindsim: Alignment of behavior, emotion, and belief in humans and llm agents amid ethical dilemmas. arXiv preprint arXiv:2410.10398 .

Li, G., Hammoud, H., Itani, H., Khizbullin, D., Ghanem, B., 2023. Camel: Communicative agents for" mind" exploration of large language model society. Advances in Neural Information Processing Systems 36, 51991–52008.

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

Liao, C.C., Liao, D., Gadiraju, S.S., 2025. AgentMaster: A multi-agent conversational framework using A2A and MCP protocols for multimodal information retrieval and analysis, in: Habernal, I., Schulam, P., Tiedemann, J. (Eds.), Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, Association for Computational Linguistics, Suzhou, China. pp. 52–72. URL: https://aclanthology.org/2025.emnlp-demos.5/, doi:10.18653/v1/2025.emnlp-demos.5.

Liu, X., Yu, H., Zhang, H., Xu, Y., Lei, X., Lai, H., Gu, Y., Ding, H., Men, K., Yang, K., Zhang, S., Deng, X., Zeng, A., Du, Z., Zhang, C., Shen, S., Zhang, T., Su, Y., Sun, H., Huang, M., Dong, Y., Tang, J., 2024a. Agentbench: Evaluating llms as agents, in: Kim, B., Yue, Y., Chaudhuri, S., Fragkiadaki, K., Khan, M., Sun, Y. (Eds.), International Conference on Learning Representations, pp. 52989–53046. URL: https://proceedings.iclr.cc/paper_files/paper/2024/file/e9df36b21ff4ee211a8b71ee8b7e9f57-Paper-Conference.pdf.

Liu, Z., Yao, W., Zhang, J., Yang, L., Liu, Z., Tan, J., Choubey, P.K., Lan, T., Wu, J., Wang, H., Heinecke, S., Xiong, C., Savarese, S., 2024b. AgentLite: A Lightweight Library for Building and Advancing Task-Oriented LLM Agent System. URL: http://arxiv.org/abs/2402.15538, doi:10.48550/arXiv.2402.15538. arXiv:2402.15538 [cs].

Nguyen, M.H., Phan Chau, T., Nguyen, P.X., Bui, N.D.Q., 2025. Agilecoder: Dynamic collaborative agents for software development based on agile methodology, in: 2025 IEEE/ACM Second International Conference on AI Foundation Models and Software Engineering (Forge), pp. 156–167. doi:10.1109/Forge66646.2025.00026.

Nijkamp, E., Pang, B., Hayashi, H., Tu, L., Wang, H., Zhou, Y., Savarese, S., Xiong, C., 2022. Codegen: An open large language model for code with multi-turn program synthesis, in: International Conference on Learning Representations. URL: https://api.semanticscholar.org/CorpusID:252668917.

OECD, 2024. Oecd principles on artificial intelligence (updated may 2024). https://www.oecd.org/en/topics/sub-issues/ai-principles.html. Updated international standard on trustworthy AI adopted in May 2024; includes values-based principles and recommendations for policymakers.

OpenAI, Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., et al., 2024. Gpt-4 technical report. URL: https://arxiv.org/abs/2303.08774, arXiv:2303.08774.

Osika, A., 2023. Gpt-engineer. URL: https://github.com/AntonOsika/gpt-engineer.

Phan, H.N., Nguyen, T.N., Nguyen, P.X., Bui, N.D.Q., 2024. HyperAgent: Generalist Software Engineering Agents to Solve Coding Tasks at Scale. URL: https://arxiv.org/abs/2409.16299, doi:10.48550/ARXIV.2409.16299.

Qi, P., 2025. Why you should stop using hotpotqa for ai agents evaluation in 2025. URL: https://qipeng.me/blog/stop-using-hotpotqa/.

Qian, C., Liu, W., Liu, H., Chen, N., Dang, Y., Li, J., Yang, C., Chen, W., Su, Y., Cong, X., Xu, J., Li, D., Liu, Z., Sun, M., 2024. ChatDev: Communicative agents for software development, in: Ku, L.W., Martins, A., Srikumar, V. (Eds.), Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Association for Computational Linguistics, Bangkok, Thailand. pp. 15174–15186. URL: https://aclanthology.org/2024.acl-long.810/, doi:10.18653/v1/2024.acl-long.810.

Rahman, M., Khatoonabadi, S., Abdellatif, A., Shihab, E., 2024. Automatic detection of llm-generated code: A case study of claude 3 haiku. URL: https://arxiv.org/abs/2409.01382, arXiv:2409.01382.

Ranjan, R., Gupta, S., Singh, S.N., 2025. Fairness in agentic ai: A unified framework for ethical and equitable multi-agent system. arXiv preprint arXiv:2502.07254 .

Scao, T.L., Fan, A., Akiki, C., Pavlick, E., Ilić, S., Hesslow, D., Castagné, R., et. al, 2023. Bloom: A 176b-parameter open-access multilingual language model. URL: https://arxiv.org/abs/2211.05100, arXiv:2211.05100.

Schuett, J., Reuel, A.K., Carlier, A., 2025. How to design an ai ethics board. AI and Ethics 5, 863–881.

Significant Gravitas, 2023. AutoGPT. URL: https://github.com/Significant-Gravitas/AutoGPT.

Stone, P., Veloso, M., 2000. Multiagent systems: A survey from a machine learning perspective. Autonomous Robots 8, 345–383.

Takerngsaksiri, W., Pasuksmit, J., Thongtanunam, P., Tantithamthavorn, C., Zhang, R., Jiang, F., Li, J., Cook, E., Chen, K., Wu, M., 2025. Human-in-the-loop software development agents, in: 2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), pp. 342–352. doi:10.1109/ICSE-SEIP66354.2025.00036.

Tamang, S., Bora, D.J., 2025. Enforcement agents: Enhancing accountability and resilience in multi-agent ai frameworks. arXiv preprint arXiv:2504.04070 .

Team, G., Anil, R., Borgeaud, S., Alayrac, J.B., Yu, J., Soricut, R., et al., 2025. Gemini: A family of highly capable multimodal models. URL: https://arxiv.org/abs/2312.11805, arXiv:2312.11805.

Tennant, E., Hailes, S., Musolesi, M., 2025. Moral alignment for LLM agents, in: The Thirteenth International Conference on Learning Representations.

Wang, H., Zhang, A., Duy Tai, N., Sun, J., Chua, T.S., et al., 2024. Ali-agent: Assessing llms' alignment with human values via agent-based evaluation. Advances in Neural Information Processing Systems 37, 99040–99088.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q.V., Zhou, D., et al., 2022. Chain-of-thought prompting elicits reasoning in large language models. Advances in neural information processing systems 35, 24824–24837.

Wu, N., Gong, M., Shou, L., Liang, S., Jiang, D., 2023. Large language models are diverse role-players for summarization evaluation, in: CCF international conference on natural language processing and Chinese computing, Springer. pp. 695–707.

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., et al., . Autogen: Enabling next-gen llm applications via multi-agent conversations, in: First Conference on Language Modeling.

Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W., Salakhutdinov, R., Manning, C.D., 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering, in: Riloff, E., Chiang, D., Hockenmaier, J., Tsujii, J. (Eds.), Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, Brussels, Belgium. pp. 2369–2380. URL: https://aclanthology.org/D18-1259/, doi:10.18653/v1/D18-1259.

Yao, S., Chen, H., Yang, J., Narasimhan, K., 2022a. Webshop: towards scalable real-world web interaction with grounded language agents, in: Proceedings of the 36th International Conference on Neural Information Processing Systems, Curran Associates Inc., Red Hook, NY, USA.

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K.R., Cao, Y., 2022b. React: Synergizing reasoning and acting in language models, in: The eleventh international conference on learning representations.

Zhang, J., Lan, T., Zhu, M., Liu, Z., Hoang, T., Kokane, S., Yao, W., Tan, J., Liu, Z., Feng, Y., Niebles, J.C., Heinecke, S., Wang, H., Savarese, S., Xiong, C., 2025. xLAM: A family of large action models to empower AI agent systems, in: Chiruzzo, L., Ritter, A., Wang, L. (Eds.), Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), Association for Computational Linguistics, Albuquerque, New Mexico. pp. 11583–11597. URL: https://aclanthology.org/2025.naacl-long.578/, doi:10.18653/v1/2025.naacl-long.578.

Zheng, Q., Xia, X., Zou, X., Dong, Y., Wang, S., Xue, Y., Shen, L., Wang, Z., Wang, A., Li, Y., Su, T., Yang, Z., Tang, J., 2023. Codegeex: A pre-trained model for code generation with multilingual benchmarking on humaneval-x, in: Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Association for Computing Machinery, New York, NY, USA. p. 5673–5684. URL: https://doi.org/10.1145/3580305.3599790, doi:10.1145/3580305.3599790.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6507135